

**OPTIMALISASI PEMILIHAN URUTAN SYUTING *SCENE* FILM
MENGUNAKAN ALGORITMA *PARTICLE SWARM OPTIMIZATION*
(Studi Kasus: Film ”Ketika Adzan Sudah Tidak Lagi Berkumandang”)**

DRAF SKRIPSI



**Langgeng Prasadewo Sukma Adi Winoto Basla
NIM. 1807065014**

**PROGRAM STUDI S1 MATEMATIKA
JURUSAN MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS MULAWARMAN
SAMARINDA
2025**

**OPTIMALISASI PEMILIHAN URUTAN SYUTING *SCENE* FILM
MENGUNAKAN ALGORITMA *PARTICLE SWARM OPTIMIZATION*
(Studi Kasus: Film "Ketika Adzan Sudah Tidak Lagi Berkumandang")**

DRAF SKRIPSI

**Diajukan kepada
Jurusan Matematika Fakultas Matematika dan Ilmu Pengetahuan Alam
Universitas Mulawarman untuk memenuhi sebagai persyaratan
memperoleh gelar Sarjana Matematika**

**Oleh:
Langgeng Prassadewo Sukma Adi Winoto Basla
NIM. 1807065014**

**PROGRAM STUDI S1 MATEMATIKA
JURUSAN MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS MULAWARMAN
SAMARINDA
2025**

HALAMAN PENGESAHAN

PRAKATA

Bismillahirrahmanirrahim

Assalamuálaikum Warahmatullahi Wabarakatuh

Segala puji dan syukur penulis panjatkan kepada Allah SWT., atas segala limpahan rahmat, karunia dan hidayah-Nya khususnya kepada penulis, sehingga penulis dapat menulis draf skripsi ini yang berjudul ”Optimalisasi Pemilihan Urutan Syuting *Scene* Film Menggunakan Algoritma *Particle Swarm Optimization* (Studi Kasus: Film ”Ketika Adzan Sudah Tidak Lagi Berkumandang”)”.

Pada kesempatan ini, penulis mengucapkan terima kasih atas bantuan dan bimbingan dalam penyusunan draf skripsi ini, terutama kepada yang terhormat:

1. Ibu Dr. Dra. Hj. Ratna Kusuma, M.Si., selalu Dekan Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Mulawarman, Samarinda.
2. Bapak Wasono, S.Si., M.Si., selaku Dosen Pembimbing I dan Bapak Fidia Deny Tisna Amijaya, S.Si., M.Si., selaku Dosen Pembimbing II.
3. Bapak Dr. Syaripuddin, S.Si., M.Si., selaku Dosen Penguji I dan Bapak Andri Azmul Fauzi, S.Si., M.Si., selaku Dosen Penguji II.
4. Ketiga orang tua tercinta dan tersayang, Bapak Harwanoto, Ibu Nor Handayani, dan Ibu Anik Suryani, serta adik saya, Liliana Devita Sari yang selalu memberikan doa, dukungan, semangat, dan kasih sayang selama proses pembuatan skripsi ini.
5. Bang David Richard selaku Sutradara sekaligus Produser dan Bang Fayed selaku Asisten Sutradara I yang telah mengizinkan penulis menggunakan karya Film ”Ketika Adzan Sudah Tidak Lagi Berkumandang” dalam tugas akhir skripsi.
6. Dana, Mahatir, dan Hafif serta seluruh kru produksi film ”Ketika Adzan Sudah Tidak Lagi Berkumandang”.
7. Tim dosen serta staf Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Mulawarman, terutama yang berada di Program Studi S1 Matematika.
8. Teman-teman Matematika angkatan 2018 yang telah memberikan dukungan, semangat, dan motivasi hingga selesainya proses penyusunan draf skripsi ini.

9. Semua pihak yang tidak dapat disebutkan satu per satu oleh penulis. Penulis mengucapkan banyak terima kasih kepada semua pihak yang terlibat.

Akhir kata, penulis bersama semua pihak yang turut membantu dalam penulisan draf skripsi ini berharap bahwa semoga draf skripsi ini dapat memberikan manfaat serta memberikan tambahan pengetahuan bagi pembacanya.

Wassalamuálaikum Warahmatullahi Wabarakatuh

Samarinda, 07 Maret 2025

Langgeng Prasadewo Sukma Adi Winoto Basla

--	--

DAFTAR ISI	
	Halaman
HALAMAN JUDUL	i
LEMBAR PENGESAHAN	ii
PRAKATA	ii
DAFTAR ISI	iv
DAFTAR GAMBAR	vii
DAFTAR TABEL	viii
DAFTAR SIMBOL	x
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Batasan Masalah	4
1.3 Rumusan Masalah	4
1.4 Tujuan Penelitian	4
1.5 Manfaat Penelitian	5
BAB II TINJAUAN PUSTAKA	6
2.1 Matematika Diskret	6
2.2 Teori Graf	6
2.2.1 Definisi Graf	7
2.2.2 Jalan, Lintasan, dan Sirkuit	8
2.2.3 Graf Berbobot	8
2.2.4 Representasi Graf Tak Berarah Dalam Matriks	9
2.2.4.1 Matriks Ketetanggaan	9
2.2.4.2 Matriks Biner	11
2.3 Optimasi	11
2.4 Konsep <i>Traveling Salesman Problem</i> (TSP)	12
2.5 Metode Heuristik	13
2.6 Algoritma <i>Particle Swarm Optimization</i> (PSO)	14

--	--

2.6.1	Definisi Algoritma <i>Particle Swarm Optimization</i>	15
2.6.2	Parameter Algoritma <i>Particle Swarm Optimization</i>	17
2.6.3	Kecepatan Partikel (<i>Velocity</i>)	19
2.6.4	<i>Personal Best</i>	20
2.6.5	<i>Global Best</i>	20
2.6.6	Pembaruan Posisi dan <i>Velocity</i>	20
2.6.7	Tahapan Algoritma <i>Particle Swarm Optimization</i>	21
2.7	Kriteria Pemberhentian Algoritma <i>Particle Swarm Optimization</i>	23
2.8	Metode <i>Min-Max Normalization</i>	24
2.9	Film "Ketika Adzan Sudah Tidak Lagi Berkumandang"	24
BAB III METODE PENELITIAN		26
3.1	Waktu dan Tempat Penelitian	26
3.2	Variabel Penelitian	26
3.3	Teknik Pengumpulan Data	26
3.4	Populasi dan Sampel Penelitian	26
3.5	Teknik Sampling	27
3.6	Teknik Analisis Data.....	27
3.7	Kerangka Penelitian	29
BAB IV HASIL DAN PEMBAHASAN		30
4.1	Model Graf dan Model Matematika Pemilihan Urutan <i>Scene</i> Syuting Film "Ketika Adzan Sudah Tidak Lagi Berkumandang"	30
4.1.1	Model Graf Pemilihan Urutan <i>Scene</i> Syuting Film "Ketika Adzan Sudah Tidak Lagi Berkumandang"	30
4.1.2	Model Matematika Konsep <i>Traveling Salesman Problem</i> untuk Pemilihan Urutan Syuting <i>Scene</i>	39
4.2	Penerapan Algoritma <i>Particle Swarm Optimization</i> (PSO) untuk Optimalisasi Pemilihan Urutan <i>Scene</i> Syuting Film "Ketika Adzan Sudah Tidak Lagi Berkumandang"	41
4.2.1	Proses Normalisasi Data	41

--	--

4.2.2	Proses Algoritma PSO	42
4.2.3	Analisis Hasil Pengujian Modifikasi Nilai Parameter PSO	58
4.2.4	Analisis Hasil Perhitungan Algoritma PSO	61
4.3	Hasil Optimalisasi Pemilihan Urutan <i>Scene</i> Syuting Film "Ketika Adzan Sudah Tidak Lagi Berkumandang" menggunakan Algoritma <i>Particle Swarm Optimization</i> (PSO)	68
BAB V	PENUTUP	70
5.1	Kesimpulan.....	70
5.2	Saran.....	71
	DAFTAR PUSTAKA	72
	LAMPIRAN.....	76

DAFTAR GAMBAR

	Halaman
Gambar 2.1 Graf Representasi "Tujuh Jembatan Königsberg"	7
Gambar 2.2 Graf G	8
Gambar 2.3 Graf Contoh 2.4	10
Gambar 2.4 Graf Contoh 2.5	10
Gambar 3.1 <i>Flowchart</i> Kerangka Penelitian	29
Gambar 4.1 Peta Lokasi Syuting <i>Scene</i> Film "Ketika Adzan Sudah Tidak Lagi Berkumandang"	32
Gambar 4.2 Graf Lengkap Representasi Permasalahan	38
Gambar 4.3 Grafik Hasil Pengujian Ukuran <i>Swarm</i> (N)	59
Gambar 4.4 Grafik Hasil Pengujian Iterasi Maksimum (n)	60
Gambar 4.5 Grafik Hasil Pengujian Iterasi Maksimum (n)	61
Gambar 4.6 Grafik Perubahan Nilai Fungsi <i>Fitness</i> G_{best} pada Setiap Iterasi	67
Gambar 4.7 Grafik Perubahan Total Biaya Syuting (Rp) G_{best} pada Setiap Iterasi	68

DAFTAR TABEL

	Halaman
Tabel 3.1 Variabel Penelitian	26
Tabel 4.1 Data <i>Scene</i>	31
Tabel 4.2 Data Biaya <i>Talent</i> dalam Satu <i>Scene</i>	32
Tabel 4.3 Data Total Biaya <i>Talent</i> tiap <i>Scene</i>	33
Tabel 4.4 Jarak Antar Lokasi tiap <i>Scene</i> dalam Satuan Kilometer	35
Tabel 4.5 Data Biaya Transportasi Antar <i>Scene</i> dalam Rupiah	36
Tabel 4.6 Data Biaya Antar <i>Scene</i> dalam Rupiah.....	36
Tabel 4.7 Contoh Matriks $[a_{j,k}]$	40
Tabel 4.8 Hasil Perhitungan Normalisasi Biaya Antar <i>Scene</i>	41
Tabel 4.9 <i>Velocity</i> Awal	43
Tabel 4.10 Posisi Awal	44
Tabel 4.11 Rute Awal.....	44
Tabel 4.12 Total Biaya Syuting dan Nilai <i>Fitness</i> tiap Partikel.....	46
Tabel 4.13 P_{best} Iterasi ke-0	47
Tabel 4.14 G_{best} Iterasi ke-0	48
Tabel 4.15 <i>Velocity</i> pada Iterasi ke-1	51
Tabel 4.16 Posisi pada Iterasi ke-1.....	52
Tabel 4.17 Rute pada Iterasi ke-1	53
Tabel 4.18 Total Biaya Syuting dan Nilai <i>Fitness</i> tiap Partikel.....	54
Tabel 4.19 P_{best} Iterasi ke-1	56
Tabel 4.20 G_{best} Iterasi ke-1	57
Tabel 4.21 G_{best} pada Setiap Iterasi	62
Tabel 4.22 Rute G_{best} pada Setiap Iterasi.....	63
Tabel 4.23 Total Biaya Syuting G_{best} pada Setiap Iterasi.....	64
Tabel 4.24 Urutan <i>Scene</i> dan Total Biaya G_{best} pada Setiap Iterasi.....	65

DAFTAR LAMPIRAN

	Halaman
Lampiran 1 Contoh lampiran	77
Lampiran 2 Daftar Total Biaya <i>Talent</i>	78
Lampiran 3 Contoh lampiran	79
Lampiran 4 Contoh lampiran	80
Lampiran 5 Contoh lampiran	81
Lampiran 6 Kecepatan Awal	82
Lampiran 7 Posisi Awal	83
Lampiran 8 Rute Awal	84
Lampiran 9 Total Biaya Syuting dan Nilai <i>Fitness</i> tiap Partikel	85
Lampiran 10 <i>Velocity</i> Partikel Iterasi $i = 1$	86
Lampiran 11 Posisi Partikel Iterasi $i = 1$	87
Lampiran 12 Rute pada Partikel Iterasi $i = 1$	88
Lampiran 13 Total Biaya Syuting dan Nilai <i>Fitness</i> Iterasi $i = 1$	89
Lampiran 14 P_{best} Iterasi $i = 1$	90
Lampiran 15 G_{best} Seluruh Iterasi	91
Lampiran 16 Rute G_{best} Pada Setiap Iterasi	92
Total Biaya Syuting G_{best} pada Setiap Iterasi	93

DAFTAR SIMBOL

Simbol	Arti
G	Graf G
$V(G)$	Himpunan simpul-simpul dalam graf G
$E(G)$	Himpunan sisi-sisi dalam graf G
$d(v)$	Derajat simpul v
N	Ukuran <i>swarm</i>
j	Indeks dari partikel dalam <i>swarm</i>
n	Ukuran iterasi maksimum
i	Indeks dari iterasi sekarang
d	Ukuran dari dimensi partikel
$\mathbf{X}_j^{(i)}$	Vektor posisi partikel j pada iterasi ke- i
$x_{j,d}^{(i)}$	Posisi partikel j pada dimensi ke- d saat iterasi ke- i
$\mathbf{V}_j^{(i)}$	Vektor kecepatan partikel j pada iterasi ke- i
$v_{j,d}^{(i)}$	Kecepatan partikel j pada dimensi d saat iterasi ke- i
$r_1^{(i)}$	Bilangan acak pertama berdistribusi <i>uniform</i> saat iterasi ke- i
$e_2^{(i)}$	Bilangan acak kedua berdistribusi <i>unifrom</i> saat iterasi ke- i
c_1	Parameter kecerdasan kognitif partikel
c_2	Parameter kecerdasan sosial partikel
$\omega^{(i)}$	Bobot inersia saat iterasi ke- i
$\mathbf{P}_{best,j}^{(i)}$	Vektor posisi terbaik partikel $\mathbf{X}_j$ saat iterasi ke- i
$P_{j,d}^{(j)}$	Posisi terbaik partikel j pada dimensi ke- d saat iterasi ke- i
$G_{best}^{(i)}$	Posisi terbaik dari semua partikel saat iterasi ke- i
$F(\mathbf{X}_j^{(i)})$	Nilai fungsi <i>fitness</i> hasil rute pada partikel j saat iterasi ke- i
$f(\mathbf{X}_j^{(i)})$	Total biaya yang dikeluarkan dengan hasil rute pada partikel j saat iterasi ke- i

BAB I

PENDAHULUAN

1.1 Latar Belakang

Syuting atau kegiatan pengambilan gambar dalam proses produksi film, sering kali memiliki susunan urutan adegan-adegan (*scenes*) yang tidak diambil secara teratur seperti urutan saat film tersebut ditayangkan (Qin dkk., 2016). Salah satu faktor utama yang dipertimbangkan dalam pemilihan urutan *scenes* adalah lokasi dan biaya. Besar biaya transportasi yang dikeluarkan berbanding lurus dengan semakin banyak *scenes* dengan lokasi syuting yang berbeda. Jumlah biaya transportasi yang bertambah perlu diwaspadai agar tidak menyebabkan permasalahan pada biaya operasional produksi film.

Tahap produksi sebuah film merupakan proses eksekusi semua hal yang telah dipersiapkan pada tahap pra-produksi, termasuk kegiatan syuting untuk setiap *scene* (Haren, 2020). Pelaksanaan tahap produksi mengharuskan seluruh kru dan pemain (*talent*) produksi film berpindah-pindah lokasi sesuai lokasi syuting *scene*. Akibatnya terdapat biaya tambahan berupa biaya transportasi yang dikeluarkan saat melakukan perpindahan lokasi syuting. Oleh karena itu, pemilihan urutan *scene* syuting mempertimbangkan faktor lokasi dengan memilih total biaya pelaksanaan syuting dan biaya perpindahan antar lokasi *scene* yang minimum. Permasalahan pemilihan urutan *scene* dapat dipandang sebagai permasalahan rute terpendek dengan pengoptimalan biaya perpindahan lokasi *scene* sebagai objek bahasan. Matematika diskret dapat diterapkan terhadap optimasi biaya dalam permasalahan rute terpendek (Qin dkk., 2016).

Matematika diskret merupakan salah satu cabang matematika dengan ruang lingkup kajian berupa objek-objek permasalahan secara diskret (Nasir dkk., 2022). Suatu objek dapat dikatakan objek secara diskret jika objek tersebut terdiri dari sejumlah berhingga anggota dan tiap anggotanya berbeda atau tidak terhubung. Salah satu penerapan dari matematika diskret adalah sistem penyimpanan informasi pada komputer digital yang disimpan dalam bentuk diskret. Cakupan dalam matematika diskret mempelajari diantaranya himpunan, relasi dan fungsi, induksi matematika,

kombinatorial, dan teori graf (Yurinanda dan Rozi, 2023).

Teori graf termasuk ke dalam cakupan matematika diskret. Konsep teori graf pertama kali diperkenalkan oleh Leonhard Euler saat memodelkan solusi untuk melewati ketujuh jembatan Königsberg di Rusia dengan masing-masing jembatan hanya boleh dilewati tepat satu kali dan kembali ke tempat semula perjalanan dimulai. Sebuah graf G merupakan pasangan terurut himpunan berhingga yang tidak kosong dari simpul (*vertices*) dan himpunan pasangan dari simpul-simpul secara tidak terurut. Implementasi teori graf sering digunakan untuk memecahkan permasalahan optimasi, seperti optimasi pencarian rute terpendek pada konsep *traveling salesman problem* (TSP) (Setiawati dkk., 2023).

Optimasi atau optimalisasi merupakan suatu prinsip pencarian solusi layak yang bersifat tepat, efektif, dan efisien (optimum) (Atiqoh, 2020). Pengertian optimasi juga dapat diartikan sebagai tindakan tersistematis yang memaksimalkan sumber daya yang tersedia dengan tujuan menemukan solusi optimal. Penerapan optimasi sering diterapkan dalam banyak permasalahan yang memiliki keterbatasan sumber daya, seperti permasalahan pencarian rute terpendek pada konsep *traveling salesman problem* (TSP).

Traveling Salesman Problem (TSP) merupakan konsep rute terpendek pada seorang penjual (*salesman*) yang harus mengunjungi semua kota dan harus kembali ke kota awal serta satu kota hanya boleh dikunjungi satu kali (Rahimi dkk., 2023). Permasalahan optimasi TSP termasuk ke dalam persoalan optimasi kombinatorial dengan memanfaatkan graf sebagai representasi dari model permasalahan. Representasi masalah TSP umumnya dimodelkan ke dalam graf dengan simpul (*vertices*) sebagai lokasi dan sisi (*edge*) sebagai bobot atau biaya yang akan dioptimalkan. Permasalahan TSP dapat diselesaikan menggunakan penerapan algoritma *Particle Swarm Optimization* (PSO).

Particle Swarm Optimization (PSO) merupakan algoritma optimasi stokasi terinspirasi dari perilaku sosial burung atau ikan yang berkelompok dalam mencari makan (Azhari dkk., 2018). Algoritma *Particle Swarm Optimization* (PSO) ini termasuk ke dalam algoritma heuristik berbasis populasi. Pencarian solusi optimal menggunakan perilaku partikel dalam populasi itu sendiri. Setiap partikel berperilaku sebagai pencari lingkungan dengan kecerdasan untuk saling berkontribusi dan berkomunikasi

dalam mencari letak objek bahasan.

Penelitian sebelumnya tentang algoritma *Particle Swarm Optimization* (PSO) telah dilakukan oleh beberapa peneliti terdahulu, antara lain 2018 dalam penelitian yang membahas tentang penentuan rute penjemputan angkutan sekolah di MI Salafiyah Kasim. Dengan 2 kloter pengantaran, 150 partikel, 35 iterasi, dan 5 kali percobaan yang dihitung menghasilkan 3 percobaan dengan rute yang lebih optimal dari rute aktual. Natalia dkk. (2019) dalam penelitian yang membahas tentang penentuan jarak penjemputan penumpang optimal di CV. Eira Saudara. Dengan 8 titik lokasi penjemputan dan 90 partikel yang dihitung menghasilkan rute optimal dengan total jarak 59,2 Km. Dalyono dkk. (2017) dalam penelitian yang membahas tentang penentuan rute kunjungan tempat pariwisata di Kota Bandung. Dengan 2 set lokasi pariwisata dihitung dengan 2 kali pengujian, yaitu pengujian pertama dilakukan dengan mengubah 5 parameter pada set pertama dan pengujian kedua membandingkan waktu tempuh luaran antara algoritma *Particle Swarm Optimization* (PSO) dan algoritma *Artificial Immune System* (AIS).

Pada penelitian ini, permasalahan pemilihan urutan *scene* syuting dapat dipandang ke dalam permasalahan optimasi dengan konsep TSP. Sehingga penulis akan menggunakan algoritma *Particle Swarm Optimization* (PSO) untuk mencari solusi dalam pemilihan urutan *scene* syuting yang optimal pada film "Ketika Adzan Sudah Tidak Lagi Berkumandang". Film "Ketika Adzan Sudah Tidak Lagi Berkumandang" merupakan film berkategori film layar lebar dengan genre horor. Film ini diproduksi oleh komunitas East Borneo Film (EBF). Proses produksi film ini menggunakan tiga lokasi utama yang berbeda, yaitu Waduk Tenggarong, Desa Loa Raya, dan Desa Kedang Ipil. Setiap lokasi utama terdapat beberapa titik lokasi yang menjadi lokasi syuting *scene* yang berbeda. Hal ini memerlukan strategi dalam pemilihan urutan *scene* syuting guna menjaga peningkatan biaya transportasi agar biaya produksi tetap minimum.

Berdasarkan uraian di atas, penulis akan membahas mengenai masalah pemilihan urutan *scene* syuting dengan menggunakan algoritma *Particle Swarm Optimization* (PSO) dengan judul "Optimalisasi Pemilihan Urutan Syuting *Scene* Film Menggunakan Algoritma *Particle Swarm Optimization* (Studi Kasus: Film "Ketika Adzan Sudah Tidak Lagi Berkumandang")".

1.2 Batasan Masalah

Batasan masalah yang digunakan pada penelitian ini untuk mencapai tujuan yang diharapkan adalah:

1. Data yang digunakan adalah data dari *Master Breakdown* film "Ketika Adzan Sudah Tidak Lagi Berkumandang".
2. Data lokasi yang digunakan adalah data lokasi syuting dari setiap *scene*.
3. Asumsi yang digunakan antara lain:
 - (a) biaya perpindahan antar setiap *scene* merupakan total biaya transportasi ditambahkan biaya talent yang bermain pada *scene* tujuan,
 - (b) biaya transportasi merupakan jumlah konsumsi bahan bakar dalam Rupiah,
 - (c) kendaraan berjumlah tiga, yaitu satu kendaraan untuk seluruh pemain (*talent*) dan dua kendaraan untuk perlengkapan produksi,
 - (d) harga bahan bakar yang digunakan adalah harga bahan bakar jenis Pertalite per bulan Juni 2024, yaitu seharga Rp. 10.000, —.
4. Graf yang digunakan dalam penelitian ini adalah graf tak berarah.

1.3 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah pada penelitian ini adalah:

1. Bagaimana model graf dan model matematika pemilihan urutan *scene* syuting film "Ketika Adzan Sudah Tidak Lagi Berkumandang"?
2. Bagaimana penerapan algoritma *Particle Swarm Optimization* (PSO) untuk optimalisasi pemilihan urutan *scene* syuting film "Ketika Adzan Sudah Tidak Lagi Berkumandang"?
3. Bagaimana hasil optimalisasi pemilihan urutan *scene* syuting film "Ketika Adzan Sudah Tidak Lagi Berkumandang" menggunakan algoritma *particle swarm optimization* (PSO)?

1.4 Tujuan Penelitian

Berdasarkan rumusan masalah yang telah diuraikan, maka tujuan yang diharapkan dapat dicapai pada penelitian ini adalah:

1. Mengetahui model graf *scene* film "Ketika Adzan Sudah Tidak Lagi Berkumandang".

2. Mengetahui penerapan algoritma *Particle Swarm Optimization* (PSO) untuk optimalisasi pemilihan urutan *scene* syuting film "Ketika Adzan Sudah Tidak Lagi Berkumandang".
3. Mengetahui hasil optimalisasi pemilihan urutan *scene* syuting film "Ketika Adzan Sudah Tidak Lagi Berkumandang" menggunakan algoritma *particle swarm optimization* (PSO).

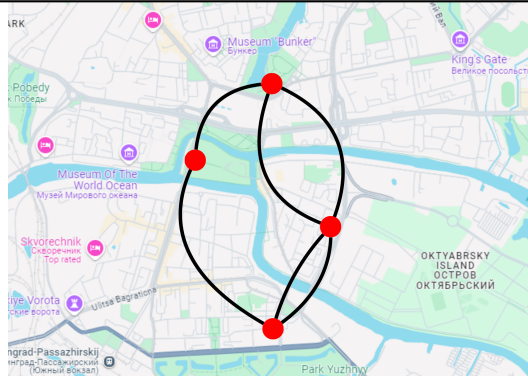
1.5 Manfaat Penelitian

Manfaat yang diharapkan pada penelitian ini adalah:

1. Bagi penulis dan pembaca, dapat memberikan pemahaman lebih luas mengenai penerapan algoritma *Particle Swarm Optimization* (PSO).
2. Bagi universitas, dapat memberikan kontribusi pengembangan ilmu pengetahuan serta dapat menjadi bahan referensi bagi mahasiswa lain.
3. Bagi pihak terkait, dapat memberikan pemahaman mengenai penerapan algoritma *Particle Swarm Optimization* (PSO) dan dapat digunakan untuk membantu pemilihan urutan *scene* syuting film yang akan mendatang.

--

BAB II TINJAUAN PUSTAKA Pada bab ini akan disajikan mengenai pengertian dan penjelasan materi yang berkaitan dengan matematika diskret, teori graf, konsep <i>Traveling Salesman Problem</i> (TSP), metode heuristik, algoritma <i>Particle Swarm Optimization</i> (PSO), kriteria pemberhentian algoritma, dan Film "Ketika Adzan Sudah Tidak Lagi Berkumandang". 2.1 Matematika Diskret Matematika diskret merupakan salah satu dari cabang matematika dengan ruang lingkup kajian mengenai segala objek-objek permasalahan yang bersifat diskret. Objek-objek yang bersifat diskret memiliki konsep bahwa setiap objek tersebut adalah objek yang terpisah dan berbeda dari yang lain (tidak kontinu). Objek dalam domain matematika seperti bilangan bulat, logika, dan graf dapat diamati sebagai entitas diskret. Beberapa topik utama yang termasuk ke dalam matematika diskret seperti teori himpunan, permutasi, teori kombinatorial, logika, dan teori graf (Sari dkk., 2024). 2.2 Teori Graf Teori graf merupakan kajian matematika dalam lingkup matematika diskrit yang relatif baru . Kajian mengenai teori graf pertama kali dilakukan oleh matematikawan terkemuka bernama Leonhard Euler pada Tahun 1735. Permasalahan mengenai "Tujuh Jembatan Königsberg" menjadi inspirasi awal mula teori graf dikaji oleh Euler. Euler merepresentasikan jembatan sebagai sisi dan tempat yang dihubungkan oleh jembatan sebagai simpul, sehingga tercipta graf sederhana mengenai "Tujuh Jembatan Königsberg" (Levin, 2021).	
---	--



Gambar 2.1 Graf Representasi "Tujuh Jembatan Königsberg"

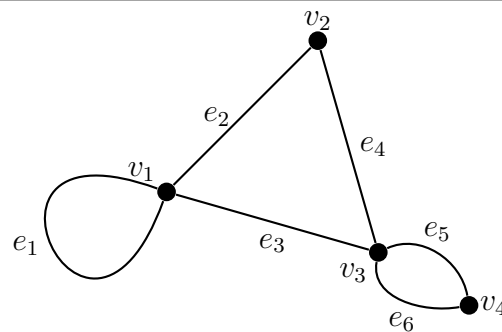
2.2.1 Definisi Graf

Terminologi dari sebuah graf dapat memiliki makna yang berbeda akibat dari luasnya aplikasi graf pada berbagai bidang. Sebuah simpul dari graf pada studi kasus yang berbeda mungkin menyatakan objek yang berbeda. Oleh karena itu secara kasar, graf dapat dipandang sebagai suatu diagram yang memuat sejumlah informasi tertentu dari sebuah studi kasus.

Suatu graf dinotasikan dengan $G = (V, E)$ dalam matematika memiliki definisi sebagai berikut.

Definisi 2.1 Suatu graf $G = (V, E)$ terdiri 2 himpunan pasangan berurut yang berhingga, yaitu himpunan tidak kosong dari titik-titik disebut simpul atau "vertices" (dinotasikan $V(G)$) dan himpunan garis-garis dua elemen subset dari $V(G)$ disebut sisi atau "edges" (dinotasikan $E(G)$).

Setiap sisi di dalam graf G berhubungan dengan satu atau dua simpul. Sisi yang terhubung hanya dengan satu simpul disebut *loop*. Simpul di dalam graf G dapat terhubung dengan lebih dari satu sisi yang berbeda. Dua sisi berbeda yang terhubung dengan satu simpul sama disebut sebagai garis atau sisi paralel. Sisi yang terhubung dengan dua simpul berbeda menyebabkan kedua simpul tersebut menjadi terhubung (*adjacent*) (Jek, 2016).



Gambar 2.2 Graf G

2.2.2 Jalan, Lintasan, dan Sirkuit

Simpul-simpul di dalam graf G dapat dihubungkan oleh lebih dari satu sisi. Jumlah sisi yang terhubung dengan suatu simpul pada graf disebut derajat simpul sesuai dalam definisi berikut.

Definisi 2.2 Misalkan v merupakan simpul dalam suatu graf G . Derajat simpul v (dinotasikan $d(v)$) adalah jumlah sisi yang terhubung dengan simpul v dan sisi suatu loop dihitung dua kali. Derajat total G didapat dengan menjumlahkan derajat semua simpul dalam G .

Derajat suatu simpul dalam graf berarah dapat dinyatakan sebagai jumlahan sisi berarah masuk ke simpul dan jumlahan sisi berarah keluar dari simpul (Nurdiyanto dan Susanti, 2019).

Barisan yang diawali dari simpul awal dan diakhiri pada simpul akhir serta terdapat sisi-sisi dan simpul-simpul secara selang-seling disebut jalan (*walk*). Jalan sederhana dengan panjang n dari v_0 sampai v_n dituliskan sebagai berikut : $v_0 e_1 v_1 \dots v_{n-1} e_n v_n$. Jalan dapat dimulai dari simpul awal sampai dengan simpul akhir yang semua sisinya berbeda akan membuat sebuah lintasan (*path*). Lintasan dengan simpul awal dan simpul akhir yang sama dapat disebut sebagai sirkuit (*circuit*) (Rahayuningsih, 2018).

2.2.3 Graf Berbobot

Suatu graf berbobot (*Weighted Graph*) merupakan konsep dalam teori graf dengan memberikan bobot atau nilai bertipe numerik terhadap sisi yang terhubung dengan dua simpul pada suatu graf. Bobot ini merepresentasikan suatu atribut tertentu yang terkait dengan relasi antara simpul-simpul sesuai dengan informasi peman-

faatan graf tersebut. Salah satu pemanfaatan graf berbobot dapat digunakan untuk pencarian rute terpendek dengan bobot yang mungkin merepresentasikan jarak, biaya transportasi, atau waktu (Pratama dan Darmawan, 2022).

2.2.4 Representasi Graf Tak Berarah Dalam Matriks

Selaras dengan terminologi graf yang memiliki banyak makna bergantung pada fungsi suatu graf dalam aplikasinya, Suatu graf tak berarah dapat dipetakan menjadi sebuah matriks untuk merepresentasikan hubungan antar elemen-elemen dalam graf tersebut. Keuntungan dari merepresentasikan graf dalam matriks dapat mempermudah perhitungan yang diperlukan. Namun, keterbatasan matriks untuk mencakup keseluruhan informasi pada suatu graf menjadi kesulitan utama. Terdapat beberapa matriks untuk menyatakan suatu graf tak berarah, diantaranya matriks ketetanggaan dan matriks ketetanggaan berbobot (Chen dkk., 2020).

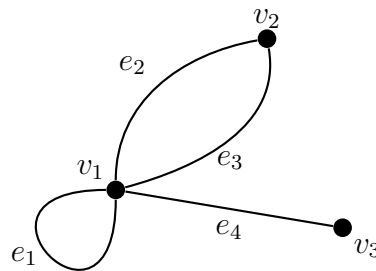
2.2.4.1 Matriks Ketetanggaan

Matriks ketetanggaan (*adjacency matrix*) merupakan matriks yang menyatakan hubungan jumlah sisi dan simpul dari suatu graf yang didefinisikan sebagai berikut.

Definisi 2.3 Misalkan G adalah graf tak berarah dengan simpul-simpul berhingga (n). Matriks ketetanggaan yang sesuai dengan graf G adalah matriks $A = (a_{ij})$; $i, j = 1, 2, \dots, n$ dengan a_{ij} merepresentasikan jumlah sisi yang menghubungkan antar simpul yang terhubung.

Perhatikan pada graf tak berarah bahwa jumlah sisi yang menghubungkan simpul v_i ke simpul v_j selalu sama dengan jumlah sisi yang menghubungkan simpul v_j ke simpul v_i . Akibatnya, matriks ketetanggaan yang merepresentasikan graf tak berarah selalu merupakan matriks yang simetris (Jek, 2016).

Contoh 2.4 Diberikan suatu graf G tak berarah sebagai $G = (\{v_1, v_2, v_3\}, \{(v_1, v_1), (v_1, v_2), (v_1, v_2), (v_1, v_3)\})$. Nyatakan graf G ke dalam bentuk matriks!



Gambar 2.3 Graf Contoh 2.4

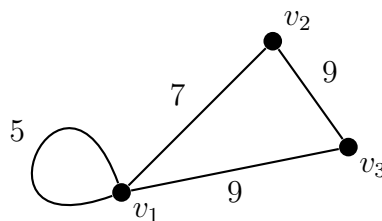
Penyelesaian:

Matriks ketetanggaan yang merepresentasikan graf G adalah sebagai berikut.

$$M_G = \begin{matrix} & \begin{matrix} v_1 & v_2 & v_3 \end{matrix} \\ \begin{matrix} v_1 \\ v_2 \\ v_3 \end{matrix} & \begin{bmatrix} 1 & 2 & 1 \\ 2 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix} \end{matrix}$$

Misal suatu graf tak berarah G memiliki bobot pada setiap sisinya, maka matriks ketetanggaan menyatakan bobot dari simpul yang terhubung kemudian disebut sebagai matriks ketetanggaan berbobot (*weighted adjacency matrix*). Matriks ketetanggaan berbobot yang merepresentasikan graf tak berarah akan bernilai 0 jika simpul-simpul tersebut tidak terhubung (Aakhirina dan Afrizal, 2020).

Contoh 2.5 Diberikan suatu graf G tak berarah berbobot direpresentasikan dalam gambar berikut.



Gambar 2.4 Graf Contoh 2.5

Nyatakan graf G ke dalam matriks ketetanggaan berbobot!

Penyelesaian:

Matriks ketetanggaan berbobot yang merepresentasikan graf G sebagai berikut.

$$A_G = \begin{matrix} & \begin{matrix} v_1 & v_2 & v_3 \end{matrix} \\ \begin{matrix} v_1 \\ v_2 \\ v_3 \end{matrix} & \begin{bmatrix} 5 & 7 & 9 \\ 7 & 0 & 9 \\ 9 & 9 & 0 \end{bmatrix} \end{matrix}$$

Apabila suatu graf G tak berarah memiliki dua simpul (v_1 dan v_2) dihubungkan oleh dua sisi dengan bobot berbeda, maka representasi matriks ketetanggaan berbobot untuk dua simpul tersebut menyesuaikan pengaplikasiannya. Graf tak berarah yang merepresentasikan permasalahan maksimasi akan mengambil bobot dengan nilai terbesar dan berlaku sebaliknya.

2.2.4.2 Matriks Biner

Matriks biner dapat juga disebut matriks $(0 - 1)$ atau matriks insidensi (*incidence matrix*) adalah matriks yang menyatakan hubungan keterhubungan simpul dengan sisi dari suatu graf yang didefinisikan sebagai berikut.

Definisi 2.6 Misalkan G adalah graf tanpa loop dengan n simpul $v_1, v_2, \dots, v_n$ dan k sisi $e_1, e_2, \dots, e_k$. Matriks biner yang sesuai dengan graf G adalah matriks A berukuran $n \times k$ yang elemennya terdiri dari

$$a_{ij} = \begin{cases} 1 & , \text{jika simpul } v_i \text{ berhubungan dengan sisi } e_j \\ 0 & , \text{jika simpul } v_i \text{ tidak berhubungan dengan sisi } e_j \end{cases} \quad (2.1)$$

Matriks biner sering digunakan ke dalam bidang teori kode, logika digital, dan operasi komputasi (Jek, 2016).

2.3 Optimasi

Optimasi atau optimalisasi merupakan suatu prinsip pencarian solusi layak yang bersifat tepat, efektif, dan efisien (optimum) (Atiqoh, 2020). Pengertian teknis dari optimasi dapat diartikan sebagai tindakan tersistematis yang memaksimalkan sumber daya yang tersedia dengan tujuan menemukan solusi optimal. Solusi dalam optimasi berada dalam daerah layak (*feasible region*) yang memiliki nilai minimum atau maksimum dari fungsi objektif. Proses pencarian solusi optimum dalam optimasi terkait dengan permasalahan komputasional yang bertujuan menemukan so-

lusi terbaik dan layak dari sejumlah alternatif solusi terbatas. Penerapan optimasi sering diterapkan dalam banyak permasalahan yang memiliki keterbatasan sumber daya, seperti permasalahan pencarian rute terpendek pada konsep *traveling salesman problem* (TSP) (Devita dan Wibawa, 2020).

2.4 Konsep *Traveling Salesman Problem* (TSP)

Traveling salesman problem (TSP) merupakan salah satu konsep klasik dalam teori graf dan kombinatorial mengenai pencarian rute terpendek atau lintasan tertutup yang mengunjungi setiap simpul (lokasi) tepat satu kali dan kembali ke simpul awal dengan mengoptimalkan total jarak atau biaya perjalanan agar minimum. Representasi suatu permasalahan TSP umumnya dimodelkan ke dalam graf dengan simpul sebagai lokasi dan sisi sebagai bobot yang akan dioptimalkan. Meskipun konsep TSP terlihat sederhana, namun tingkat kesulitan akan meningkat seiring eksponensial jumlah kemungkinan jalur saat jumlah simpul meningkat (Sinaga dan Marpaung, 2023).

Konsep TSP memiliki banyak penerapan praktis untuk memecahkan permasalahan optimasi yang melibatkan penentuan rute terpendek sehingga setiap simpul dapat dikunjungi dengan biaya atau waktu minimal (Bisma dan Sanggala, 2023). Setiap simpul yang dilewati dalam konsep TSP hanya dilewati tepat satu kali. Sehingga fungsi tujuan dari solusi adalah meminimumkan biaya yang diperlukan untuk melewati simpul-simpul tersebut. Konsep TSP dapat dimodelkan melalui *integer programming* seperti pada persamaan 2.2 berikut.

Fungsi tujuan

$$\min(Z) = \sum_{i=1}^n \sum_{j=1}^n a_{ij}(v_i v_j) \quad (2.2)$$

Fungsi kendala

$$\begin{aligned} \sum_{j=1}^n a_{ij} &= 1, \quad i = 1, 2, \dots, n \\ \sum_{i=1}^n a_{ij} &= 1, \quad j = 1, 2, \dots, n \\ a_{ij} &\in \{0, 1\}, \quad i \neq j \end{aligned}$$

dengan

$v_i v_j$ = biaya dari simpul i menuju simpul j , dan
 a_{ij} = perpindahan dari simpul i menuju simpul j . Bernilai 0 jika tidak terjadi perpindahan dan 1 jika terjadi perpindahan.

Pemecahan solusi optimal untuk permasalahan TSP dengan jumlah simpul yang besar akan menjadi cukup sulit. Pendekatan dengan berbagai metode banyak dilakukan agar dapat memperoleh solusi yang optimal, salah satunya adalah metode pendekatan secara heuristik (Saud dkk., 2018).

2.5 Metode Heuristik

Metode heuristik merupakan strategi pendekatan komputasional untuk menyelesaikan permasalahan pengambilan keputusan yang mungkin sulit bahkan tidak mungkin diselesaikan secara eksak dalam waktu yang wajar. Tujuan utama dari metode heuristik adalah untuk menghasilkan solusi yang cukup baik dengan biaya komputasi yang juga terjangkau. Heuristik mengacu pada aturan praktis atau strategi mental berbasis pengalaman, pengetahuan, atau asumsi yang digunakan dalam mengambil keputusan dengan penyederhanaan prosesnya. Meskipun metode heuristik dapat membantu terhadap permasalahan yang kompleks atau tidak pasti, namun solusi yang dihasilkan tidak menjamin optimal dan sering menyebabkan bias dalam pemikiran (Anwar dkk., 2024).

Metode heuristik secara umum memberikan kerangka kerja yang sederhana dan cepat dengan berbasis pengalaman dan pengetahuan. Adapun beberapa metode heuristik secara umum adalah sebagai berikut:

1. Heuristik Ketersediaan

Heuristik ketersediaan mengambil keputusan didasarkan pada seberapa mudah mengingat kembali contoh-contoh atau informasi terkait. Contohnya orang yang baru mendengar kabar mengerikan mengenai kecelakaan kapal akan berpikir bahwa berlayar itu pilihan tidak aman.

2. Heuristik Representatif

Heuristik representatif mengambil keputusan didasarkan pada asumsi suatu objek atau kondisi serupa dengan pola atau kategori tertentu seperti kesamaan fitur atau kesamaan karakteristik lain. Contohnya penampilan seseorang dinilai mewakili karakteristik kelompok tertentu.

3. Heuristik Ancoran dan Penyesuaian

Heuristik estimasi awal (ancoran) dan penyesuaian mengambil keputusan dengan membuat estimasi awal berdasarkan informasi awal yang kemudian mengubah estimasi tersebut berdasarkan informasi yang diterima selanjutnya. Contohnya tawar-menawar harga mobil dengan estimasi awal mengikuti harga jual awal oleh penjual kemudian berubah berdasarkan informasi spesifikasi mobil tersebut.

4. Heuristik Kesederhanaan

Heuristik kesederhanaan mengambil keputusan secara sederhana berdasarkan pada informasi yang paling mudah diakses untuk menggantikan informasi sebenarnya. Contohnya memutuskan membeli sepatu merk A menggunakan opini yang sedang *trending* di media sosial.

5. Heuristik Atribusi

Heuristik atribusi mengambil keputusan didasarkan pengetahuan saat ini untuk memberikan penjelasan singkat terhadap kondisi atau perilaku tertentu. Contohnya memberikan penjelasan bahwa orang yang terlambat itu akibat dari rasa malas dari orang tersebut.

Metode heuristik banyak diaplikasikan ke dalam berbagai bidang yang dihadapkan pengambilan keputusan tertentu. Salah satu contoh penggunaan metode heuristik adalah algoritma *particle swarm optimization* yang termasuk ke dalam metode heuristik representatif dengan menggunakan perilaku sosial kawanan hewan dalam mencari mangsa sebagai representatif permasalahan optimasi (Muhammad. dkk., 2023).

2.6 Algoritma *Particle Swarm Optimization* (PSO)

Algoritma *particle swarm optimization* (PSO) merupakan algoritma optimasi heuristik berbasis populasi yang diperkenalkan oleh James Kennedy dan Russel Eberhart pada tahun 1995. Basis populasi terinspirasi dari perilaku sosial hewan seperti burung dalam suatu kawanan (*swarm*) pada saat menelusuri area tertentu untuk mencari makanan. Perilaku sosial kawanan tersebut dalam mencari makanan meliputi kecerdasan, kecepatan, dan komunikasi yang terorganisir (Kusrahman dkk., 2020).

2.6.1 Definisi Algoritma *Particle Swarm Optimization*

Algoritma *particle swarm optimization* merupakan salah satu teknik optimasi komputasional. Perilaku sosial hewan dalam mencari makan pada suatu daerah menginspirasi konsep dasar algoritma *particle swarm optimization*. Konsep untuk mengeksplorasi individu dalam pencarian solusi, dilakukan dengan populasi yang disebut *swarm* dan individu disebut *particle*. Setiap *particle* akan bergerak semakin mendekati solusi dengan kecepatan (*velocity*) yang terus beradaptasi terhadap daerah pencarian dan informasi posisi terbaik yang pernah dicapai selalu disimpan (Muhammad. dkk., 2023).

Konsep penelusuran algoritma *particle swarm optimization* memiliki kesamaan dengan algoritma genetika, yaitu populasi awal yang digunakan adalah populasi acak dalam bentuk matriks. Representasi populasi pada matriks menggunakan baris sebagai *particle* untuk algoritma *particle swarm optimization* atau kromosom untuk algoritma genetika. Perbedaan mendasar antara kedua algoritma ini adalah algoritma *particle swarm optimization* tidak memiliki operator yang dapat mengevolusi partikel seperti *crossover* dan mutasi pada algoritma genetika (Zahro dan Wahyuni, 2020).

Algoritma *particle swarm optimization* menggunakan beberapa istilah untuk mencakup informasi dari suatu permasalahan optimasi. Adapun istilah umum yang sering digunakan dalam algoritma *particle swarm optimization* dijelaskan sebagai berikut:

1. *Swarm* : populasi dari algoritma *particle swarm optimization*.
2. *Particle* (X_j) : individu di dalam suatu *swarm*. Dalam satu terdiri dari beberapa dimensi (d) sesuai dengan dimensi dari model permasalahan yang akan diselesaikan. Seluruh *particle* merepresentasikan solusi potensial pada saat proses optimisasi. Posisi setiap *particle* selalu diperbarui berdasarkan kecepatan (*velocity*) tertentu.
3. *Velocity* (v): vektor yang memperbarui posisi dan arah *particle* berpindah dari posisi semula.
4. *Inertia Weight* (ω) : parameter untuk mengendalikan dampak dari *velocity* sebelumnya terhadap *velocity* sekarang suatu *particle*.
5. *Personal Best* (P_{best}) : posisi terbaik yang pernah dicapai oleh *particle* yang

dipersiapkan untuk mendapatkan solusi terbaik.

6. *Global Best* (G_{best}) : posisi terbaik *particle* pada *swarm*.

(Muhardeny dkk., 2023).

Partikel solusi dalam algoritma *particle swarm optimization* diperoleh dari partikel dengan posisi yang menghasilkan nilai fungsi *fitness* terbaik di semua iterasi. Partikel berisikan posisi dari sejumlah dimensi partikel, dimana dimensi partikel merupakan jumlah parameter yang akan dioptimalkan. Proses pencarian solusi pada algoritma *particle swarm optimization* memandang bahwa untuk setiap partikel cenderung akan bergerak ke arah solusi berdasarkan kecepatan (*velocity*). Vektor posisi dan kecepatan partikel pada iterasi sekarang dapat dinotasikan dalam persamaan berikut:

$$\mathbf{X}_j^{(i)} = \begin{bmatrix} x_{j,1}^{(i)} \\ \vdots \\ x_{j,d}^{(i)} \end{bmatrix} \quad (2.3)$$

dan

$$\mathbf{V}_j^{(i)} = \begin{bmatrix} v_{j,1}^{(i)} \\ \vdots \\ v_{j,d}^{(i)} \end{bmatrix} \quad (2.4)$$

dengan j adalah indeks dari partikel dan d adalah jumlah dimensi partikel (Akpudo dan Jang-Wook, 2020).

Penggunaan algoritma *particle swarm optimization* memiliki keuntungan utama yaitu memiliki waktu komputasi yang rendah, parameter yang perlu disesuaikan relatif sedikit, dan mampu membangun model matematika yang akurat untuk suatu permasalahan kompleks. Keuntungan lain penggunaan algoritma *particle swarm optimization* adalah tidak terjadi tumpah tindih atau mutasi pada saat proses perhitungan. Keuntungan tersebut sudah cukup menjadikan algoritma *particle swarm optimization* sering untuk digunakan. Namun algoritma ini memiliki kelemahan, salah satunya adalah memerlukan penyimpanan data lebih untuk memperbarui *velocity* maupun posisi terbaik saat iterasi (Gad, 2022).

2.6.2 Parameter Algoritma *Particle Swarm Optimization*

Parameter dalam algoritma *particle swarm optimization* memberikan pengaruh besar terhadap kinerja algoritma. Pemilihan parameter yang digunakan dan penentuan nilainya dapat meningkatkan kinerja algoritma menjadi lebih efisien. Terdapat beberapa parameter dasar dari algoritma *particle swarm optimization* dijelaskan sebagai berikut:

1. Ukuran *Swarm*

Ukuran *swarm* merupakan nilai banyaknya *swarm* yang menjadi populasi dari algoritma. Penentuan ukuran *swarm* yang besar dapat mengurangi jumlah iterasi untuk mencapai perhitungan yang konvergen. Akan tetapi, penggunaan ukuran *swarm* yang besar tentu akan meningkatkan perhitungan tiap iterasi menjadi lebih kompleks. Hal ini mengakibatkan waktu untuk menyelesaikan perhitungan dalam satu iterasi menjadi semakin lama. Besar dari sebuah populasi juga dapat menentukan ukuran *swarm*. Beberapa penelitian tentang implementasi algoritma *particle swarm optimization* klasik menggunakan interval $N \in [20, 50]$ untuk ukuran *swarm* (Piotrowski dkk., 2020).

2. Jumlah iterasi

Jumlah iterasi merupakan jumlah pengulangan dari proses perhitungan yang akan dilakukan dalam algoritma. Penentuan jumlah iterasi pada algoritma *particle swarm optimization* mempengaruhi pencarian solusi optimal. Jika jumlah iterasi yang digunakan terlalu kecil akan berdampak pada proses perhitungan menjadi berhenti saat solusi belum mencapai optimal. Namun sebaliknya, jumlah iterasi yang terlalu besar akan menyebabkan proses perhitungan menjadi semakin kompleks, menambah waktu perhitungan, serta menambahkan perhitungan yang tidak diperlukan (Jain dkk., 2022).

3. *Learning Rates* (c_1 dan c_2)

Learning rates atau *acceleration coefficients* terdiri dari dua parameter adaptif dalam algoritma *particle swarm optimization* yang mengatur keseimbangan dari kecerdasan posisi partikel terhadap pengalaman pribadi yaitu (P_{best}) dan pengalaman kolektif yaitu (G_{best}). Parameter kognitif (c_1) merupakan parameter yang mengatur pengaruh P_{best} terhadap posisi partikel. Sedangkan, parameter sosial (c_2) merupakan parameter yang mengatur pengaruh G_{best} terhadap posisi

partikel. Terdapat beberapa penentuan nilai c_1 dan c_2 , yaitu sebagai berikut:

- (a). Jika $c_1 = c_2 = 0$, maka setiap partikel akan bergerak dengan kecepatan partikel tersebut secara konstan sampai batas ruang pencarian.
- (b). Jika $c_1 > 0$ dan $c_2 = 0$, maka setiap partikel akan bergerak dengan kecepatan partikel terhadap P_{best} masing-masing (independen) karena komponen sosial tidak mempengaruhi. Sedangkan sebaliknya, maka setiap partikel akan bergerak dengan kecepatan partikel terhadap G_{best} sehingga partikel hanya akan tertarik pada satu arah titik.
- (c). Jika $c_1 = c_2$, maka setiap partikel akan bergerak dengan kecepatan partikel terhadap rata-rata dari P_{best} dan G_{best} .
- (d). Jika $c_1 > c_2$, maka kecepatan partikel untuk setiap partikel akan lebih dipengaruhi oleh P_{best} daripada G_{best} . Sedangkan sebaliknya, maka kecepatan partikel akan lebih dipengaruhi G_{best} daripada P_{best} .

Penentuan nilai dari c_1 dan c_2 secara umum bersifat statis yang diperoleh dari beberapa penelitian terdahulu. Kesalahan dalam penentuan nilai c_1 dan c_2 menyebabkan proses pencarian solusi menjadi divergen. Dalam berbagai penelitian, telah diusulkan bahwa penentuan nilai c_1 dan c_2 dapat menggunakan nilai $c_1 = c_2 = 2$ (2022).

4. Inertia Weight (ω)

Inertia weight atau bobot inersia (ω) merupakan parameter yang mengontrol efek dari *velocity* sebelumnya terhadap *velocity* sekarang dari suatu partikel. Penentuan *inertia weight* yang terlalu besar akan menyebabkan peningkatan berlebihan pada *velocity* saat diperbarui, akibatnya partikel akan terlalu jauh bergerak dan terlalu cepat bahkan melewati solusi optimal. Terdapat beberapa penentuan nilai *inertia weight*, yaitu sebagai berikut:

- (a). Jika $\omega \geq 1$, maka nilai *velocity* akan meningkat secara berlebih saat diperbarui dan tidak dapat bergerak ke arah yang optimal sehingga solusi akan menjadi divergen.
- (b). Jika $\omega < 1$, maka akan terdapat momentum kecil pada *velocity* sebelumnya sehingga dapat digunakan untuk melakukan perubahan arah secara cepat menuju ke posisi yang lebih baik saat proses pencarian solusi.
- (c). Jika $\omega = 0$, maka partikel akan bergerak tanpa mengetahui *velocity* se-

belumnya sehingga partikel tidak dapat mengetahui perubahan posisinya secara benar bahkan dapat keluar dari arah solusi optimal.

Nilai *inertia weight* yang sesuai dengan ketentuan tersebut adalah pada interval $\omega \in [0.1, 0.9]$. Shi dan Eberhart memperkenalkan strategi *constant inertia weight* dan *random inertia weight*. Strategi *constant inertia weight* akan mengambil nilai *inertia weight* di setiap iterasi adalah konstan ($\omega^{(i)} = \omega$). Sedangkan pada strategi *random inertia weight*, nilai *inertia weight* diambil secara acak namun tetap berada di atas rata-rata dari interval ketentuan. Nilai *inertia weight* di setiap iterasi i dengan strategi *random inertia weight* ditentukan menggunakan persamaan berikut:

$$\omega^{(i)} = 0.5 + \frac{r}{2} \quad (2.5)$$

dengan nilai r adalah bilangan acak berdistribusi *uniform* $r \sim U(0, 1)$ (Zdiri dkk., 2021).

2.6.3 Kecepatan Partikel (*Velocity*)

Velocity atau kecepatan partikel (v) merupakan vektor yang memperbarui posisi dan arah *particle* berpindah dari posisi sebelumnya. Penentuan *velocity* yang terlalu besar dapat menyebabkan *particle* bergerak tidak menentu dan melewati solusi optimal. Dan sebaliknya, *velocity* yang terlalu kecil dapat menyebabkan *particle* bergerak terlalu lambat bahkan tidak bergerak sehingga posisinya terjebak hanya di lokal optimal. Eberhart dan Kennedy pertama kali memperkenalkan strategi batas penentuan kecepatan (*velocity clamping*). Strategi *velocity clamping* menentukan kecepatan maksimum (V_{max}) berdasarkan inisialisasi posisi dan konstanta acak. Kecepatan maksimum ditentukan menggunakan persamaan berikut:

$$V_{max} = \varepsilon(X_{max} - X_{min}) \quad (2.6)$$

dengan ε adalah faktor *clamping* untuk *velocity*. Nilai ε berada pada interval nilai acak $\varepsilon \in [0, 1]$ dan X_{max}, X_{min} adalah batas atas dan bawah dari inisialisasi posisi (X) (Wang dkk., 2022).

2.6.4 *Personal Best*

Personal best atau lokal optimum (P_{best}) dalam algoritma *particle swarm optimization* merupakan vektor yang menyimpan posisi terbaik untuk setiap partikel X_j di setiap iterasi i . Posisi terbaik didasarkan pada posisi dari X_j yang menghasilkan nilai fungsi *fitness* terbaik yang pernah dicapai di semua iterasi sampai iterasi terakhir $i = n$. Untuk setiap iterasi, hasil perhitungan nilai fungsi *fitness* partikel X_j sekarang akan dibandingkan dengan nilai fungsi *fitness* menggunakan posisi dari P_{best} sebelumnya. Dengan kata lain, *personal best* berisikan semua posisi terbaik yang pernah dikunjungi oleh partikel X_j hingga iterasi ke n . *Personal best* dapat dinotasikan sebagai berikut:

$$\mathbf{P}_{best,j}^{(i)} = \begin{bmatrix} p_{j,1}^{(i)} \\ \vdots \\ p_{j,d}^{(i)} \end{bmatrix} \in X_j \quad (2.7)$$

dengan d adalah jumlah dimensi di dalam X_j (Ramadhan dkk., 2023).

2.6.5 *Global Best*

Global best atau global optimum (G_{best}) dalam algoritma *particle swarm optimization* merupakan vektor yang menyimpan posisi dari P_{best} terbaik saat iterasi sekarang. Untuk setiap iterasi i , G_{best} didasarkan pada P_{best} dari setiap partikel X_j pada iterasi i yang memiliki nilai fungsi *fitness* terbaik. Dengan kata lain, *global best* berisikan posisi dari P_{best} terbaik pada iterasi sekarang. *Global best* dapat dinotasikan sebagai berikut:

$$\mathbf{G}_{best}^{(i)} = \begin{bmatrix} g_1^{(i)} \\ \vdots \\ g_d^{(i)} \end{bmatrix} \quad (2.8)$$

dengan d adalah jumlah dimensi di dalam X (Darmawan dkk., 2022).

2.6.6 *Pembaruan Posisi dan Velocity*

Proses pencarian solusi pada algoritma *particle swarm optimization* memandang bahwa setiap partikel cenderung akan bergerak ke arah solusi berdasarkan kecepatan (*velocity*) tertentu. Posisi dan *velocity* dari partikel akan selalu diperbarui

di setiap iterasi sampai iterasi terakhir $i = n$. Nilai dari *velocity* akan menentukan arah perpindahan dari setiap partikel di setiap iterasi i agar dapat menemukan ruang solusi terbaik. Dalam pembaruan nilai *velocity* akan terpengaruh oleh *velocity*, *personal best*, dan *global best* dari iterasi sebelumnya. Pembaruan posisi dan *velocity* ditentukan menggunakan persamaan berikut:

$$\mathbf{V}_j^{(i)} = \omega^{(i)} \mathbf{V}_j^{(i-1)} + c_1 r_1^{(i)} \left(\mathbf{P}_{best,j}^{(i-1)} - \mathbf{X}_j^{(i-1)} \right) + c_2 r_2^{(i)} \left(\mathbf{G}_{best} - \mathbf{X}_j^{(i-1)} \right) \quad (2.9)$$

dan

$$\mathbf{X}_j^{(i)} = \mathbf{X}_j^{(i-1)} + \mathbf{V}_j^{(i)} \quad (2.10)$$

dengan $r_1^{(i)}, r_2^{(i)}$ adalah bilangan acak berdistribusi *uniform* $r \sim U(0, 1)$ di setiap iterasi i (Chafi dan Afrakhte., 2021).

2.6.7 Tahapan Algoritma *Particle Swarm Optimization*

Proses pencarian solusi optimum pada algoritma *particle swarm optimization* dilakukan hingga iterasi maksimum atau kriteria pemberhentian terpenuhi. Untuk setiap iterasi, perpindahan posisi selalu diperbarui sehingga mendekati solusi optimum. Tahapan dan proses perhitungan algoritma *particle swarm optimization* untuk konsep *traveling salesman problem* sebagai berikut:

1. Menginisialisasi nilai parameter algoritma *particle swarm optimization* seperti jumlah iterasi maksimum (n), ukuran *swarm* (N), (d), (c_1, c_2), batas posisi (X_{min}, X_{max}), posisi awal (X_i), batas *velocity* maksimum (V_{max}), dan *velocity* awal (V_j^0). Inisialisasi nilai awal dapat dilakukan melalui tahap berikut:
 - (a). Menentukan jumlah iterasi maksimum (n).
 - (b). Menentukan ukuran *swarm* (N) dan dimensi partikel (d) didasarkan pada beberapa penelitian tentang implementasi algoritma *particle swarm optimization* klasik yaitu menggunakan interval $N \in [20, 50]$ serta besar dimensi partikel menyesuaikan dari model permasalahan yang akan diselesaikan (Piotrowski dkk., 2020).
 - (c). Menentukan *learning rates* (c_1) dan (c_2) didasarkan pada beberapa penelitian tentang *learning rates* dalam algoritma *particle swarm optimization* yaitu menggunakan nilai $c_1 = c_2 = 2$ (Jain dkk., 2022).

- (d). Menentukan batas atas (X_{max}) dan batas bawah (X_{min}) untuk posisi partikel.
- (e). Menentukan batas *velocity* maksimum (V_{max}) diperoleh secara acak berdasarkan persamaan 2.6, sedangkan *velocity* minimum (V_{min}) diperoleh dari $V_{min} = -V_{max}$.
- (f). Membangkitkan *velocity* awal (V_j^0) sejumlah N partikel secara acak yang tetap berada di batas *velocity* menggunakan persamaan berikut:

$$v_{j,d}^{(0)} = U(V_{min}, V_{max}) \quad (2.11)$$

- (g). Membangkitkan posisi awal (X_j^0) sejumlah N partikel secara acak yang tetap berada di batas posisi partikel menggunakan persamaan berikut:

$$x_{j,d}^{(0)} = U(X_{min}, X_{max}) \quad (2.12)$$

- (h). Mengevaluasi hasil rute setiap partikel berdasarkan urutan dimensi dalam partikel dengan nilai posisi terkecil hingga terbesar.

2. Mengevaluasi nilai fungsi *fitness* hasil rute setiap partikel dengan menggunakan persamaan berikut:

$$F(X_j^{(i)}) = \frac{1}{f(X_j^{(i)})} \quad (2.13)$$

dengan $f(X_j)$ adalah total biaya yang dikeluarkan untuk menggunakan hasil rute dari partikel j .

3. Menentukan $P_{best}^{(0)}$ dan $G_{best}^{(0)}$ awal.
4. Membangkitkan nilai *inertia weight* berdasarkan strategi *random inertia weight* menggunakan persamaan 2.5.
5. Memperbarui *velocity* setiap partikel menggunakan persamaan 2.9 dengan nilai $r_1^{(i)}$ dan $r_2^{(i)}$ dibangkitkan secara acak menggunakan distribusi *uniform* $r \sim U(0, 1)$. Jika *velocity* terbaru tidak berada di dalam batas yang diizinkan maka dilakukan penyesuaian sebagai berikut:

$$v_{j,d}^{(i)} = \begin{cases} V_{min} & , v_{j,d}^{(i)} < V_{min} \\ V_{max} & , v_{j,d}^{(i)} > V_{max} \end{cases} \quad (2.14)$$

6. Memperbarui posisi setiap partikel menggunakan persamaan 2.10 dengan *velocity* yang telah diperbarui. Jika posisi terbaru tidak berada di dalam batas yang diizinkan maka dilakukan penyesuaian sebagai berikut:

$$x_{j,d}^{(i)} = \begin{cases} X_{min} & , x_{j,d}^{(i)} < X_{min} \\ X_{max} & , x_{j,d}^{(i)} > X_{max} \end{cases} \quad (2.15)$$

7. Mengevaluasi hasil rute setiap partikel.
 8. Mengevaluasi nilai fungsi *fitness* hasil rute setiap partikel.
 9. Menentukan $P_{best,j}^{(i)}$ setiap partikel dan $G_{best}^{(i)}$. Untuk setiap partikel, $P_{best,j}^{(i)}$ ditentukan dengan menggunakan persamaan berikut:

$$P_{best,j}^{(i)} = \begin{cases} X_j^{(i)} & , F(X_j^{(i)}) \geq F(P_{best,j}^{(i-1)}) \\ P_{best,j}^{(i-1)} & , F(X_j^{(i)}) < F(P_{best,j}^{(i-1)}) \end{cases} \quad (2.16)$$

dan untuk $G_{best}^{(i)}$ berdasarkan posisi dengan nilai fungsi *fitness* paling terbaik yang pernah dicapai di antara seluruh partikel pada semua iterasi. Posisi terbaik dari setiap partikel di setiap iterasi disimpan ke dalam P_{best} . Sehingga G_{best} pada iterasi ke- i dapat ditentukan dengan membandingkan nilai fungsi *fitness* dari P_{best} terbaik pada iterasi ke- i dan nilai fungsi *fitness* dari G_{best} pada iterasi sebelumnya.

10. Mengevaluasi kriteria pemberhentian terhadap solusi terbaru yang diperoleh. Jika solusi terbaru telah memenuhi kriteria pemberhentian maka iterasi berhenti, jika tidak memenuhi maka iterasi diperbarui dan kembali ke langkah 4.

2.7 Kriteria Pemberhentian Algoritma *Particle Swarm Optimization*

Proses pencarian solusi pada algoritma *particle swarm optimization* selalu diperbarui di setiap iterasi. Jika solusi terbaru telah memenuhi kriteria pemberhentian, maka pencarian solusi dihentikan. Beberapa kriteria pemberhentian pada algoritma *particle swarm optimization* untuk konsep *traveling salesman problem* adalah sebagai berikut:

1. Iterasi maksimum, proses pencarian solusi akan dihentikan jika iterasi yang berjalan sudah mencapai batas iterasi maksimum.

2. Nilai fungsi *fitness* maksimum, proses pencarian solusi akan dihentikan jika nilai fungsi *fitness* telah mencapai batas yang diberikan, yaitu total biaya maksimum.
3. Populasi konvergen, proses pencarian solusi akan dihentikan jika standar deviasi dari posisi partikel dalam P_{best} kurang dari batas toleransi.
4. Nilai fungsi *fitness* konvergen, proses pencarian solusi akan dihentikan jika beda antara nilai fungsi *fitness* maksimum dan minimum dalam satu iterasi kurang dari batas toleransi.
5. Solusi (G_{best}) konvergen, proses pencarian solusi akan dihentikan jika beda antara nilai fungsi *fitness* dari G_{best} sekarang dan iterasi sebelumnya kurang dari batas toleransi.

Kriteria pemberhentian akan mencegah sumber daya tidak terpakai secara sia-sia (Eltamaly, 2021).

2.8 Metode *Min-Max Normalization*

Metode *Min-Max Normalization* merupakan salah satu metode normalisasi data dengan mengubah ukuran dari rentang data asli ke dalam rentang 0 sampai 1 (Ambarwari dkk., 2019). Normalisasi data dapat mempercepat proses perhitungan data dari suatu analisis. Proses normalisasi data dilakukan tanpa mengubah distorsi dari rentang data asli. Normalisasi data dengan metode *Min-Max Normalization* dilakukan menggunakan persamaan berikut.

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (2.17)$$

2.9 Film "Ketika Adzan Sudah Tidak Lagi Berkumandang"

Film berjudul "Ketika Adzan Sudah Tidak Lagi Berkumandang" merupakan film layar lebar dokumenter bergenre horor yang diproduksi oleh East Borneo Film. Dalam film ini menceritakan tentang dua wartawan yang bertugas di sebuah desa terpencil untuk membuat sebuah film dokumenter di sana. Pembuatan film ini diproduksi sekaligus disutradarai oleh David Richard. Keseluruhan proses pembuatan film ini dilakukan di Kalimantan Timur.

Naskah film "Ketika Adzan Sudah Tidak Lagi Berkumandang" menghasilkan banyak pecahan *scene* yang harus diambil dalam proses syuting. Proses syuting

pada film ini menggunakan tiga lokasi utama yang berbeda, yaitu Waduk Tenggarong, Desa Loa Raya, dan Desa Kedang Ipil. Setiap lokasi utama terbagi menjadi beberapa titik lokasi yang menjadi lokasi syuting *scene* yang berbeda. Perpindahan lokasi pada proses syuting antar *scene* akan memindahkan seluruh alat dan properti sehingga memerlukan biaya transportasi. Peningkatan biaya transportasi yang berlebih dapat membebani biaya produksi dari film ini.

BAB III

METODE PENELITIAN

3.1 Waktu dan Tempat Penelitian

Penelitian ini dilaksanakan pada bulan Maret 2025 sampai Mei 2025. Pengambilan data dilakukan pada film "Ketika Adzan Sudah Tidak Lagi Berkumandang". Pengolahan data dalam penelitian ini dilakukan di Laboratorium Matematika Dasar dan Laboratorium Matematika Komputasi yang terletak di Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Mulawarman, Samarinda, Kalimantan Timur.

3.2 Variabel Penelitian

Variabel yang digunakan dalam penelitian ini adalah lokasi syuting setiap *scene* dan biaya perpindahan antar setiap *scene* pada film "Ketika Adzan Sudah Tidak Lagi Berkumandang". Penelitian ini memfokuskan pada pemilihan urutan syuting *scene* dengan konsep *traveling salesman problem* menggunakan algoritma *particle swarm optimization* sehingga tahap syuting setiap *scene* berjalan lebih efektif dan efisien. Variabel yang digunakan dapat dilihat pada tabel berikut:

Tabel 3.1 Variabel Penelitian

Variabel	Keterangan	Satuan
v_i	Titik atau lokasi ke- i	-
$v_i v_j$	Biaya antara titik ke- i dan titik ke- j	Rupiah (Rp)

3.3 Teknik Pengumpulan Data

Teknik pengumpulan data yang digunakan dalam penelitian ini adalah menggunakan data sekunder. Data sekunder yang diambil dari *Master Breakdown* film "Ketika Adzan Sudah Tidak Lagi Berkumandang" terdiri dari lokasi syuting setiap *scene*, pemain (*talent*) di setiap *scene*, dan biaya setiap pemain (*talent*). Sedangkan data sekunder yang diambil dari aplikasi Google Maps adalah jarak dari lokasi syuting setiap *scene*.

3.4 Populasi dan Sampel Penelitian

Populasi pada penelitian ini adalah rute dan biaya perpindahan antar setiap *scene* pada film "Ketika Adzan Sudah Tidak Lagi Berkumandang". Sampel yang

digunakan pada penelitian ini sama dengan populasi penelitian, yaitu seluruh rute dan biaya perpindahan antar setiap *scene* pada film "Ketika Adzan Sudah Tidak Lagi Berkumandang".

3.5 Teknik Sampling

Teknik sampling yang digunakan dalam penelitian ini adalah teknik *purposive sampling*. Teknik *purposive sampling* merupakan teknik penentuan sampel dengan mempertimbangkan maksud dan tujuan tertentu karena sampel tersebut memiliki informasi yang diperlukan (Amin dkk., 2023). Pada penelitian ini sampel yang digunakan sama dengan populasi karena bertujuan untuk mengambil seluruh data pada film "Ketika Adzan Sudah Tidak Lagi Berkumandang".

3.6 Teknik Analisis Data

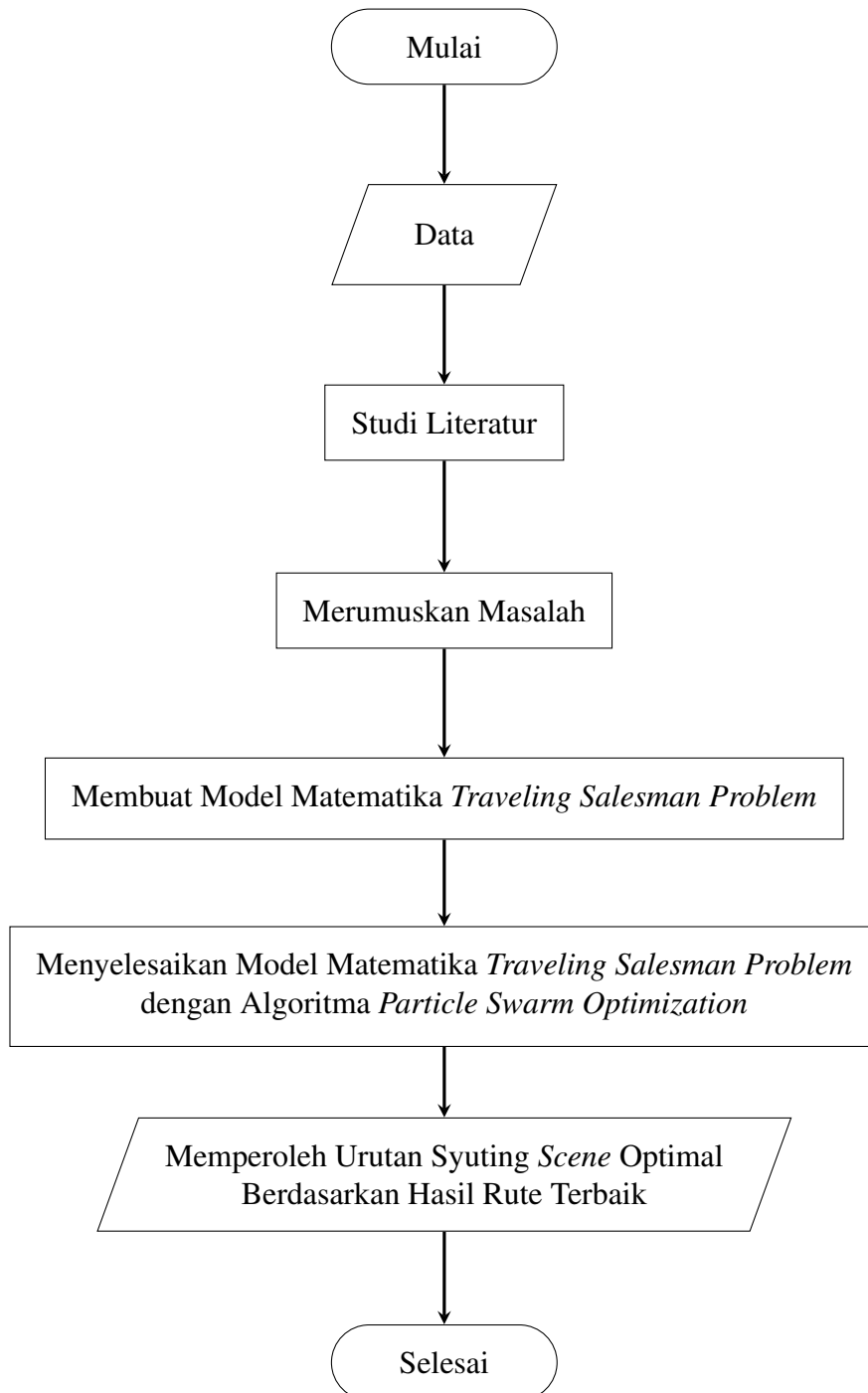
Teknik analisis data pada penelitian ini menggunakan konsep *traveling salesman problem* dengan metode algoritma *particle swarm optimization*. Adapun tahapan yang digunakan sebagai berikut:

1. Mengumpulkan dan mempersiapkan data, dengan tahapan sebagai berikut:
 - (a) Mengumpulkan data lokasi syuting setiap *scene*, pemain (*talent*) di setiap *scene*, biaya pemain (*talent*) di setiap *scene*, dan jarak antar lokasi syuting setiap *scene*.
 - (b) Menghitung biaya perpindahan antar setiap *scene*.
2. Studi Literatur
 Studi literatur dilakukan untuk mendapatkan informasi dari buku-buku dan catatan penelitian terdahulu mengenai konsep *traveling salesman problem* dengan metode algoritma *particle swarm optimization*. Informasi tersebut akan menjadi pendukung dari ide dalam penyelesaian masalah.
3. Merumuskan masalah dan variabel penelitian
4. Melakukan tahap inisialisasi, dengan tahapan sebagai berikut:
 - (a) Menentukan jumlah iterasi maksimum (n).
 - (b) Menentukan ukuran *swarm* (N) dan dimensi partikel (d).
 - (c) Menentukan *learning rates* (c_1 dan c_2).
 - (d) Menentukan batas atas (X_{max}) dan batas bawah (X_{min}) dari posisi partikel.
 - (e) Menentukan *velocity* maksimum (V_{max}).

- (f) Membangkitkan *velocity* awal ($\mathbf{V}_j^{(0)}$) sejumlah N partikel.
 - (g) Membangkitkan posisi awal ($\mathbf{X}_j^{(0)}$) sejumlah N partikel.
 - (h) Mengevaluasi hasil rute setiap partikel.
 - (i) Mengevaluasi nilai fungsi *fitness* setiap partikel.
 - (j) Menentukan $P_{best,j}^{(0)}$ setiap partikel dan $G_{best}^{(0)}$.
5. Melakukan tahap perulangan, dengan tahapan sebagai berikut:
- (a) Membangkitkan *inertia weight* ($\omega^{(i)}$).
 - (b) Membangkitkan $r_1^{(i)}$ dan $r_2^{(i)}$.
 - (c) Memperbarui *velocity* setiap partikel.
 - (d) Memperbarui posisi setiap partikel.
 - (e) Mengevaluasi hasil rute setiap partikel.
 - (f) Mengevaluasi nilai fungsi *fitness* setiap partikel.
 - (g) Menentukan $P_{best}^{(i)}$ setiap partikel dan $G_{best}^{(i)}$.
 - (h) Mengevaluasi kriteria pemberhentian.
6. Menarik kesimpulan urutan syuting *scene* menggunakan hasil rute terbaik dari G_{best} .

3.7 Kerangka Penelitian

Kerangka dari penelitian ini disajikan dalam *flowchart* sebagai berikut:



Gambar 3.1 *Flowchart* Kerangka Penelitian

BAB IV

HASIL DAN PEMBAHASAN

Pada bab 4 akan dibahas tentang hasil penelitian yang telah dilakukan yaitu menentukan urutan syuting *scene* film menggunakan algoritma *particle swarm optimization* pada film "Ketika Adzan Sudah Tidak Lagi Berkumandang". Hasil penelitian dibahas ke dalam beberapa bagian pembahasan. Adapun hasil penelitian yang dibahas yaitu model graf *scene* film "Ketika Adzan Sudah Tidak Lagi Berkumandang", penerapan algoritma *Particle Swarm Optimization* (PSO) untuk optimalisasi pemilihan urutan syuting *scene* film "Ketika Adzan Sudah Tidak Lagi Berkumandang", dan hasil optimalisasi pemilihan urutan syuting *scene* film "Ketika Adzan Sudah Tidak Lagi Berkumandang" menggunakan algoritma *particle swarm optimization* (PSO)

4.1 Model Graf dan Model Matematika Pemilihan Urutan Scene Syuting Film "Ketika Adzan Sudah Tidak Lagi Berkumandang"

Pemilihan urutan *scene* syuting film "Ketika Adzan Sudah Tidak Lagi Berkumandang" dimodelkan secara matematis ke dalam konsep *traveling salesman problem* (TSP) pada penelitian ini. Konsep TSP diterapkan dengan memanfaatkan model graf sebagai representasi dari keseluruhan *scene* yang ada. Seluruh *scene* yang ada pada data penelitian digunakan dalam membentuk model graf serta model matematika dalam penelitian ini.

4.1.1 Model Graf Pemilihan Urutan Scene Syuting Film "Ketika Adzan Sudah Tidak Lagi Berkumandang"

Model graf *scene* film "Ketika Adzan Sudah Tidak Lagi Berkumandang" dibuat dengan melihat data penelitian. Data yang digunakan dalam penelitian ini adalah data *scene* dan data biaya syuting tiap *scene* film "Ketika Adzan Sudah Tidak Lagi Berkumandang". Data ini diperoleh dari data pada *Master Breakdown*, *Call Sheet* dan Keuangan dalam proses produksi film "Ketika Adzan Sudah Tidak Lagi Berkumandang". Informasi yang diperoleh dari sumber data tersebut meliputi daftar *scene*, urutan syuting *scene*, lokasi syuting setiap *scene*, *talent* (pemain) yang

bermain di setiap *scene*, biaya *talent*, dan jumlah kendaraan.

Data *scene* pada film "Ketika Adzan Sudah Tidak Lagi Berkumandang" terdiri atas 60 *scene*. Untuk setiap *scene* memiliki lokasi kegiatan syuting yang berbeda-beda. Data *scene* pada film "Ketika Adzan Sudah Tidak Lagi Berkumandang" dapat ditampilkan pada Tabel 4.1 berikut.

Tabel 4.1 Data *Scene*

Indeks	<i>Scene</i>	Lokasi	Variabel
0	4	Kebun Bunga Waduk Panji Tenggarong	v_0
1	1C	Masjid Waduk Panji Tenggarong	v_1
2	1D	Hutan Bambu Desa Kedang Ipil	v_2
3	1E	Air Terjun Desa Kedang Ipil	v_3
4	2A	Rumah Warga Desa Kedang Ipil	v_4
5	2B	Rumah Warga Desa Kedang Ipil	v_5
6	3	Kebun Bunga Waduk Panji Tenggarong	v_6
...	...	...	...
53	33B	Hutan Bambu Desa Kedang Ipil	v_{53}
54	34	Kebun Bunga Waduk Panji Tenggarong	v_{54}
55	34B	Hutan Bambu Desa Kedang Ipil	v_{55}
56	35A	Kebun Bunga Waduk Panji Tenggarong	v_{56}
57	35B	Kebun Bunga Waduk Panji Tenggarong	v_{57}
58	36	Masjid Waduk Panji Tenggarong	v_{58}
59	37	Kebun Bunga Waduk Panji Tenggarong	v_{59}

Sumber : Lampiran 1

Berdasarkan Tabel 4.1, dapat dilihat bahwa terdapat 60 *scene* dan lokasinya. Proses syuting dalam film "Ketika Adzan Sudah Tidak Lagi Berkumandang" dilakukan pada lokasi yang berbeda-beda secara keseluruhan. Rute dari lokasi geografis untuk setiap *scene* diperoleh dengan *software* Google Earth. Lokasi ge-

ografis dari setiap *scene* pada Tabel 4.1 dapat ditampilkan pada Gambar 4.1 berikut.



Gambar 4.1 Peta Lokasi Syuting *Scene* Film "Ketika Adzan Sudah Tidak Lagi Berkumandang"

Berdasarkan Gambar 4.1, terlihat bahwa jarak antara setiap lokasi syuting *scene* saling terhubung sehingga dapat direpresentasikan ke dalam bentuk graf lengkap. Tiap *scene* dalam proses syuting film "Ketika Adzan Sudah Tidak Lagi Berkumandang" dianalogikan sebagai suatu *vertex* dalam graf. Sedangkan *edge* di dalam graf tersebut merepresentasikan bobot biaya syuting antar tiap *scene* film "Ketika Adzan Sudah Tidak Lagi Berkumandang". Bobot biaya syuting antar setiap *scene* diperoleh dari hasil perhitungan pada data biaya syuting tiap *scene*.

Data biaya syuting tiap *scene* film "Ketika Adzan Sudah Tidak Lagi Berkumandang" dibagi menjadi dua, yaitu data biaya *talent* dan data biaya transportasi. Data biaya *talent* merupakan data biaya yang harus dikeluarkan untuk menyewa *talent* yang bermain dalam satu *scene*. Daftar total biaya *talent* yang dapat dilihat pada Lampiran 2, kemudian dibagi jumlah *scene* dari *talent* tersebut bermain sehingga diperoleh data biaya tiap *talent* dalam satu *scene*. Adapun data biaya tiap *talent* dalam satu *scene* dapat dilihat pada Tabel 4.2.

Tabel 4.2 Data Biaya *Talent* dalam Satu *Scene*

No	<i>Talent</i>	Peran	Biaya <i>Talent</i> tiap <i>Scene</i> (Rp)
1	Daniel Imran	Wawan	27.777,77778
2	A. Muslih Navis	Manto	26.470,58824
3	KimKim	Usman	36.000

Table 4.2 Data Biaya *Talent* dalam Satu *Scene* (Lanjutan)

No	<i>Talent</i>	Peran	Biaya <i>Talent</i> tiap <i>Scene</i> (Rp)
4	Emilda Sohaya	Rita	25.000
5	Aqila Yumi	Nina	43.333,33333
6	Dedi Nala Arung	Ustadz Udas	23.529,41176
7	Ika Puspita	Irah	44.444,44444
8	Nanda Fajar Amrullah	Mali	33.333,33333
9	Muhtadin	Ismail	36.363,63636
10	Muhammad Syabir	Kakek Tua	100.000

Sumber: Lampiran 2

Berdasarkan data biaya *talent* pada Tabel 4.2 dan data *talent* yang bermain di setiap *scene* pada Lampiran 1, selanjutnya dihitung total biaya *talent* yang bermain dalam setiap *scene*. Proses perhitungan total biaya *talent* dapat dijelaskan sebagai berikut.

Pada *scene* 1C, peran *talent* yang digunakan adalah "Irah, Manto, Wawan" sehingga total biaya *talent* pada *scene* 1C = biaya *talent* Irah + biaya *talent* Manto + biaya *talent* Wawan = Rp44.444, 44444 + Rp26.470, 58824 + Rp27.777, 77778 = Rp98.692, 81.

Tabel 4.3 Data Total Biaya *Talent* tiap *Scene*

<i>Scene</i>	Peran yang Bermain	Total Biaya <i>Talent</i> (Rp)
4	Irah, Manto, Wawan	98.692,81
1C	Irah, Manto, Wawan	98.692,81
1D	-	0
1E	-	0
2A	Irah, Manto, Wawan	98.692,81

Tabel 4.3 Data Total Biaya *Talent* tiap *Scene* (Lanjutan)

<i>Scene</i>	Peran yang Bermain	Total Biaya <i>Talent</i> (Rp)
2B	Irah, Manto, Wawan, Ustadz Udas, Ismail, Mali	191.919,2
3	Irah, Manto, Wawan	98.692,81
⋮	⋮	⋮
33B	Wawan, Usman	63.777,78
34	Usman, Wawan, Irah	108.222,2
34B	Wawan, Usman	63.777,78
35A	Usman, Wawan	63.777,78
35B	Usman, Wawan, Rita, Manto, Nina, Mali	191.915
36	Usman, Wawan, Ustadz Udas, Mali, Ismail	157.004,2
37	Ustadz Udas	23.529,41

Sumber: Lampiran 1 dan Lampiran 2

Data biaya transportasi adalah data biaya konsumsi bahan bakar minyak yang dikeluarkan untuk berpindah dari satu *scene* ke *scene* lainnya. Proses syuting film ”Ketika Adzan Sudah Tidak Lagi Berkumandang” menggunakan 3 buah mobil untuk berpindah lokasi. Mobil yang digunakan terdiri atas 2 mobil Innova dan 1 mobil *pick up* Grand Max. Dalam penelitian ini diasumsikan bahwa konsumsi bahan bakar setiap mobil yang digunakan adalah sama per kilomernya. Bahan bakar minyak berjenis Pertalite digunakan oleh ketiga mobil dengan harga sebesar Rp10.0000 per liter. Biaya konsumsi bahan bakar minyak untuk 3 mobil tersebut per kilometer dapat dijelaskan sebagai berikut.

Biaya konsumsi bahan bakar minyak untuk 3 mobil per kilometer = jumlah mobil $\times$ (biaya Pertalite per liter : jarak tempuh mobil per liter) = $3 \times (\text{Rp}10.000/\text{liter} : 8 \text{ km/liter}) = 3 \times (\text{Rp}1.250/\text{km}) = \text{Rp}3.750/\text{km}$.

Jarak lokasi antar *scene* dalam pembuatan film ”Ketika Adzan Sudah Tidak Lagi Berkumandang” dapat ditampilkan pada Tabel 4.4.

Tabel 4.4 Jarak Antar Lokasi tiap Scene dalam Satuan Kilometer

<i>Scene</i>	4	1C	1D	1E	2A	...	34B	35A	35B	37
4	0	0,3	72,1	72,9	72	...	0	0	0,3	0
1C	0,3	0	71,8	73,7	71,7	...	0,3	0,3	0	0,3
1D	72,1	71,8	0	2	0,3	...	72,1	72,1	71,8	72,1
1E	72,9	73,7	2	0	0,5	...	72,9	72,9	73,7	72,9
2A	72	71,7	0,3	0,5	0	...	72	72	71,7	72
2B	72	71,7	0,3	0,5	0	...	72	72	71,7	72
3	0	0,3	72,1	72,9	72	...	0	0	0,3	0
5	0,3	0	71,8	73,7	71,7	...	0,3	0,3	0	0,3
6A	0	0,3	72,1	72,9	72	...	0	0	0,3	0
:	:	:	:	:	:	...	:	:	:	:
35A	0	0,3	72,1	72,9	72	...	0	0	0,3	0
35B	0	0,3	72,1	72,9	72	...	0	0	0,3	0
36	0,3	0	71,8	73,7	71,7	...	0,3	0,3	0	0,3
37	0	0,3	72,1	72,9	72	...	0	0	0,3	0

Sumber: Lampiran 1 dan Google Earth

Berdasarkan Tabel 4.4, dapat dihitung data biaya transportasi dalam pembuatan film "Ketika Adzan Sudah Tidak Lagi Berkumandang" dengan cara mengalikan biaya konsumsi bahan bakar minyak untuk 3 mobil per kilometer dan jarak antar lokasi *scene*. Proses perhitungan biaya transportasi antar *scene* dapat dijelaskan sebagai berikut.

Biaya transportasi dari *scene* 4 ke *scene* 1C = biaya konsumsi bahan bakar minyak untuk 3 mobil per kilometer $\times$ jarak antara lokasi *scene* 4 dan *scene* 1C = $\text{Rp}3.750 \times 0,3 \text{ m} = \text{Rp}1.125$.

Data biaya transportasi antar *scene* dalam pembuatan film "Ketika Adzan Sudah Tidak Lagi Berkumandang" dapat dilihat pada Tabel 4.5.

Tabel 4.5 Data Biaya Transportasi Antar *Scene* dalam Rupiah

<i>Scene</i>	4	1C	1D	1E	2A	...	35B	36	37
4	0	1125	270375	273375	270000	...	0	1125	0
1C	1125	0	269250	276375	268875	...	1125	0	1125
1D	270375	269250	0	7500	1125	...	270375	269250	270375
1E	273375	276375	7500	0	1875	...	273375	276375	273375
2A	270000	268875	1125	1875	0	...	270000	268875	270000
2B	270000	268875	1125	1875	0	...	270000	268875	270000
3	0	1125	270375	273375	270000	...	0	1125	0
5	1125	0	269250	276375	268875	...	1125	0	1125
6A	0	1125	270375	273375	270000	...	0	1125	0
⋮	⋮	⋮	⋮	⋮	⋮	...	⋮	⋮	⋮
34B	270375	269250	0	7500	1125	...	270375	269250	270375
35A	0	1125	270375	273375	270000	...	0	1125	0
35B	0	1125	270375	273375	270000	...	0	1125	0
36	1125	0	269250	276375	268875	...	1125	0	1125
37	0	1125	270375	273375	270000	...	0	1125	0

Berdasarkan data total biaya *talent* tiap *scene* pada Tabel 4.3 dan data biaya transportasi antar *scene* pada Tabel 4.5, selanjutnya dihitung data biaya antar *scene* film ”Ketika Adzan Sudah Tidak Lagi Berkumandang”. Data biaya antar *scene* film ”Ketika Adzan Sudah Tidak Lagi Berkumandang” merupakan hasil penjumlahan antara biaya transportasi dari *scene* ke *scene* lain dan total biaya *talent* pada *scene* tujuan. Proses perhitungan biaya antar *scene* dapat dijelaskan sebagai berikut.

Biaya antara *scene* 4 ke *scene* 1D = biaya transportasi dari *scene* 4 ke *scene* 1D + total biaya *talent* yang bermain pada *scene* 1D = Rp270375+ Rp0 = Rp270375.

Data biaya antar *scene* film ”Ketika Adzan Sudah Tidak Lagi Berkumandang” dapat dilihat pada Tabel 4.6.

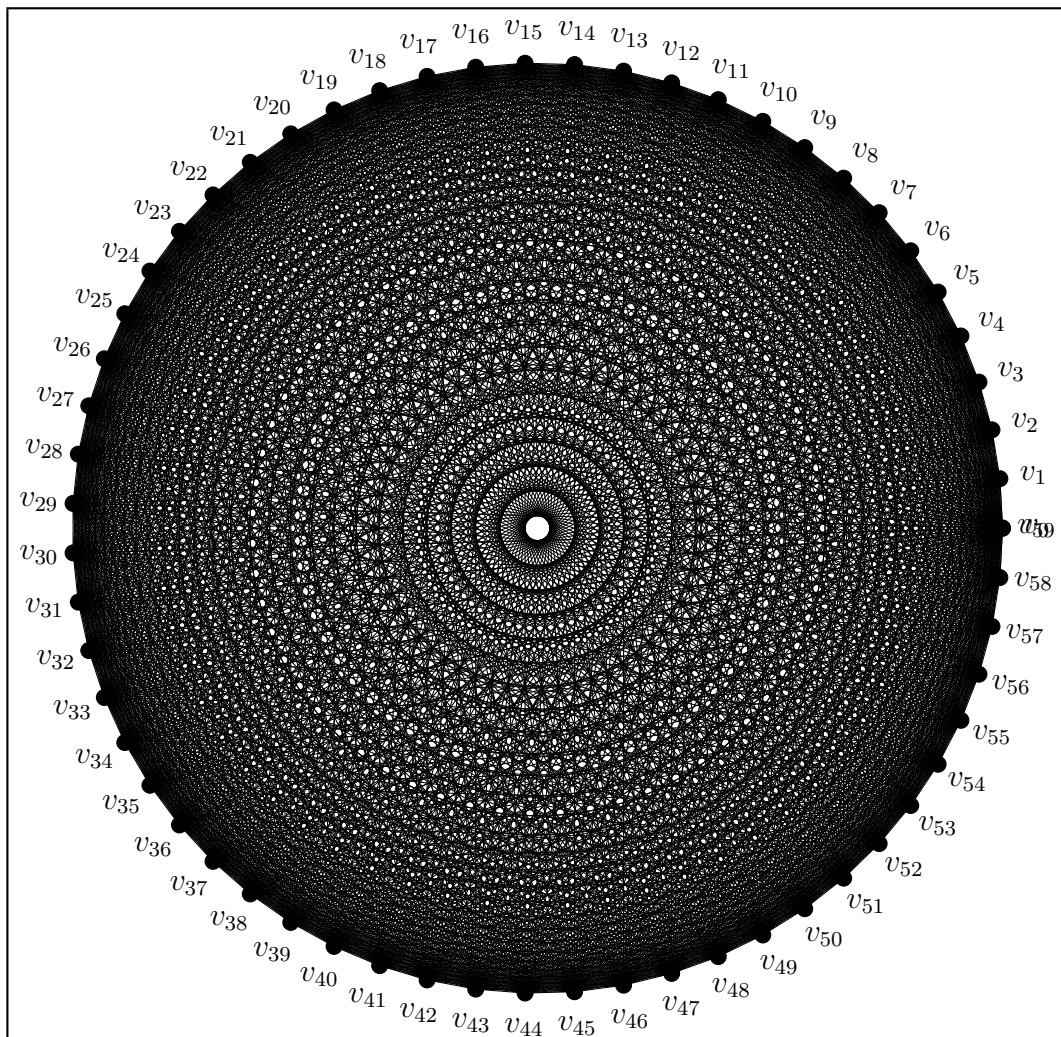
Tabel 4.6 Data Biaya Antar *Scene* dalam Rupiah

<i>Scene</i>	4	1C	1D	1E	2A	...	36	37
4	0	99817,81	270375	273375	368692,81	...	158129,16	23529,41
1C	99817,81	0	269250	276375	367567,81	...	157004,16	24654,41
1D	369067,81	367942,81	0	7500	99817,81	...	426254,16	293904,41
1E	372067,81	375067,81	7500	0	100567,81	...	433379,16	296904,41
2A	368692,81	367567,81	1125	1875	0	...	425879,16	293529,41
2B	368692,81	367567,81	1125	1875	98692,81	...	425879,16	293529,41

Tabel 4.6 Data Biaya Antar *Scene* dalam Rupiah (Lanjutan)

<i>Scene</i>	4	1C	1D	1E	2A	...	36	37
3	98692,81	99817,81	270375	273375	368692,81	...	158129,16	23529,41
5	99817,81	98692,81	269250	276375	367567,81	...	157004,16	24654,41
6A	98692,81	99817,81	270375	273375	368692,81	...	158129,16	23529,41
⋮	⋮	⋮	⋮	⋮	⋮	...	⋮	⋮
34B	369067,81	367942,81	0	7500	99817,81	...	426254,16	293904,41
35A	98692,81	99817,81	270375	273375	368692,81	...	158129,16	23529,41
35B	98692,81	99817,81	270375	273375	368692,81	...	158129,16	23529,41
36	99817,81	98692,81	269250	276375	367567,81	...	0	24654,41
37	98692,81	99817,81	270375	273375	368692,81	...	158129,16	0

Berdasarkan data *scene* pada Tabel 4.1 dan data biaya syuting antar *scene* pada Tabel 4.6, dapat direpresentasikan ke dalam bentuk graf lengkap. Tiap *scene* dalam data *scene* dianalogikan sebagai suatu *vertex* dalam graf. Sedangkan data biaya syuting antar *scene* sebagai *edge* di dalam graf tersebut. Graf yang dapat dibuat untuk memodelkan masalah ini sebagai masalah *traveling salesman problem* dapat dilihat dapat dilihat pada Gambar 4.2.



Gambar 4.2 Graf Lengkap Representasi Permasalahan

Urutan *scene* pada proses syuting akan menentukan total biaya yang dikeluarkan untuk proses syuting tersebut. Konsep *Travel Salesman Problem* berperan dalam menginterpretasikan perhitungan total biaya yang dikeluarkan untuk proses syuting antar *scene* yang telah direpresentasikan ke dalam bentuk graf. Kemudian, algoritma *Particle Swarm Optimization* diterapkan ke dalam konsep *Traveling Salesman Problem* untuk mempermudah perhitungan dalam mencari solusi urutan syuting *scene* optimum.

4.1.2 Model Matematika Konsep *Traveling Salesman Problem* untuk Pemilihan Urutan Syuting *Scene*

Model matematika dari konsep *Traveling Salesman Problem* untuk pemilihan urutan syuting *scene* film "Ketika Adzan Sudah Tidak Lagi Berkumandang" diperoleh berdasarkan representasi model graf pada Gambar 4.2 dan matriks biaya antar *scene* pada Tabel 4.6. Representasi model graf dan matriks biaya antar *scene* dibentuk ke dalam model matematika melalui bentuk *integer programming* pada Persamaan 2.2. Model matematika konsep *Traveling Salesman Problem* untuk pemilihan urutan syuting *scene* film "Ketika Adzan Sudah Tidak Lagi Berkumandang" dapat dituliskan pada Persamaan 5.1 berikut.

Fungsi tujuan

$$\min f(a_{j,k}) = \sum_{i=0}^{59} \sum_{j=0}^{59} a_{j,k}(v_j v_k) \quad (4.1)$$

Fungsi kendala

$$\begin{aligned} \sum_{j=0}^{59} a_{j,k} &= 1, \quad k = 0, 1, \dots, 59 \\ \sum_{k=0}^{59} a_{j,k} &= 1, \quad j = 0, 1, \dots, 59 \\ a_{j,k} &\in \{0, 1\}, \quad j \neq k \end{aligned}$$

Dimana $v_j v_k$ merupakan matriks biaya antar *scene* dari *scene* ke- j menuju *scene* ke- k sesuai Tabel 4.6 dan

$$a_{j,k} = \begin{cases} 0, & \text{jika tidak terjadi perpindahan} \\ 1, & \text{jika terjadi perpindahan dari scene } j \text{ ke } k. \end{cases}$$

Matriks $[a_{j,k}]$ berisi calon solusi dari konsep *Traveling Salesman Problem*. Solusi optimum dari konsep *Traveling Salesman Problem* adalah matriks $[a_{j,k}]$ yang menghasilkan solusi paling minimum dari Persamaan 5.1. Contoh matriks $[a_{j,k}]$ dapat ditampilkan pada Tabel 4.7

Tabel 4.7 Contoh Matriks $[a_{j,k}]$

a	0	1	2	3	4	5	...	54	55	56	57	58	59
0	0	0	0	0	0	0	...	0	0	0	0	0	0
1	0	0	0	0	0	0	...	0	0	0	0	0	0
2	0	0	0	0	0	0	...	0	0	0	0	0	0
3	0	0	0	0	0	0	...	0	0	0	0	0	0
4	0	0	0	0	0	1	...	0	0	0	0	0	0
5	0	0	0	0	0	0	...	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
54	0	0	0	0	0	0	...	0	0	0	0	0	0
55	0	0	0	0	1	0	...	0	0	0	0	0	0
56	0	0	0	0	0	0	...	0	0	0	1	0	0
57	0	0	0	0	0	0	...	1	0	0	0	0	0
58	0	0	0	0	0	0	...	0	0	0	0	0	0
59	0	0	0	0	0	0	...	0	0	1	0	0	0

Contoh matriks $[a_{j,k}]$ pada Tabel 4.7 merepresentasikan urutan indeks syuting *scene* yaitu 0-6-8-9-59-56-57-54-32-33-20-21-7-1-34-58-12-13-14-28-46-47-48-22-27-15-31-40-42-23-24-25-26-45-39-41-43-10-11-17-18-50-51-44-35-29-30-2-36-37-38-49-52-53-55-4-5-19-3-16-0 dengan total biaya antar *scene* sebesar 6.968.984, 48.

Urutan syuting *scene* pada Tabel 4.7 dapat dibaca dengan langkah berikut:

pertama, urutan syuting *scene* awal dimulai dari *scene* 4 dengan indeks 0. Selanjutnya, perhatikan baris dengan indeks 0. Kemudian, diperiksa kolom-kolom pada baris tersebut yang memiliki nilai 1. Indeks pada nama kolom merupakan urutan selanjutnya setelah indeks 0 misal 6 seperti pada Tabel 4.7. Proses selanjutnya, perhatikan baris dengan indeks nama kolom tersebut dan diperiksa kolom-kolom pada baris yang bernilai 1. Langkah-langkah diulangi hingga kembali mendapat indeks awal saat dimulai.

Dengan langkah yang telah dijelaskan, didapatkan urutan syuting *scene* yang menghubungkan simpul-simpul dengan nilai 1 dalam matriks $[a_{j,k}]$. Untuk mendapatkan

urutan syuting *scene* yang optimal dicari menggunakan algoritma *Particle Swarm Optimization*.

4.2 Penerapan Algoritma *Particle Swarm Optimization* (PSO) untuk Optimalisasi Pemilihan Urutan *Scene* Syuting Film "Ketika Adzan Sudah Tidak Lagi Berkumandang"

Algoritma *particle swarm optimization* (PSO) diterapkan ke dalam konsep *Traveling Salesman Problem* (TSP) untuk optimalisasi pemilihan urutan *scene* syuting film "Ketika Adzan Sudah Tidak Lagi Berkumandang". Pencarian solusi yang optimal menggunakan penerapan algoritma PSO berupa urutan *scene* optimal. Penerapan algoritma PSO yang dilakukan dalam penelitian ini yaitu proses normalisasi data biaya antar *scene*, proses algoritma PSO, pengujian modifikasi parameter PSO, dan analisis hasil perhitungan algoritma PSO.

4.2.1 Proses Normalisasi Data

Penerapan algoritma PSO dalam penelitian ini dimulai dengan proses normalisasi data biaya antar *scene* pada Tabel 4.6. Proses normalisasi data ini dilakukan untuk memudahkan interpretasi solusi dalam perhitungan nilai *fitness*. Metode normalisasi data yang digunakan adalah metode *Min-Max Normalization* menggunakan Persamaan 2.17 kemudian nama *scene* disesuaikan mengikuti indeks yang dapat dilihat pada Tabel 4.1. Proses perhitungan *Min-Max Normalization* dapat dijelaskan sebagai berikut:

$$\begin{aligned}\text{Normalisasi biaya scene 4 ke scene 1D} &= \frac{\text{biaya scene 4 ke scene 1D} - \min(\text{biaya scene})}{\max(\text{biaya scene}) - \min(\text{biaya scene})} \\ &= \frac{\text{Rp270.375} - \text{Rp0}}{\text{Rp580.109,48} - \text{Rp0}} \\ &= 0,466076\end{aligned}$$

Hasil proses normalisasi data biaya antar *scene* dapat ditampilkan pada Tabel 4.8 berikut.

Tabel 4.8 Hasil Perhitungan Normalisasi Biaya Antar *Scene*

<i>Scene</i>	0	1	2	3	4	...	57	58	59
0	0	0,17207	0,46608	0,47125	0,63556	...	0,33083	0,27259	0,04056
1	0,17207	0	0,46414	0,47642	0,63362	...	0,33277	0,27065	0,04250
2	0,63620	0,63426	0	0,01293	0,17207	...	0,79690	0,73478	0,50664

Tabel 4.8 Hasil Perhitungan Normalisasi Biaya Antar *Scene* (Lanjutan)

<i>Scene</i>	0	1	2	3	4	...	57	58	59
3	0,64138	0,64655	0,01293	0	0,17336	...	0,80207	0,74706	0,51181
4	0,63556	0,63362	0,00194	0,00323	0	...	0,79626	0,73414	0,50599
5	0,63556	0,63362	0,00194	0,00323	0,17013	...	0,79626	0,73414	0,50599
6	0,17013	0,17207	0,46608	0,47125	0,63556	...	0,33083	0,27259	0,04056
7	0,17207	0,17013	0,46414	0,47642	0,63362	...	0,33277	0,27065	0,04250
8	0,17013	0,17207	0,46608	0,47125	0,63556	...	0,33083	0,27259	0,04056
⋮	⋮	⋮	⋮	⋮	⋮	...	⋮	⋮	⋮
55	0,63620	0,63426	0	0,01293	0,17207	...	0,79690	0,73478	0,50664
56	0,17013	0,17207	0,46608	0,47125	0,63556	...	0,33083	0,27259	0,04056

Data biaya antar *scene* yang telah dinormalisasi ini digunakan sebagai data masukan dalam proses optimalisasi pemilihan urutan *scene* syuting film "Ketika Adzan Sudah Tidak Lagi Berkumandang" menggunakan algoritma PSO.

4.2.2 Proses Algoritma PSO

Penyelesaian urutan *scene* syuting film "Ketika Adzan Sudah Tidak Lagi Berkumandang" dalam penelitian ini menggunakan algoritma PSO. Proses perhitungan algoritma PSO untuk optimalisasi urutan *scene* syuting film "Ketika Adzan Sudah Tidak Lagi Berkumandang" dapat dijelaskan sebagai berikut:

1. Inisialisasi nilai parameter PSO, kecepatan (*velocity*) awal (V_j^0) dan posisi awal (X_j^0) sehingga diperoleh rute awal.

- (a) Inisialisasi nilai parameter PSO yang akan digunakan.

Ukuran *swarm*: $N = 50$

Iterasi maksimum: $n = 100$

Learning rates: $c_1 = c_2 = 2$

Inertia Weight: $\omega^0 = 0$

Dimensi sebesar jumlah seluruh *scene* yang ada dikurangi 1 bertujuan untuk mengeluarkan *scene* keberangkatan dari perhitungan. *Scene* awal yang digunakan adalah *scene* 4 dengan indeks 0, sehingga dimensi dari setiap partikel sebesar 59. Ukuran *swarm* sebesar 50 berdasarkan interval ukuran *swarm* pada algoritma PSO klasik (Piotrowski dkk., 2020). Nilai *learning rates* yang terdiri dari c_1 dan c_2 sebesar 2 agar arah pergerakan partikel condong ke arah rata-rata P_{best} dan G_{best} (Jain dkk., 2022). Strategi *Ran-*

dom *Inertia Weight* diterapkan ke dalam proses perhitungan, sehingga untuk kondisi awal nilai *inertia weight* dimisalkan $\omega^0 = 0$.

- (b) Kecepatan (*velocity*) awal dan posisi awal dibangkitkan secara acak dengan penerapan strategi pembatasan (*Clamping*).

Jumlah baris yang dibangkitkan sama dengan ukuran *swarm* dan jumlah kolom sama dengan ukuran dimensi. Posisi awal dibangkitkan menggunakan Persamaan 2.12 dengan *clamping* minimum $X_{min} = -10$ dan posisi maksimum $X_{max} = 10$. Kecepatan awal dibangkitkan menggunakan Persamaan 2.11 dengan *velocity clamping* pada Persamaan 2.6.

Misal $\varepsilon = 0.2$, maka

$$V_{max} = \varepsilon(X_{max} - X_{min})$$

$$V_{max} = 0.2(10 - (-10))$$

$$V_{max} = 4$$

Sehingga diperoleh *velocity* maksimum sebesar 4, sedangkan untuk *velocity* minimum $V_{min} = -V_{max} = -4$. *Velocity* awal dapat ditampilkan pada Tabel 4.9 dan Lampiran 6, sedangkan posisi awal dapat ditampilkan pada Tabel 4.10 dan Lampiran 7.

Tabel 4.9 *Velocity* Awal

V_j^0	1	2	3	4	5	...	57	58	59
V_1^0	-2,16528	0,89027	3,46434	1,35812	-1,48245	...	-3,4891	-2,64855	-0,91931
V_2^0	3,99737	-3,61299	0,91479	-2,6657	-0,46566	...	3,21946	2,77825	2,37025
V_3^0	3,82534	-0,90769	1,02992	0,9915	2,5288	...	-0,44853	-0,54852	-1,38243
V_4^0	2,45539	-3,90966	-0,36888	1,77577	-3,87413	...	0,41641	0,13947	-0,38577
V_5^0	1,82912	0,6222	-0,6398	-3,87109	-0,39123	...	-0,59685	-0,54466	-1,95952
V_6^0	-3,02692	2,62879	-0,6713	-0,95287	-0,91637	...	-1,32162	2,45119	-0,44718
V_7^0	3,6896	-1,76421	0,35196	-2,10728	2,85127	...	-1,39565	1,97397	3,24564
V_8^0	0,51646	1,01958	-0,19885	1,09131	-1,02176	...	-3,08582	0,76127	-3,03922
V_9^0	1,30571	-0,43843	3,86682	1,5284	1,27028	...	0,03108	-1,71748	1,32427
V_{10}^0	-2,41937	1,76656	-2,86669	0,10994	2,50789	...	1,51926	2,76962	-3,80081
...	...	...	...	...	...	...	...	...	...
V_{46}^0	-0,90611	1,43487	-0,68767	-2,85052	3,83716	...	0,74573	0,6788	2,55445
V_{47}^0	2,24512	2,69909	2,21367	-3,33426	-1,91667	...	1,97714	2,21482	-2,98005
V_{48}^0	-1,72583	-1,07504	3,03866	1,21924	-3,11848	...	3,59674	3,75702	2,86491
V_{49}^0	2,44165	2,34375	-1,13627	1,58296	3,67684	...	-2,79745	1,35159	-0,20271
V_{50}^0	-3,18509	0,60991	3,78284	0,68927	2,50309	...	-0,82081	2,08809	1,98133

Tabel 4.10 Posisi Awal

X_j^0	1	2	3	4	5	...	57	58	59
X_1^0	0,05399	8,84622	-9,53092	-1,22655	-1,56802	...	7,74551	-7,5931	2,54871
X_2^0	-2,96541	-3,76962	5,54813	-1,59419	-5,03711	...	6,22599	-2,35698	0,48148
X_3^0	-9,64862	-0,97641	2,24335	-0,88499	-3,99483	...	8,18512	-0,27442	6,13968
X_4^0	2,53456	-6,57045	7,78262	-3,89194	5,48713	...	1,20523	3,0871	-3,89433
X_5^0	3,90363	3,71686	1,81943	-5,60334	-2,97723	...	9,32497	-3,54684	-0,50911
X_6^0	5,05232	8,08277	-9,75775	-1,11905	1,70159	...	5,37216	6,36173	-9,81629
X_7^0	-2,56848	3,96685	-5,91593	-9,33513	8,27528	...	-7,67784	7,05032	6,14002
X_8^0	-3,25428	4,89507	8,90235	7,99547	9,21897	...	-5,19454	-3,30457	-0,62536
X_9^0	-8,15918	-1,86934	-0,10474	-0,71733	5,3006	...	1,73662	0,87569	-0,8173
X_{10}^0	8,11716	-3,99829	4,98738	7,00341	2,18338	...	-5,26742	7,21959	-9,0677
...	...	...	...	...	...	...	...	...	...
X_{46}^0	-9,96221	-0,29616	-8,17018	7,62301	-2,06952	...	0,16774	8,57348	-3,57204
X_{47}^0	8,40066	-7,18732	-9,67606	-2,88435	9,42831	...	1,15472	0,29264	-8,42359
X_{48}^0	-2,48912	-2,91492	-6,04942	-6,83251	9,87979	...	-6,05294	3,74278	7,37851
X_{49}^0	7,00152	5,15928	8,85317	8,35978	-7,69392	...	-5,68648	3,02622	1,67261
X_{50}^0	3,5536	9,95634	1,67159	-1,5621	5,15217	...	-8,97304	1,06679	-8,8946

Strategi *clamping* pada posisi digunakan untuk membatasi posisi agar tidak keluar dari area pencarian. Sedangkan strategi *clamping* pada *velocity* digunakan untuk membatasi *velocity* agar partikel tidak bergerak terlalu cepat atau terlalu lambat.

- (c) Posisi awal dari setiap partikel diberikan indeks sesuai kolom dan diurutkan berdasarkan nilai posisi terkecil sehingga diperoleh urutan indeks *scene* syuting awal yang ditampilkan pada Tabel 4.11 dan Lampiran 8.

Tabel 4.11 Rute Awal

X_j^0	Rute
X_1^0	0-33-24-3-42-5-52-11-8-40-55-56-57-31-43-15-54-27-19-41-18-21-6-4-7-10-48-13-39-51-47-44-23-46-38-22-16-59-45-9-50-36-53-29-25-35-26-1-30-14-37-49-20-17-12-2-34-28-58-32-0
X_2^0	0-50-20-2-58-4-33-46-47-28-41-57-55-37-25-26-42-24-40-21-6-27-15-31-45-56-22-3-16-38-11-52-23-12-39-43-7-9-36-29-51-30-53-59-8-34-14-32-49-1-18-10-44-17-48-54-35-19-5-13-0
X_3^0	0-58-15-23-25-49-50-32-54-13-41-3-12-2-33-38-24-11-20-46-47-14-9-51-56-22-31-7-30-44-57-39-8-1-10-29-45-37-35-48-43-19-42-53-16-59-5-34-18-26-28-27-52-17-36-6-4-40-55-21-0
X_4^0	0-10-31-50-40-36-39-19-18-43-44-57-7-53-4-27-9-59-20-5-28-48-56-42-37-11-17-13-24-58-14-29-2-34-21-8-47-35-55-45-54-25-49-30-52-38-15-22-46-1-23-16-12-33-51-6-3-26-32-41-0
X_5^0	0-20-27-33-32-31-52-28-6-19-22-3-17-40-9-2-30-38-26-45-56-35-54-5-37-36-10-7-24-11-23-55-57-18-21-58-39-15-34-46-1-8-53-41-16-47-48-4-13-43-51-12-14-29-25-44-59-42-49-50-0

Tabel 4.11 Rute Awal (Lanjutan)

X_j^0	Rute
X_6^0	0-56-24-12-10-42-46-15-21-17-39-26-16-18-4-27-11-32-43-38-20-51-40-58-45-1-36-47-44-48-30-31-23-13-9-41-8-19-33-57-28-29-25-59-54-35-5-49-53-22-6-7-3-34-50-55-2-37-14-52-0
X_7^0	0-37-33-28-40-59-21-14-47-53-32-4-17-42-7-29-8-5-3-16-22-13-52-34-46-41-30-45-20-11-1-51-15-23-2-10-31-57-56-43-39-19-6-58-36-12-24-35-25-44-26-55-48-54-38-49-18-9-50-27-0
X_8^0	0-25-53-35-23-55-32-8-28-36-19-52-48-5-37-43-34-6-45-38-49-10-4-54-30-57-27-51-9-50-17-59-1-47-3-42-33-58-20-2-11-41-16-13-21-31-39-7-15-44-24-18-12-56-26-40-46-29-14-22-0
X_9^0	0-4-57-55-29-13-21-7-23-26-52-25-39-15-27-44-16-36-34-19-37-10-58-8-32-20-5-28-3-1-49-43-12-6-11-40-53-56-48-47-35-31-46-2-38-45-33-17-22-14-18-59-24-42-50-51-54-9-41-30-0
X_{10}^0	0-30-46-51-38-35-11-1-54-29-22-7-33-56-55-20-10-5-48-45-50-49-44-15-26-37-13-12-31-58-14-52-24-43-17-57-40-59-21-19-2-8-36-16-32-41-34-28-4-25-27-9-53-23-47-42-3-18-6-39-0
...	...
X_{46}^0	0-1-32-41-38-7-54-40-34-3-14-59-15-10-52-27-31-28-8-43-9-36-49-44-53-12-20-39-19-18-58-33-55-24-45-51-50-35-6-11-26-57-16-56-30-2-22-25-42-21-23-29-4-47-48-17-13-5-37-46-0
X_{47}^0	0-39-34-23-47-13-56-46-3-29-53-25-44-21-9-15-24-16-48-28-6-43-17-7-57-42-32-38-1-8-19-52-33-18-12-49-22-2-26-11-36-4-30-20-50-45-51-35-55-5-14-31-37-40-10-27-59-58-54-41-0
X_{48}^0	0-2-44-54-20-34-6-39-59-40-36-29-7-4-46-51-31-3-33-43-41-10-25-15-24-27-22-1-56-17-9-32-16-47-57-18-21-19-37-48-45-58-14-49-26-38-52-23-53-30-50-55-5-12-35-28-11-8-13-42-0
X_{49}^0	0-6-21-46-10-13-42-18-39-32-34-14-8-7-40-36-52-28-37-30-35-4-31-9-23-41-57-2-59-54-27-19-53-50-1-56-51-3-48-20-11-25-24-44-29-33-38-43-5-16-49-15-26-47-45-58-17-22-55-12-0
X_{50}^0	0-26-11-7-2-50-28-24-27-34-21-55-54-45-48-1-56-22-19-46-20-3-25-43-44-15-53-59-10-29-51-58-52-18-49-57-33-12-23-42-40-9-16-39-37-38-6-4-36-47-32-5-35-8-17-31-41-13-30-14-0

2. Mengevaluasi nilai fungsi *fitness* dari hasil rute setiap partikel.

Nilai fungsi *fitness* didapatkan setelah menghitung total biaya syuting dari hasil rute setiap partikel. Proses perhitungan total biaya syuting untuk partikel pertama (X_1^0) dapat dijelaskan sebagai berikut.

Hasil rute dari partikel X_1^0 pada Tabel 4.11, yaitu

$$\begin{aligned} \text{Rute } X_1^0 = & [0 - 33 - 24 - 3 - 42 - 5 - 52 - 11 - 8 - 40 - 55 - 56 - 57 - \\ & 31 - 43 - 15 - 54 - 27 - 19 - 41 - 18 - 21 - 6 - 4 - 7 - 10 - \\ & 48 - 13 - 39 - 51 - 47 - 44 - 23 - 46 - 38 - 22 - 16 - 59 - \\ & 45 - 9 - 50 - 36 - 53 - 29 - 25 - 35 - 26 - 1 - 30 - 14 - 37 - \\ & 49 - 20 - 17 - 12 - 2 - 34 - 28 - 58 - 32 - 0] \end{aligned}$$

Kemudian, dihitung total biaya syuting antar *scene* yang dilalui oleh partikel X_1^0 menggunakan biaya antar *scene* yang telah dinormalisasi pada Tabel 4.8.

$$\begin{aligned}
f(X_1^0) = & 0,08598 + 0,27337 + 0,4706 + 0,64138 + 0,31822 + 0,19656 + 0,40119 \\
& + 0,27259 + 0,35487 + 0,0 + 0,11317 + 0,57796 + 0,73145 + 0,2223 \\
& + 0,27624 + 0,12581 + 0,52875 + 0,16656 + 0,04788 + 0,35637 + 0,80596 \\
& + 0,17013 + 0,73815 + 0,32873 + 0,27253 + 0,17207 + 0,61511 + 0,51267 \\
& + 0,34059 + 0,0431 + 0,78106 + 0,46414 + 0,46414 + 0,31628 + 0,633 \\
& + 0,00323 + 0,85899 + 0,50664 + 0,13601 + 0,11188 + 0,19721 + 0,57796 \\
& + 0,13661 + 0,27882 + 0,9124 + 0,22065 + 0,10994 + 0,52002 + 0,57408 \\
& + 0,46414 + 0,3507 + 0,57472 + 0,61511 + 0,39796 + 0,4168 + 0,23357 \\
& + 0,47642 + 0,60015 + 0,08598 + 0,2561 + \\
& = 22,43496
\end{aligned}$$

dari total biaya syuting *scene* partikel X_1^0 , dihitung nilai *fitness* menggunakan Persamaan 2.13.

$$F(X_1^0) = \frac{1}{f(X_1^0)} = \frac{1}{22,43496} = 0,04457$$

Proses evaluasi total biaya syuting untuk partikel lainnya dihitung menggunakan bahasa pemrograman Python seperti yang ditampilkan pada Tabel 4.12 dan Lampiran 9.

Tabel 4.12 Total Biaya Syuting dan Nilai *Fitness* tiap Partikel

X_j^0	Biaya Antar <i>Scene</i>						Total Biaya	<i>Fitness</i>
X_1^0	0,08598	0,27337	0,4706	...	0,08598	0,2561	22,43496	0,04457
X_2^0	0,24155	0,11317	0,52811	...	0,33196	0,2561	20,7439	0,04821
X_3^0	0,25419	0,0431	0,25416	...	0,58636	0,6362	23,60694	0,04236
X_4^0	0,22258	0,32873	0,0	...	0,73815	0,2561	25,54817	0,03914
X_5^0	0,22258	0,7365	0,47642	...	0,4168	0,17013	20,6715	0,04838
X_6^0	0,04056	0,47125	0,51848	...	0,57602	0,17013	25,4693	0,03926
X_7^0	0,73714	0,64138	0,63556	...	0,00323	0,2574	23,01093	0,04346
X_8^0	0,35264	0,7365	0,5137	...	0,62035	0,2561	23,26304	0,04299
X_9^0	0,57602	0,31628	0,13661	...	0,27337	0,2561	19,80464	0,05049
X_{10}^0	0,35934	0,1476	0,13752	...	0,248892	0,156954	21,37886	0,04678
...	...	...	...	...	...	...	...	...
X_{46}^0	0,17207	0,08404	0,11317	...	0,26667	0,2561	21,67605	0,04613
X_{47}^0	0,24155	0,4706	0,47642	...	0,04788	0,2561	22,94241	0,04359
X_{48}^0	0,35934	0,12258	0,2223	...	0,33277	0,63556	24,62054	0,04062
X_{49}^0	0,60269	0,10994	0,8218	...	0,0	0,2561	24,62959	0,0406
X_{50}^0	0,31393	0,42657	0,4168	...	0,47642	0,6362	24,28698	0,04117

3. Menentukan P_{best} dan G_{best} awal.

- (a) P_{best} merupakan posisi dengan nilai *fitness* terbaik yang pernah dicapai oleh setiap partikel. Pada iterasi pertama, setiap partikel hanya pernah bergerak satu kali yaitu pada posisi awal yang telah ditentukan. Sehingga posisi awal tersebut menjadi posisi dengan nilai *fitness* terbaik yang pernah dicapai oleh partikel tersebut. Akibatnya, P_{best} awal untuk setiap partikel sama dengan posisi awal dari setiap partikel. P_{best} dari setiap partikel dapat dilihat pada Tabel 4.13.

Tabel 4.13 P_{best} Iterasi ke-0

$P_{best,j}^0$	1	2	3	...	58	59	<i>Fitness Value</i>
$P_{best,1}^0$	0,05399	8,84622	-9,53092	...	-7,5931	2,54871	0,04457
$P_{best,2}^0$	-2,96541	-3,76962	5,54813	...	-2,35698	0,48148	0,04821
$P_{best,3}^0$	-9,64862	-0,97641	2,24335	...	-0,27442	6,13968	0,04236
$P_{best,4}^0$	2,53456	-6,57045	7,78262	...	3,0871	-3,89433	0,03914
$P_{best,5}^0$	3,90363	3,71686	1,81943	...	-3,54684	-0,50911	0,04838
$P_{best,6}^0$	5,05232	8,08277	-9,75775	...	6,36173	-9,81629	0,03926
$P_{best,7}^0$	-2,56848	3,96685	-5,91593	...	7,05032	6,14002	0,04346
$P_{best,8}^0$	-3,25428	4,89507	8,90235	...	-3,30457	-0,62536	0,04299
$P_{best,9}^0$	-8,15918	-1,86934	-0,10474	...	0,87569	-0,8173	0,05049
$P_{best,10}^0$	8,11716	-3,99829	4,98738	...	7,21959	-9,0677	0,04235
...	...	...	...	...	...	...	...
$P_{best,46}^0$	-9,96221	-0,29616	-8,17018	...	8,57348	-3,57204	0,04613
$P_{best,47}^0$	8,40066	-7,18732	-9,67606	...	0,29264	-8,42359	0,04359
$P_{best,48}^0$	-2,48912	-2,91492	-6,04942	...	3,74278	7,37851	0,04062
$P_{best,49}^0$	7,00152	5,15928	8,85317	...	3,02622	1,67261	0,0406
$P_{best,50}^0$	3,5536	9,95634	1,67159	...	1,06679	-8,8946	0,04117

- (b) G_{best} merupakan posisi dengan nilai *fitness* paling terbaik yang pernah dicapai di antara seluruh partikel pada semua iterasi. Pada iterasi pertama, semua partikel hanya bergerak satu kali yaitu pada posisi awal. Sehingga P_{best} dari seluruh partikel merupakan posisi awal setiap partikel. Akibatnya G_{best} awal ditentukan dari P_{best} dengan nilai *fitness* paling terbaik di antara semua partikel pada iterasi pertama. Pada Tabel 4.13 dapat dilihat bahwa nilai *fitness* paling terbaik di antara semua partikel adalah sebesar 0,05049 yang dimiliki oleh $P_{best,9}^0$. Posisi dari partikel $P_{best,9}^0$ dipilih menjadi G_{best} awal yaitu

$$G_{best}^0 = [-8,15918 \quad -1,86934 \quad -0,10474 \quad \dots \quad 0,87569 \quad -0,8173]$$

Kemudian G_{best} dapat ditampilkan dalam Tabel 4.14 berikut.

Tabel 4.14 G_{best} Iterasi ke-0

G_{best}^i	1	2	3	...	58	59	Total Biaya Syuting	Fitness Value
G_{best}^0	-8,15918	-1,86934	-0,10474	...	0,87569	-0,8173	19,80464	0,05049

G_{best} merupakan solusi optimal yang diperoleh menggunakan algoritma PSO. Agar dapat melihat solusi berupa total biaya syuting yang optimal dari G_{best} , yaitu dengan mencari rute yang dihasilkan dari posisi pada G_{best} dan menghitung total biaya syuting menggunakan rute tersebut. Sedangkan untuk melihat solusi berupa urutan *scene* syuting yang optimal dari G_{best} , yaitu dengan mencari rute yang dihasilkan dari posisi pada G_{best} dan mengubah indeks rute menjadi nama *scene* menggunakan indeks *scene* pada Tabel 4.1. Sebagai contoh, rute optimal pada iterasi awal adalah sebagai berikut.

$$\begin{aligned} \text{Rute } G_{best}^0 = & [0 - 29 - 36 - 49 - 1 - 33 - 15 - 45 - 40 - 42 - 38 - 54 \\ & - 41 - 6 - 51 - 21 - 16 - 2 - 37 - 19 - 18 - 44 - 32 \\ & - 35 - 22 - 50 - 46 - 59 - 30 - 4 - 3 - 47 - 58 - 13 \\ & - 56 - 8 - 57 - 34 - 52 - 31 - 23 - 39 - 26 - 17 - 10 \\ & - 7 - 5 - 43 - 24 - 55 - 20 - 27 - 48 - 9 - 25 - 14 \\ & - 11 - 53 - 12 - 28 - 0] \end{aligned}$$

Selanjutnya, dihitung total biaya syuting dari rute G_{best} menggunakan biaya antar *scene* yang telah dinormalisasi pada Tabel 4.8.

$$\begin{aligned}
f(G_{best}^0) = & 0.57602 + 0.31628 + 0.13661 + 0.63426 + 0.43599 + 0.35252 + 0.00323 \\
& + 0.11317 + 0.10994 + 0.85899 + 0.65263 + 0.35264 + 0.2561 + 0.30827 \\
& + 0.4113 + 0.74231 + 0.01293 + 0.31628 + 0.15097 + 0.99418 + 0.01034 \\
& + 0.52261 + 0.0 + 0.3574 + 0.1476 + 0.05408 + 0.12654 + 0.46608 \\
& + 0.17207 + 0.00323 + 0.60721 + 0.35468 + 0.13192 + 0.19592 \\
& + 0.13407 + 0.33083 + 0.00194 + 0.57408 + 0.57796 + 0.17144 \\
& + 0.32873 + 0.10994 + 0.35637 + 0.32491 + 0.21811 + 0.79432 \\
& + 0.73145 + 0.04788 + 0.57472 + 0.61511 + 0.23961 + 0.0 + 0.3999 \\
& + 0.20856 + 0.34059 + 0.27083 + 0.57472 + 0.46478 + 0.27337 + 0.2561 + \\
& = 19,80464
\end{aligned}$$

Total biaya syuting rute G_{best} masih dalam bentuk yang telah dinormalisasi. Agar dapat diperoleh total biaya syuting rute G_{best} dalam Rupiah digunakan Persamaan 2.17.

$$\begin{aligned}
\text{biaya syuting} &= f(G_{best}^0) \cdot (\max(\text{biaya scene}) - \min(\text{biaya scene})) + \min(\text{biaya scene}) \\
&= 19,80464 \cdot (\text{Rp}580.109,48 - \text{Rp}0) + \text{Rp}0 \\
&= \text{Rp}11.488.859.41199
\end{aligned}$$

Kemudian, diubah indeks *scene* pada rute G_{best} agar didapatkan urutan *scene* syuting yaitu 4-19-22C-31-1C-21B-10-29B-25-27-23B-34-26-3-32B-14B-11-1D-23A-13-12B-29A-21A-22B-15-32A-30A-37-19B-2A-1E-30B-36-9A-35A-6A-35B-22A-33A-20-16A-24-16D-12A-7A-5-2B-28-16B-34B-14A-17-30C-6B-16C-9B-7B-33B-8-18-4.

Karena tahap inisialisasi termasuk ke dalam iterasi $i = 0$ dan iterasi kurang dari iterasi maksimum sehingga iterasi dilanjutkan.

Iterasi ke-1 ($i = 1$)

4. Membangkitkan nilai *inertia weight* berdasarkan strategi *random inertia weight* dengan menggunakan Persamaan 2.5.

Misal $r = U(0, 1) = 0,38900$, maka

$$\begin{aligned}
\omega^1 &= 0,5 + \frac{r}{2} \\
&= 0,5 + \frac{0,38900}{2} \\
&= 0,69450
\end{aligned}$$

Sehingga diperoleh nilai *inertia weight* untuk iterasi ke-1 sebesar 0,69450. Nilai *inertia weight* selalu dibangkitkan di setiap iterasi menggunakan nilai r acak berdistribusi *uniform* $U(0, 1)$.

5. Memperbarui *velocity* setiap partikel menggunakan Persamaan 2.9 dengan menerapkan strategi *clamping* yang telah ditetapkan pada langkah inisialisasi.

Nilai dari r_1 dan r_2 dibangkitkan secara acak berdistribusi *uniform* $U(0, 1)$.

Misal $r_1 = 0,82681$ dan $r_2 = 0,93175$, maka

$$\begin{aligned} V_j^i &= \omega^i V_j^{i-1} + c_1 r_1^i (P_{best,j}^{i-1} - X_j^{i-1}) + c_2 r_2^i (G_{best} - X_j^{i-1}) \\ V_j^1 &= \omega^1 V_j^0 + c_1 r_1^1 (P_{best,j}^0 - X_j^0) + c_2 r_2 (G_{best} - X_j^0) \\ V_j^1 &= (0,69450) V_j^0 + (2)(0,82681) (P_{best,j}^0 - X_j^0) \\ &\quad + (2)(0,93175) (G_{best} - X_j^0) \\ V_j^1 &= (0,69450) V_j^0 + (1,65362) (P_{best,j}^0 - X_j^0) + (1,86350) (G_{best} - X_j^0) \end{aligned} \quad (4.2)$$

Kemudian, pembaruan *velocity* untuk semua partikel pada iterasi 1 dilakukan menggunakan Persamaan 4.2. Proses pembaruan *velocity* untuk partikel pertama dapat dituliskan sebagai berikut.

$$\begin{aligned} V_1^1 &= (0,69450) V_j^0 + (1,65362) (P_{best,j}^0 - X_j^0) + (1,86350) (G_{best} - X_j^0) \\ &= (0,69450)[-2,16528 \ 0,89027 \ 3,46434 \ \dots \ -0,91931] \\ &\quad + (1,65362)([0,05399 \ 8,84622 \ -9,53092 \ \dots \ 2,54871]) \\ &\quad - [0,05399 \ 8,84622 \ -9,53092 \ \dots \ 2,54871]) \\ &\quad + (1,86350)([-8,15918 \ -1,86934 \ -0,10474 \ \dots \ -0,8173] \\ &\quad - [0,05399 \ 8,84622 \ -9,53092 \ \dots \ 2,54871]) \\ &= [-16,809 \ -19,3502 \ 19,97167 \ \dots \ -6,91102] \end{aligned}$$

Strategi *clamping* pada Persamaan 2.14 diterapkan ke dalam hasil pembaruan *velocity* setiap partikel. Hasil pembaruan *velocity* untuk partikel pertama pada iterasi 1 terdapat nilai yang melebihi *clamping* sehingga diterapkan Persamaan 2.14 pada nilai tersebut.

$$V_1^1 = [-4 \ -4 \ 4 \ \dots \ -4]$$

Pembaruan *velocity* untuk keseluruhan partikel dapat dilihat pada Tabel 4.15 dan Lampiran 10.

Tabel 4.15 *Velocity* pada Iterasi ke-1

V_j^1	1	2	3	4	5	...	57	58	59
V_1^1	-4,0	-4,0	4,0	1,89215	4,0	...	-4,0	4,0	-4,0
V_2^1	-4,0	1,03195	-4,0	-0,2173	4,0	...	-4,0	4,0	-0,77414
V_3^1	4,0	-2,29437	-3,66039	1,00103	4,0	...	-4,0	1,76228	-4,0
V_4^1	-4,0	4,0	-4,0	4,0	-3,03818	...	1,27944	-4,0	4,0
V_5^1	-4,0	-4,0	-4,0	4,0	4,0	...	-4,0	4,0	-1,9352
V_6^1	-4,0	-4,0	4,0	0,08684	4,0	...	-4,0	-4,0	4,0
V_7^1	-4,0	-4,0	4,0	4,0	-3,56311	...	4,0	-4,0	-4,0
V_8^1	-4,0	-4,0	-4,0	-4,0	-4,0	...	4,0	4,0	-2,46842
V_9^1	0,90682	-0,30449	2,68551	1,06147	0,88221	...	0,02159	-1,19279	0,91971
V_{10}^1	-4,0	4,0	-4,0	-4,0	4,0	...	4,0	-4,0	4,0
...	...	...	...	...	...	...	...	...	...
V_{46}^1	2,73065	-1,9351	4,0	-4,0	4,0	...	3,44152	-4,0	4,0
V_{47}^1	-4,0	4,0	4,0	1,7226	-4,0	...	2,45749	2,62471	4,0
V_{48}^1	-4,0	1,20182	4,0	4,0	-4,0	...	4,0	-2,73357	-4,0
V_{49}^1	-4,0	-4,0	-4,0	-4,0	4,0	...	4,0	-3,06883	-4,0
V_{50}^1	-4,0	-4,0	-0,68301	2,05293	2,015	...	4,0	1,09406	4,0

6. Memperbarui posisi setiap partikel menggunakan Persamaan 2.10 dengan menerapkan strategi *clamping* yang telah ditetapkan pada langkah inisialisasi. Persamaan untuk memperbarui posisi partikel pertama sampai dengan terakhir pada iterasi ke-1 dapat dituliskan sebagai Persamaan 4.3.

$$\begin{aligned} X_j^i &= X_j^{i-1} + V_j^i \\ X_j^1 &= X_j^0 + V_j^1 \end{aligned} \quad (4.3)$$

Kemudian, pembaruan posisi untuk semua partikel pada iterasi ke-1 dilakukan menggunakan Persamaan 4.3. Proses pembaruan posisi partikel pertama pada iterasi ke-1 dapat ditampilkan sebagai berikut.

$$\begin{aligned} X_j^1 &= X_j^0 + V_j^1 \\ X_1^1 &= [0,05399 \ 8,84622 \ -9,53092 \ \dots \ 6,03494 \ \dots \ 2,54871] \\ &\quad + [-4 \ -4 \ 4 \ \dots \ 4 \ \dots \ -4] \\ &= [-3,94601 \ 4,84622 \ -5,53092 \ \dots \ 10,03494 \ \dots \ -1,45129] \end{aligned}$$

Hasil pembaruan posisi untuk partikel pertama pada iterasi ke-1 terdapat nilai yang melebihi *clamping* sehingga diterapkan Persamaan 2.15 pada nilai tersebut.

$$X_1^1 = [-3,94601 \ 4,84622 \ -5,53092 \ \dots \ 10 \ \dots \ -1,45129]$$

Pembaruan posisi untuk keseluruhan partikel pada iterasi ke-1 dapat dilihat pada Tabel 4.16 dan Lampiran 11.

Tabel 4.16 Posisi pada Iterasi ke-1

X_j^1	1	2	3	4	5	...	57	58	59
X_1^1	-3,94601	4,84622	-5,53092	0,6656	2,43198	...	3,74551	-3,5931	-1,45129
X_2^1	-6,96541	-2,73767	1,54813	-1,81149	-1,03711	...	2,22599	1,64302	-0,29266
X_3^1	-5,64862	-3,27078	-1,41704	0,11604	0,00517	...	4,18512	1,48786	2,13968
X_4^1	-1,46544	-2,57045	3,78262	0,10806	2,44895	...	2,48467	-0,9129	0,10567
X_5^1	-0,09637	-0,28314	-2,18057	-1,60334	1,02277	...	5,32497	0,45316	-2,44431
X_6^1	1,05232	4,08277	-5,75775	-1,03221	5,70159	...	1,37216	2,36173	-5,81629
X_7^1	-6,56848	-0,03315	-1,91593	-5,33513	4,71217	...	-3,67784	3,05032	2,14002
X_8^1	-7,25428	0,89507	4,90235	3,99547	5,21897	...	-1,19454	0,69543	-3,09378
X_9^1	-7,25236	-2,17383	2,58077	0,34414	6,18281	...	1,75821	-0,3171	0,10241
X_{10}^1	4,11716	0,00171	0,98738	3,00341	6,18338	...	-1,26742	3,21959	-5,0677
...	...	...	...	...	...	...	...	...	...
X_{46}^1	-7,23156	-2,23126	-4,17018	3,62301	1,93048	...	3,60926	4,57348	0,42796
X_{47}^1	4,40066	-3,18732	-5,67606	-1,16175	5,42831	...	3,61221	2,91735	-4,42359
X_{48}^1	-6,48912	-1,7131	-2,04942	-2,83251	5,87979	...	-2,05294	1,00921	3,37851
X_{49}^1	3,00152	1,15928	4,85317	4,35978	-3,69392	...	-1,68648	-0,04261	-2,32739
X_{50}^1	-0,4464	5,95634	0,98858	0,49083	7,16717	...	-4,97304	2,16085	-4,8946

7. Mengevaluasi hasil rute setiap partikel.

Posisi terbaru setiap partikel yang diperoleh dari langkah sebelumnya, kemudian dievaluasi hasil rute yang terbentuk seperti pada langkah ke-1 bagian (c). Posisi dari setiap partikel yang telah diperbarui diberikan indeks sesuai kolom. Kemudian posisi dari setiap partikel diurutkan berdasarkan nilai terkecil menurut baris sehingga diperoleh urutan indeks *scene* syuting pada iterasi ke-1 yang ditampilkan pada Tabel 4.17 dan Lampiran 12.

Tabel 4.17 Rute pada Iterasi ke-1

X_j^0	Rute
X_1^1	0-11-39-49-27-31-28-3-44-13-42-23-15-12-20-54-29-24-32-19-4-5-1-33-52-8-10-53-57-36-47-55-26-45-30-56-16-41-14-18-25-59-22-51-40-17-9-46-34-7-50-38-58-48-35-21-43-6-37-2-0
X_2^1	0-52-3-59-48-15-33-5-47-54-50-53-22-8-45-42-34-21-43-32-7-36-23-6-49-13-39-51-55-11-41-17-12-9-38-2-29-1-56-25-28-40-35-14-27-19-20-46-4-24-37-31-18-58-57-44-10-16-30-26-0
X_3^1	0-37-5-17-33-10-34-7-57-16-38-28-23-55-45-9-56-46-29-11-2-22-54-47-1-43-50-35-21-41-31-44-53-15-25-52-14-18-59-30-32-27-48-58-24-13-3-6-36-19-26-12-39-20-49-4-42-8-51-40-0
X_4^1	0-15-54-57-24-29-31-40-39-51-17-3-52-9-42-18-20-14-1-25-32-12-49-22-56-21-50-34-26-27-53-35-28-47-36-13-4-55-58-19-38-11-7-30-46-5-33-37-41-23-59-8-6-2-45-10-43-48-44-16-0
X_5^1	0-28-2-52-32-37-17-51-20-11-5-48-27-58-30-38-1-40-53-19-49-50-41-18-21-54-47-59-46-43-45-15-56-31-10-9-36-29-12-16-6-44-42-7-22-13-34-8-26-57-4-39-14-25-24-35-3-33-23-55-0
X_6^1	0-39-2-53-19-14-15-22-3-17-43-34-57-30-8-49-24-16-1-54-27-37-40-23-4-31-58-28-45-10-7-47-33-26-32-9-6-11-25-29-51-21-42-46-44-12-55-35-50-5-18-56-59-52-48-38-20-41-13-36-0
X_7^1	0-5-49-35-4-31-6-24-44-13-50-43-47-17-21-27-39-59-45-19-7-10-57-15-16-52-2-23-3-9-1-20-40-55-53-8-29-22-14-54-26-46-28-30-25-36-38-11-51-33-56-32-48-12-58-42-41-37-34-18-0
X_8^1	0-1-11-2-4-9-31-26-15-34-46-50-17-33-38-25-45-47-59-52-14-28-40-42-7-24-37-44-22-36-10-18-21-23-20-39-48-54-5-55-13-51-32-43-30-56-8-6-3-27-29-53-16-35-49-12-57-19-41-58-0
X_9^1	0-23-20-58-3-44-57-45-5-26-31-33-22-9-54-10-46-2-12-19-43-40-39-48-34-25-13-30-49-28-53-36-59-27-17-42-29-47-35-1-51-4-38-14-41-50-7-21-37-15-24-52-18-11-16-6-56-32-8-55-0
X_{10}^1	0-24-34-26-35-30-14-7-38-53-56-13-21-55-43-17-46-54-51-18-22-4-10-9-45-52-19-33-3-15-29-42-5-6-50-57-23-48-36-12-16-40-27-47-2-39-58-20-44-49-11-31-32-1-37-25-28-59-8-41-0
...	...
X_{46}^1	0-11-5-49-53-27-45-16-13-9-24-1-22-23-18-29-59-20-36-41-19-2-34-33-46-8-4-48-56-37-32-52-50-25-40-10-51-21-7-43-31-14-38-26-12-35-58-6-54-3-28-55-30-47-15-17-57-44-39-42-0
X_{47}^1	0-37-8-39-4-40-50-36-55-38-51-28-34-9-58-32-49-11-12-2-26-31-48-54-57-47-10-27-29-5-24-42-16-23-1-13-56-19-6-30-25-14-17-22-18-53-20-46-7-43-15-52-59-3-35-33-44-21-41-45-0
X_{48}^1	0-19-54-34-15-37-2-35-51-3-12-47-58-9-6-4-25-56-11-1-36-26-31-53-42-40-59-41-55-7-8-29-48-17-44-28-5-30-32-10-46-45-39-57-20-27-52-18-16-43-49-50-24-38-13-23-14-33-22-21-0
X_{49}^1	0-2-26-22-52-7-50-56-39-17-5-14-59-1-42-46-49-10-30-6-13-11-34-44-24-37-25-58-12-54-48-41-28-4-35-31-29-55-27-16-33-53-36-9-51-3-23-47-40-38-45-43-15-8-18-19-32-21-57-20-0
X_{50}^1	0-23-8-49-19-47-22-5-28-43-38-29-53-13-36-37-46-18-15-21-51-6-16-54-7-41-58-44-42-25-17-9-50-56-45-10-12-20-27-3-59-39-30-11-33-55-34-32-26-2-31-57-48-4-52-40-35-14-24-1-0

8. Mengevaluasi nilai fungsi *fitness* dari hasil rute setiap partikel.

Hasil rute setiap partikel yang diperoleh dari langkah sebelumnya, selanjutnya dievaluasi nilai fungsi *fitness* seperti pada langkah ke-2. Nilai fungsi *fitness* didapatkan setelah menghitung total biaya syuting dari hasil rute setiap partikel. Proses perhitungan total biaya syuting untuk partikel pertama pada iterasi ke-1 (X_1^1) dapat dijelaskan sebagai berikut.

Hasil rute dari partikel X_1^1 pada Tabel 4.17, yaitu

Rute $X_1^1 = [0 - 11 - 39 - 49 - 27 - 31 - 28 - 3 - 44 - 13 - 42 - 23 - 15 - 12 - 20 - 54 - 29 - 24 - 32 - 19 - 4 - 5 - 1 - 33 - 52 - 8 - 10 - 53 - 57 - 36 - 47 - 55 - 26 - 45 - 30 - 56 - 16 - 41 - 14 - 18 - 25 - 59 - 22 - 51 - 40 - 17 - 9 - 46 - 34 - 7 - 50 - 38 - 58 - 48 - 35 - 21 - 43 - 6 - 37 - 2 - 0]$

Kemudian, dihitung total biaya syuting antar *scene* yang dilalui oleh partikel X_1^1 menggunakan biaya antar *scene* yang telah dinormalisasi pada Tabel 4.8.

$$\begin{aligned} f(X_1^1) &= 0,35316 + 0,00323 + 0,27337 + 0,26667 + 0,11317 + 0,47383 + 0,64138 \\ &\quad + 0,31822 + 0,19656 + 0,40119 + 0,18655 + 0,17207 + 0,3574 + 0,35468 \\ &\quad + 0,35487 + 0,78106 + 0,68773 + 0,47642 + 0,15097 + 0,31628 + 0,46802 \\ &\quad + 0,00323 + 0,08404 + 0,85511 + 0,10994 + 0,50788 + 0,12654 + 0,60269 \\ &\quad + 0,27882 + 0,9124 + 0,20662 + 0,13661 + 0,19592 + 0,61473 + 0,53101 \\ &\quad + 0,57408 + 0,17207 + 0,11188 + 0,46414 + 0,19656 + 0,1588 + 0,04788 \\ &\quad + 0,35637 + 0,80596 + 0,73145 + 0,31393 + 0,10994 + 0,32873 + 0,4168 \\ &\quad + 0,23357 + 0,05887 + 0,34059 + 0,46478 + 0,633 + 0,22005 + 0,08598 \\ &\quad + 0,21811 + 0,15097 + 0,57408 + 0,6362 \\ &= 20,91715 \end{aligned}$$

dari total biaya syuting *scene* partikel X_1^1 , dihitung nilai *fitness* menggunakan Persamaan 2.13.

$$F(X_1^1) = \frac{1}{f(X_1^1)} = \frac{1}{20,91715} = 0,04781$$

Proses evaluasi total biaya syuting untuk partikel lainnya dihitung menggunakan bahasa pemrograman Python seperti yang ditampilkan pada Tabel 4.18 dan Lampiran 13.

Tabel 4.18 Total Biaya Syuting dan Nilai *Fitness* tiap Partikel

X_j^1	Biaya Antar <i>Scene</i>						Total Biaya	<i>Fitness</i>
X_1^1	0,35316	0,00323	0,27337	...	0,57408	0,6362	20,91715	0,04781
X_2^1	0,43792	0,78041	0,10994	...	0,27083	0,2561	19,73353	0,05068
X_3^1	0,43792	0,25225	0,0431	...	0,58737	0,2561	24,2023	0,04132
X_4^1	0,60269	0,57796	0,2699	...	0,27337	0,2561	26,57017	0,03764

Tabel 4.18 Total Biaya Syuting dan Nilai *Fitness* tiap Partikel (Lanjutan)

X_j^1	Biaya Antar Scene						Total Biaya	<i>Fitness</i>
X_5^1	0,57602	0,31628	0,13661	...	0,57472	0,6362	23,35936	0,04281
X_6^1	0,08598	0,11317	0,2659	...	0,42657	0,2561	23,58173	0,04241
X_7^1	0,60269	0,63426	0,74231	...	0,34059	0,2561	23,44856	0,04265
X_8^1	0,17207	0,78041	0,10994	...	0,12258	0,2561	22,57653	0,04429
X_9^1	0,08598	0,11317	0,57796	...	0,57472	0,6362	22,93853	0,04359
X_{10}^1	0,19721	0,52261	0,57408	...	0,57472	0,6362	23,69098	0,04221
...	...	...	...	...	...	...	...	...
X_{46}^1	0,60269	0,31628	0,63426	...	0,12258	0,2561	21,49505	0,04652
X_{47}^1	0,35316	0,60462	0,57796	...	0,27337	0,2561	21,76526	0,04594
X_{48}^1	0,35316	0,10994	0,10994	...	0,62035	0,2561	21,07423	0,04745
X_{49}^1	0,78235	0,10994	0,73391	...	0,27083	0,2561	24,5462	0,04074
X_{50}^1	0,19721	0,10994	0,00323	...	0,3574	0,2561	23,27209	0,04297

9. Menentukan P_{best} setiap partikel dan G_{best} .

- (a) P_{best} dari setiap partikel selalu diperbarui pada setiap iterasi dengan menggunakan Persamaan 2.16.

Penentuan P_{best} untuk partikel pertama sampai dengan terakhir pada iterasi ke-1 dapat dituliskan sebagai Persamaan 4.4.

$$P_{best,j}^1 = \begin{cases} X_j^{(1)} & , F(X_j^{(1)}) \geq F(P_{best,j}^{(0)}) \\ P_{best,j}^{(0)} & , F(X_j^{(1)}) < F(P_{best,j}^{(0)}) \end{cases} \quad (4.4)$$

Proses penentuan P_{best} untuk partikel pertama pada iterasi ke-1 (X_1^1) menggunakan Persamaan 4.4 dapat dijelaskan sebagai berikut.

Nilai fungsi *fitness* untuk partikel pertama pada iterasi ke-1 (X_1^1) pada Tabel 4.18 sebesar 0,04781. Sedangkan nilai fungsi *fitness* untuk P_{best} partikel pertama pada iterasi ke-0 ($P_{best,1}^0$) pada Tabel 4.18 sebesar 0,04475. Terlihat bahwa nilai fungsi *fitness* X_1^1 lebih besar daripada nilai fungsi *fitness* $P_{best,1}^0$, sehingga penentuan P_{best} untuk partikel pertama pada iterasi ke-1 menggunakan Persamaan 4.4 adalah sebagai berikut.

$$P_{best,1}^1 = [-3,94601 \ 4,84622 \ -5,53092 \ \dots \ 10 \ \dots \ -1,45129]$$

Penentuan P_{best} untuk seluruh partikel pada iterasi ke-1 dapat ditampilkan pada Tabel 4.19 dan Lampiran 14.

Tabel 4.19 P_{best} Iterasi ke-1

$P_{best,j}^1$	1	2	3	...	58	59	<i>Fitness Value</i>
$P_{best,1}^1$	-3,94601	4,84622	-5,53092	...	-3,5931	-1,45129	0,04781
$P_{best,2}^1$	-6,96541	-2,73767	1,54813	...	1,64302	-0,29266	0,05068
$P_{best,3}^1$	-9,64862	-0,97641	2,24335	...	-0,27442	6,13968	0,04236
$P_{best,4}^1$	2,53456	-6,57045	7,78262	...	3,0871	-3,89433	0,03914
$P_{best,5}^1$	3,90363	3,71686	1,81943	...	-3,54684	-0,50911	0,04838
$P_{best,6}^1$	1,05232	4,08277	-5,75775	...	2,36173	-5,81629	0,04241
$P_{best,7}^1$	-2,56848	3,96685	-5,91593	...	7,05032	6,14002	0,04346
$P_{best,8}^1$	-7,25428	0,89507	4,90235	...	0,69543	-3,09378	0,04429
$P_{best,9}^1$	-8,15918	-1,86934	-0,10474	...	0,87569	-0,8173	0,05049
$P_{best,10}^1$	8,11716	-3,99829	4,98738	...	7,21959	-9,0677	0,04235
...	...	...	...	...	...	...	...
$P_{best,46}^1$	-7,23156	-2,23126	-4,17018	...	4,57348	0,42796	0,04652
$P_{best,47}^1$	4,40066	-3,18732	-5,67606	...	2,91735	-4,42359	0,04594
$P_{best,48}^1$	-6,48912	-1,7131	-2,04942	...	1,00921	3,37851	0,04745
$P_{best,49}^1$	3,00152	1,15928	4,85317	...	-0,04261	-2,32739	0,04074
$P_{best,50}^1$	-0,4464	5,95634	0,98858	...	2,16085	-4,8946	0,04297

- (b) G_{best} selalu diperbarui pada setiap iterasi berdasarkan posisi dengan nilai fungsi *fitness* paling terbaik yang pernah dicapai di antara seluruh partikel pada semua iterasi. Pada setiap iterasi, posisi terbaik dari setiap partikel disimpan sebagai P_{best} . Penentuan G_{best} di setiap iterasi mudah dilakukan dengan membandingkan nilai fungsi *fitness* P_{best} partikel terbaik pada iterasi ke- i dan nilai fungsi *fitness* G_{best} iterasi sebelumnya. Proses penentuan G_{best} pada iterasi ke-1 dapat dijelaskan sebagai berikut.

Pada Tabel 4.19, didapatkan P_{best} terbaik pada iterasi ke-1 adalah $P_{best,2}^1$ dengan nilai fungsi *fitness* sebesar 0,05068. Sedangkan pada Tabel 4.14, didapatkan G_{best} pada iterasi ke-0 dengan nilai fungsi *fitness* sebesar 0,05049. Terlihat bahwa nilai fungsi *fitness* $P_{best,2}^1$ lebih besar daripada nilai fungsi *fitness* G_{best}^0 , sehingga dapat ditentukan bahwa G_{best} untuk iterasi ke-1 diperbarui sama dengan $P_{best,2}^1$ adalah sebagai berikut.

$$G_{best}^1 = [-6,96541 \quad -2,73767 \quad 1,54813 \quad \dots \quad -0,29266]$$

Kemudian G_{best} dapat ditampilkan dalam Tabel 4.20

Tabel 4.20 G_{best} Iterasi ke-1

G_{best}^i	1	2	3	...	58	59	Total Biaya Syuting	Fitness Value
G_{best}^0	-8,15918	-1,86934	-0,10474	...	0,87569	-0,8173	19,80464	0,05049
G_{best}^1	-6,96541	-2,73767	1,54813	...	1,64302	-0,29266	19,73353	0,05068

G_{best} merupakan solusi optimal yang diperoleh menggunakan algoritma PSO. Hasil solusi optimal dari G_{best} berupa total biaya syuting yang optimal serta rute yang dihasilkan. G_{best} yang telah diperoleh harus dicari rute yang dihasilkan terlebih dahulu kemudian dapat dihitung total biaya syuting yang optimal seperti pada langkah 3 bagian (b). Proses pencarian hasil solusi optimal dari G_{best} pada iterasi ke-1 dapat dijelaskan sebagai berikut. Pada Tabel 4.20 diperoleh posisi dari G_{best}^1 .

$$G_{best}^1 = [-6,96541 \quad -2,73767 \quad 1,54813 \quad \dots \quad -0,29266]$$

Hasil rute dari G_{best} didapatkan dengan melakukan proses pencarian rute seperti pada langkah 7. Rute yang dihasilkan dari posisi pada G_{best}^1 adalah sebagai berikut.

$$\text{Rute } G_{best}^1 = 0-33-36-29-49-51-1-27-40-18-\dots-14-25-11-0$$

Kemudian, dihitung total biaya syuting antar *scene* yang dilalui oleh G_{best}^1 menggunakan biaya antar *scene* yang telah dinormalisasi pada Tabel 4.8.

$$\begin{aligned} f(G_{best}^1) &= 0.43792 + 0.78041 + 0.10994 + \dots + 0.27083 + 0.2561 \\ &= 19,73353 \end{aligned}$$

Total biaya syuting rute G_{best}^1 masih dalam bentuk yang telah dinormalisasi. Agar dapat diperoleh total biaya syuting rute G_{best} dalam Rupiah digunakan Persamaan 2.17.

$$\begin{aligned} \text{biaya syuting} &= f(G_{best}^1) \cdot (\max(\text{biaya scene}) + \min(\text{biaya scene})) + \min(\text{biaya scene}) \\ &= 19.73353 \cdot (\text{Rp}580.109,48 - \text{Rp}0) + \text{Rp}0 \\ &= \text{Rp}11.447.607,82686 \end{aligned}$$

Setelah diubah indeks *scene* pada rute G_{best}^1 agar didapatkan urutan *scene* syuting yaitu: 4-21B-22C-19-31-32B-1C-17-25-12B-15-16D-7A-26-11-20

-29B-22B-34-23B-29A-1D-32A-23A-2A-19B-33B-2B-30A-37-9A-27-35A-33A-13-14B-1E-3-21A-36-30B-6A-12A-35B-5-22A-34B-18-16B-30C-16A-10-6B-24-28-14A-8-9B-16C-7B-4

10. Mengevaluasi kriteria pemberhentian

Setelah pencarian G_{best} sebagai solusi optimal pada setiap iterasi, dilakukan evaluasi kriteria pemberhentian. Pada parameter awal yang telah diinisialisasi pada langkah 1 bagian (a), ditentukan iterasi maksimal sebesar 100. Iterasi saat ini yaitu iterasi ke-1 dan kurang dari iterasi maksimal, sehingga iterasi berlanjut dan mengulangi perhitungan dari langkah 4.

Proses perhitungan algoritma PSO untuk optimalisasi urutan *scene* syuting film "Ketika Adzan Sudah Tidak Lagi Berkumandang" yang telah dijelaskan dilakukan berulang sampai kriteria pemberhentian terpenuhi. Perhitungan algoritma PSO dapat dibantu dengan bahasa pemrograman Python untuk perhitungan dengan maksimal iterasi, ukuran *swarm*, dan dimensi yang besar. Dimensi dari konsep TSP pada optimalisasi urutan *scene* syuting film "Ketika Adzan Sudah Tidak Lagi Berkumandang" berukuran besar, sehingga proses perhitungan algoritma PSO dibantu dengan bahasa pemrograman Python.

4.2.3 Analisis Hasil Pengujian Modifikasi Nilai Parameter PSO

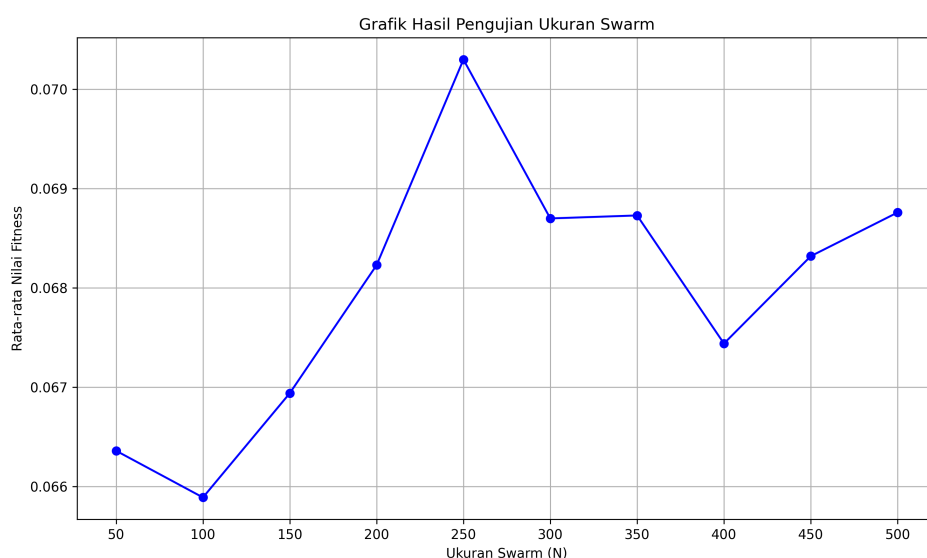
Penentuan nilai parameter PSO pada proses perhitungan algoritma PSO dapat mempengaruhi solusi optimal yang dicari. Oleh karena itu, dilakukan uji coba terlebih dahulu terhadap modifikasi nilai parameter PSO. Hasil dari uji coba dianalisis agar diperoleh bagaimana pengaruh parameter-parameter tersebut terhadap solusi optimal dari perhitungan algoritma PSO. Setelah didapatkan modifikasi nilai parameter yang terbaik, kemudian perhitungan algoritma PSO dilakukan menggunakan nilai parameter tersebut.

Terdapat 3 modifikasi nilai parameter algoritma PSO yang digunakan yaitu nilai N pada parameter ukuran *swarm*, nilai n pada parameter iterasi maksimum, dan nilai ε pada faktor *clamping* parameter *velocity* maksimum. Untuk setiap pengujian diulang sebanyak 10 kali untuk mendapatkan rata-rata nilai fungsi *fitness* yang dihasilkan G_{best} . Dalam setiap pengujian modifikasi nilai parameter digunakan parameter awal yaitu ukuran *swarm* (N) sebesar 50, iterasi maksimum (n) sebesar 100, dan faktor *clamping* (ε) pada *velocity* maksimum sebesar 0.2.

Analisis hasil pengujian modifikasi nilai parameter PSO dapat dijelaskan sebagai berikut.

1. Hasil Pengujian Ukuran *Swarm*

Pengujian nilai parameter ukuran *swarm* dilakukan menggunakan beberapa nilai parameter. Nilai parameter ukuran *swarm* yang diujikan yaitu 50, 100, 150, 200, 250, 300, 350, 400, 450, dan 500. Pengujian dari setiap nilai parameter ukuran *swarm* diulang sebanyak 10 kali. Hasil pengujian nilai parameter ukuran *swarm* dapat ditampilkan dalam grafik pada Gambar 4.3 berikut.

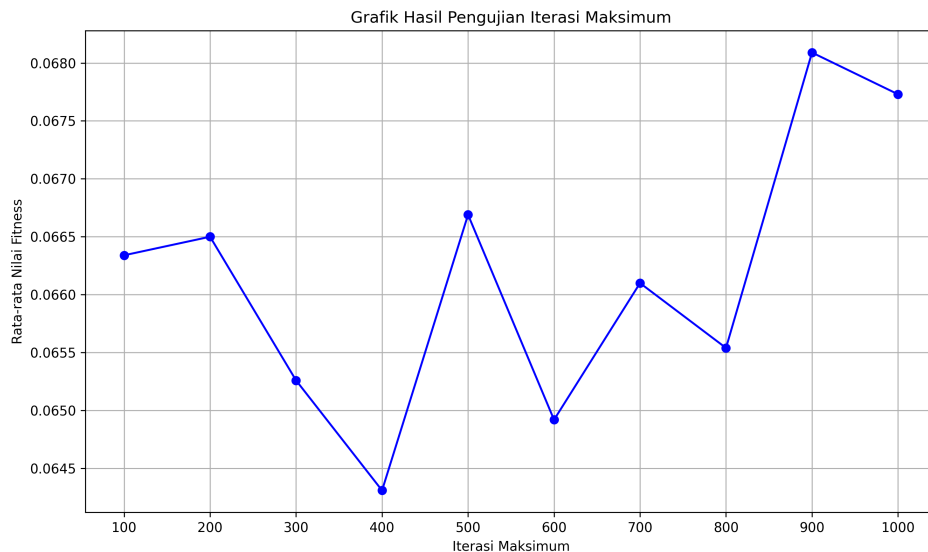


Gambar 4.3 Grafik Hasil Pengujian Ukuran *Swarm* (N)

Berdasarkan grafik pada Gambar 4.3 menunjukkan bahwa nilai fungsi *fitness* terbaik dicapai pada ukuran *swarm* sebesar 250 partikel dengan rata-rata nilai fungsi *fitness* sebesar 0,0703. Nilai parameter ukuran *swarm* tersebut dipilih sebagai parameter awal pada proses perhitungan algoritma PSO untuk optimalisasi pemilihan urutan *scene* syuting film "Ketika Adzan Sudah Tidak Lagi Berku-mandang".

2. Hasil Pengujian Iterasi Maksimum

Pengujian nilai parameter iterasi maksimum dilakukan menggunakan beberapa nilai parameter. Nilai parameter iterasi maksimum yang diujikan yaitu 100, 200, 300, 400, 500, 600, 700, 800, 900, dan 1000. Pengujian dari setiap nilai parameter iterasi maksimum diulang sebanyak 10 kali. Hasil pengujian parameter iterasi maksimum dapat ditampilkan dalam grafik pada Gambar 4.4.

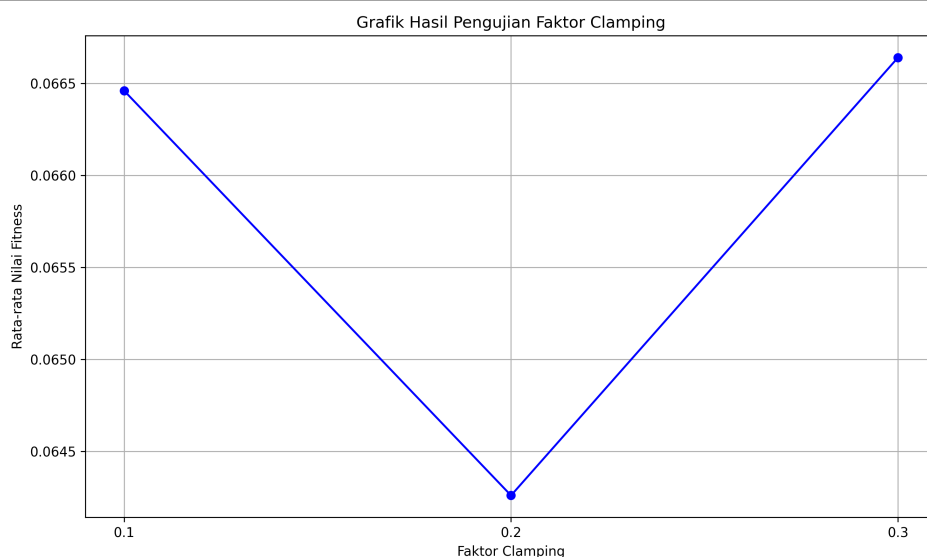


Gambar 4.4 Grafik Hasil Pengujian Iterasi Maksimum (n)

Berdasarkan grafik pada Gambar 4.4 menunjukkan bahwa nilai fungsi *fitness* terbaik dicapai pada iterasi maksimum sebesar 900 dengan rata-rata nilai fungsi *fitness* sebesar 0,06809. Nilai parameter iterasi maksimum tersebut dipilih sebagai parameter awal pada proses perhitungan algoritma PSO untuk optimalisasi pemilihan urutan *scene* syuting film ”Ketika Adzan Sudah Tidak Lagi Berku-mandang”.

3. Hasil Pengujian Faktor *Clamping* pada *Velocity* Maksimum

Pengujian nilai parameter faktor *clamping* pada *velocity* maksimum dilakukan menggunakan beberapa nilai parameter. Nilai parameter faktor *clamping* yang diujikan yaitu 0, 1, 0, 2, dan 0, 3. Pengujian dari setiap nilai parameter iterasi maksimum diulang sebanyak 10 kali. Hasil pengujian parameter faktor *clamping* pada *velocity* maksimum dapat ditampilkan dalam grafik pada Gambar 4.5.



Gambar 4.5 Grafik Hasil Pengujian Iterasi Maksimum (n)

Berdasarkan grafik pada Gambar 4.5 menunjukkan nilai fungsi *fitness* terbaik dicapai pada faktor *clamping* sebesar 0.3 dengan rata-rata nilai fungsi *fitness* sebesar 0,06664. Nilai parameter faktor *clamping* pada *velocity* maksimum tersebut dipilih sebagai parameter awal pada proses perhitungan algoritma PSO untuk optimalisasi pemilihan urutan *scene* syuting film "Ketika Adzan Sudah Tidak Lagi Berkumandang".

Dari ketiga hasil pengujian modifikasi nilai parameter yang telah dilakukan, selanjutnya dilakukan proses perhitungan algoritma PSO menggunakan nilai-nilai parameter terbaik yang telah diujikan.

4.2.4 Analisis Hasil Perhitungan Algoritma PSO

Hasil nilai parameter dari pengujian modifikasi nilai parameter PSO digunakan ke dalam perhitungan algoritma PSO. Parameter terbaik dari hasil pengujian modifikasi parameter digunakan dalam proses perhitungan algoritma untuk mendapatkan hasil optimal. Parameter awal tersebut ditetapkan pada langkah inisialisasi pada proses perhitungan algoritma PSO. Penetapan parameter awal pada langkah inisialisasi dalam proses perhitungan algoritma PSO untuk hasil optimalisasi pemilihan urutan *scene* syuting film "Ketika Adzan Sudah Tidak Lagi Berkumandang" dapat dijelaskan sebagai berikut.

1. Iterasi maksimum: $n = 900$
2. Ukuran *swarm*: $N = 250$

3. Batas posisi: $X_{min} = -10$ dan $X_{max} = 10$
4. Faktor *clamping* pada *velocity* maksimum: $\varepsilon = 0,3$
5. Batas *velocity* maksimum: $V_{max} = \varepsilon(X_{max} - X_{min}) = 0,3(10 - (-10)) = 6$
6. Batas *velocity* minimum: $V_{min} = -V_{max} = -6$
7. *Inertia Weight*: Strategi *Random Inertia Weight*

Proses perhitungan algoritma PSO dilakukan dengan menggunakan bantuan bahasa pemrograman Python. Langkah-langkah perhitungan algoritma PSO ditulis dalam bahasa pemrograman Python, kemudian dijalankan proses perhitungan dengan menggunakan parameter awal yang telah ditetapkan. Setelah proses perhitungan dilakukan didapatkan nilai G_{best} pada iterasi awal sampai dengan maksimum iterasi yang dapat ditampilkan pada Tabel 4.21 dan Lampiran 15.

Tabel 4.21 G_{best} pada Setiap Iterasi

G_{best}^i	1	2	3	...	57	58	59	Fitness Value
G_{best}^0	-9,31285	7,24042	-8,98804	...	-1,99376	8,13782	4,2582	0,05224
G_{best}^1	-9,31285	7,24042	-8,98804	...	-1,99376	8,13782	4,2582	0,05224
G_{best}^2	-9,31285	7,24042	-8,98804	...	-1,99376	8,13782	4,2582	0,05224
G_{best}^3	-6,41733	-1,2997	-4,0	...	2,33926	2,73148	-0,34552	0,05253
G_{best}^4	-6,41733	-1,2997	-4,0	...	2,33926	2,73148	-0,34552	0,05253
G_{best}^5	-7,83898	-0,84755	0,55751	...	6,63288	5,3918	-5,14845	0,05371
G_{best}^6	-7,83898	-0,84755	0,55751	...	6,63288	5,3918	-5,14845	0,05371
G_{best}^7	-7,83898	-0,84755	0,55751	...	6,63288	5,3918	-5,14845	0,05371
G_{best}^8	-7,83898	-0,84755	0,55751	...	6,63288	5,3918	-5,14845	0,05371
G_{best}^9	-7,83898	-0,84755	0,55751	...	6,63288	5,3918	-5,14845	0,05371
G_{best}^{10}	-7,83898	-0,84755	0,55751	...	6,63288	5,3918	-5,14845	0,05371
...	...	...	...	...	...	...	...	...
G_{best}^{894}	-10,0	7,62661	-4,8845	...	-2,84657	10,0	0,33786	0,07443
G_{best}^{895}	-10,0	7,62661	-4,8845	...	-2,84657	10,0	0,33786	0,07443
G_{best}^{896}	-10,0	7,62661	-4,8845	...	-2,84657	10,0	0,33786	0,07443
G_{best}^{897}	-10,0	7,62661	-4,8845	...	-2,84657	10,0	0,33786	0,07443
G_{best}^{898}	-10,0	7,62661	-4,8845	...	-2,84657	10,0	0,33786	0,07443
G_{best}^{899}	-10,0	7,62661	-4,8845	...	-2,84657	10,0	0,33786	0,07443
G_{best}^{900}	-10,0	7,62661	-4,8845	...	-2,84657	10,0	0,33786	0,07443

G_{best} yang diperoleh dapat menjadi solusi dari proses optimalisasi, yaitu dengan mencari rute yang dihasilkan dari posisi pada G_{best} . Pencarian rute pada G_{best} dilakukan seperti yang telah dijelaskan pada proses perhitungan algoritma PSO. Hasil pencarian rute pada G_{best} dapat ditampilkan pada Tabel 4.22 dan Lampiran 16.

Tabel 4.22 Rute G_{best} pada Setiap Iterasi

G_{best}^i	Rute
G_{best}^0	0, 35, 1, 18, 36, 3, 49, 9, 41, 24, 13, 52, 55, 4, 19, 53, 39, 10, 48, 22, 56, 43, 42, 44, 29, 57, 51, 8, 50, 7, 14, 33, 30, 5, 20, 31, 11, 32, 46, 26, 54, 59, 12, 34, 37, 21, 6, 27, 15, 25, 23, 17, 38, 2, 40, 58, 47, 28, 45, 16, 0
G_{best}^1	0, 35, 1, 18, 36, 3, 49, 9, 41, 24, 13, 52, 55, 4, 19, 53, 39, 10, 48, 22, 56, 43, 42, 44, 29, 57, 51, 8, 50, 7, 14, 33, 30, 5, 20, 31, 11, 32, 46, 26, 54, 59, 12, 34, 37, 21, 6, 27, 15, 25, 23, 17, 38, 2, 40, 58, 47, 28, 45, 16, 0
G_{best}^2	0, 35, 1, 18, 36, 3, 49, 9, 41, 24, 13, 52, 55, 4, 19, 53, 39, 10, 48, 22, 56, 43, 42, 44, 29, 57, 51, 8, 50, 7, 14, 33, 30, 5, 20, 31, 11, 32, 46, 26, 54, 59, 12, 34, 37, 21, 6, 27, 15, 25, 23, 17, 38, 2, 40, 58, 47, 28, 45, 16, 0
G_{best}^3	0, 35, 4, 42, 44, 1, 48, 51, 50, 39, 3, 36, 49, 16, 52, 30, 55, 34, 13, 25, 33, 22, 5, 2, 27, 20, 41, 45, 53, 59, 18, 29, 24, 15, 23, 10, 56, 43, 8, 57, 14, 17, 58, 7, 37, 19, 9, 32, 40, 46, 26, 11, 31, 28, 21, 47, 6, 54, 12, 38, 0
G_{best}^4	0, 35, 4, 42, 44, 1, 48, 51, 50, 39, 3, 36, 49, 16, 52, 30, 55, 34, 13, 25, 33, 22, 5, 2, 27, 20, 41, 45, 53, 59, 18, 29, 24, 15, 23, 10, 56, 43, 8, 57, 14, 17, 58, 7, 37, 19, 9, 32, 40, 46, 26, 11, 31, 28, 21, 47, 6, 54, 12, 38, 0
G_{best}^5	0, 42, 1, 34, 35, 59, 55, 23, 51, 14, 15, 37, 18, 50, 22, 33, 46, 4, 2, 16, 44, 11, 25, 48, 20, 30, 36, 3, 49, 53, 27, 21, 39, 28, 52, 45, 40, 17, 12, 6, 47, 13, 41, 24, 38, 5, 29, 10, 58, 7, 56, 43, 26, 57, 8, 9, 32, 19, 31, 54, 0
G_{best}^6	0, 42, 1, 34, 35, 59, 55, 23, 51, 14, 15, 37, 18, 50, 22, 33, 46, 4, 2, 16, 44, 11, 25, 48, 20, 30, 36, 3, 49, 53, 27, 21, 39, 28, 52, 45, 40, 17, 12, 6, 47, 13, 41, 24, 38, 5, 29, 10, 58, 7, 56, 43, 26, 57, 8, 9, 32, 19, 31, 54, 0
G_{best}^7	0, 42, 1, 34, 35, 59, 55, 23, 51, 14, 15, 37, 18, 50, 22, 33, 46, 4, 2, 16, 44, 11, 25, 48, 20, 30, 36, 3, 49, 53, 27, 21, 39, 28, 52, 45, 40, 17, 12, 6, 47, 13, 41, 24, 38, 5, 29, 10, 58, 7, 56, 43, 26, 57, 8, 9, 32, 19, 31, 54, 0
G_{best}^8	0, 42, 1, 34, 35, 59, 55, 23, 51, 14, 15, 37, 18, 50, 22, 33, 46, 4, 2, 16, 44, 11, 25, 48, 20, 30, 36, 3, 49, 53, 27, 21, 39, 28, 52, 45, 40, 17, 12, 6, 47, 13, 41, 24, 38, 5, 29, 10, 58, 7, 56, 43, 26, 57, 8, 9, 32, 19, 31, 54, 0
G_{best}^9	0, 42, 1, 34, 35, 59, 55, 23, 51, 14, 15, 37, 18, 50, 22, 33, 46, 4, 2, 16, 44, 11, 25, 48, 20, 30, 36, 3, 49, 53, 27, 21, 39, 28, 52, 45, 40, 17, 12, 6, 47, 13, 41, 24, 38, 5, 29, 10, 58, 7, 56, 43, 26, 57, 8, 9, 32, 19, 31, 54, 0
G_{best}^{10}	0, 42, 1, 34, 35, 59, 55, 23, 51, 14, 15, 37, 18, 50, 22, 33, 46, 4, 2, 16, 44, 11, 25, 48, 20, 30, 36, 3, 49, 53, 27, 21, 39, 28, 52, 45, 40, 17, 12, 6, 47, 13, 41, 24, 38, 5, 29, 10, 58, 7, 56, 43, 26, 57, 8, 9, 32, 19, 31, 54, 0
...	...
G_{best}^{894}	0, 1, 35, 42, 15, 18, 50, 38, 36, 4, 55, 37, 53, 52, 29, 49, 3, 10, 51, 39, 24, 34, 6, 57, 21, 13, 43, 17, 44, 33, 56, 59, 8, 40, 12, 11, 46, 41, 23, 16, 5, 30, 2, 19, 7, 54, 9, 31, 14, 22, 27, 26, 28, 45, 47, 25, 48, 20, 32, 58, 0
G_{best}^{895}	0, 1, 35, 42, 15, 18, 50, 38, 36, 4, 55, 37, 53, 52, 29, 49, 3, 10, 51, 39, 24, 34, 6, 57, 21, 13, 43, 17, 44, 33, 56, 59, 8, 40, 12, 11, 46, 41, 23, 16, 5, 30, 2, 19, 7, 54, 9, 31, 14, 22, 27, 26, 28, 45, 47, 25, 48, 20, 32, 58, 0

Tabel 4.22 Rute G_{best} pada Setiap Iterasi (Lanjutan)

G_{best}^i	Rute
G_{best}^{896}	0, 1, 35, 42, 15, 18, 50, 38, 36, 4, 55, 37, 53, 52, 29, 49, 3, 10, 51, 39, 24, 34, 6, 57, 21, 13, 43, 17, 44, 33, 56, 59, 8, 40, 12, 11, 46, 41, 23, 16, 5, 30, 2, 19, 7, 54, 9, 31, 14, 22, 27, 26, 28, 45, 47, 25, 48, 20, 32, 58, 0
G_{best}^{897}	0, 1, 35, 42, 15, 18, 50, 38, 36, 4, 55, 37, 53, 52, 29, 49, 3, 10, 51, 39, 24, 34, 6, 57, 21, 13, 43, 17, 44, 33, 56, 59, 8, 40, 12, 11, 46, 41, 23, 16, 5, 30, 2, 19, 7, 54, 9, 31, 14, 22, 27, 26, 28, 45, 47, 25, 48, 20, 32, 58, 0
G_{best}^{898}	0, 1, 35, 42, 15, 18, 50, 38, 36, 4, 55, 37, 53, 52, 29, 49, 3, 10, 51, 39, 24, 34, 6, 57, 21, 13, 43, 17, 44, 33, 56, 59, 8, 40, 12, 11, 46, 41, 23, 16, 5, 30, 2, 19, 7, 54, 9, 31, 14, 22, 27, 26, 28, 45, 47, 25, 48, 20, 32, 58, 0
G_{best}^{899}	0, 1, 35, 42, 15, 18, 50, 38, 36, 4, 55, 37, 53, 52, 29, 49, 3, 10, 51, 39, 24, 34, 6, 57, 21, 13, 43, 17, 44, 33, 56, 59, 8, 40, 12, 11, 46, 41, 23, 16, 5, 30, 2, 19, 7, 54, 9, 31, 14, 22, 27, 26, 28, 45, 47, 25, 48, 20, 32, 58, 0
G_{best}^{900}	0, 1, 35, 42, 15, 18, 50, 38, 36, 4, 55, 37, 53, 52, 29, 49, 3, 10, 51, 39, 24, 34, 6, 57, 21, 13, 43, 17, 44, 33, 56, 59, 8, 40, 12, 11, 46, 41, 23, 16, 5, 30, 2, 19, 7, 54, 9, 31, 14, 22, 27, 26, 28, 45, 47, 25, 48, 20, 32, 58, 0

Kemudian, dari rute yang telah diperoleh tersebut dihitung total biaya syuting dari masing-masing G_{best} menggunakan biaya antar *scene* yang telah dinormalisasi pada Tabel 4.8. Total biaya syuting dari masing-masing G_{best} dapat ditampilkan pada Tabel 4.23 dan Lampiran 17.

Tabel 4.23 Total Biaya Syuting G_{best} pada Setiap Iterasi

G_{best}^i	Biaya Antar <i>Scene</i>						Total Biaya	<i>Fitness</i>
G_{best}^0	0,00194	0,17013	0,61279	...	0,7365	0,64138	19,1427	0,05224
G_{best}^1	0,00194	0,17013	0,61279	...	0,7365	0,64138	19,1427	0,05224
G_{best}^2	0,00194	0,17013	0,61279	...	0,7365	0,64138	19,1427	0,05224
G_{best}^3	0,00194	0,63362	0,57796	...	0,85576	0,6362	19,03668	0,05253
G_{best}^4	0,00194	0,63362	0,57796	...	0,85576	0,6362	19,03668	0,05253
G_{best}^5	0,19721	0,25675	0,0	...	0,27382	0,17013	18,61844	0,05371
G_{best}^6	0,19721	0,25675	0,0	...	0,27382	0,17013	18,61844	0,05371
G_{best}^7	0,19721	0,25675	0,0	...	0,27382	0,17013	18,61844	0,05371
G_{best}^8	0,19721	0,25675	0,0	...	0,27382	0,17013	18,61844	0,05371
G_{best}^9	0,19721	0,25675	0,0	...	0,27382	0,17013	18,61844	0,05371
G_{best}^{10}	0,19721	0,25675	0,0	...	0,27382	0,17013	18,61844	0,05371
...	...	...	...	...	...	...	...	...
G_{best}^{894}	0,17207	0,0	0,19656	...	0,27065	0,17207	13,43601	0,07443
G_{best}^{895}	0,17207	0,0	0,19656	...	0,27065	0,17207	13,43601	0,07443
G_{best}^{896}	0,17207	0,0	0,19656	...	0,27065	0,17207	13,43601	0,07443
G_{best}^{897}	0,17207	0,0	0,19656	...	0,27065	0,17207	13,43601	0,07443

Tabel 4.23 Total Biaya Syuting G_{best} pada Setiap Iterasi (Lanjutan)

G_{best}^i	Biaya Antar Scene						Total Biaya	<i>Fitness</i>
G_{best}^{898}	0,17207	0,0	0,19656	...	0,27065	0,17207	13,43601	0,07443
G_{best}^{899}	0,17207	0,0	0,19656	...	0,27065	0,17207	13,43601	0,07443
G_{best}^{900}	0,17207	0,0	0,19656	...	0,27065	0,17207	13,43601	0,07443

Total biaya syuting yang diperoleh masih dalam bentuk yang telah dinormalisasi dan rute yang diperoleh sebelumnya masih dalam bentuk indeks *scene*. Sehingga total biaya syuting dari setiap rute G_{best} diubah dalam Rupiah menggunakan Persamaan 2.17 dan mengubah indeks *scene* menjadi nama *scene* sesuai pada Tabel 4.1. Total biaya syuting dalam Rupiah dan urutan *scene* syuting dari setiap rute G_{best} dapat ditampilkan dalam Tabel 4.24.

Tabel 4.24 Urutan Scene dan Total Biaya G_{best} pada Setiap Iterasi

G_{best}^i	Rute	Total Biaya Syuting (Rp)
G_{best}^0	4-22B-1C-12B-22C-1E-31-6B-26-16B-9A-33A-34B-2A-13-33B-24-7A-30C-15-35A-28-27-29A-19-35B-32B-6A-32A-5-9B-21B-19B-2B-14A-20-7B-21A-30A-16D-34-37-8-22A-23A-14B-3-17-10-16C-16A-12A-23B-1D-25-36-30B-18-29B-11-4	11.104.859,48
G_{best}^1	4-22B-1C-12B-22C-1E-31-6B-26-16B-9A-33A-34B-2A-13-33B-24-7A-30C-15-35A-28-27-29A-19-35B-32B-6A-32A-5-9B-21B-19B-2B-14A-20-7B-21A-30A-16D-34-37-8-22A-23A-14B-3-17-10-16C-16A-12A-23B-1D-25-36-30B-18-29B-11-4	11.104.859,48
G_{best}^2	4-22B-1C-12B-22C-1E-31-6B-26-16B-9A-33A-34B-2A-13-33B-24-7A-30C-15-35A-28-27-29A-19-35B-32B-6A-32A-5-9B-21B-19B-2B-14A-20-7B-21A-30A-16D-34-37-8-22A-23A-14B-3-17-10-16C-16A-12A-23B-1D-25-36-30B-18-29B-11-4	11.104.859,48
G_{best}^3	4-22B-2A-27-29A-1C-30C-32B-32A-24-1E-22C-31-11-33A-19B-34B-22A-9A-16C-21B-15-2B-1D-17-14A-26-29B-33B-37-12B-19-16B-10-16A-7A-35A-28-6A-35B-9B-12A-36-5-23A-13-6B-21A-25-30A-16D-7B-20-18-14B-30B-3-34-8-23B-4	11.043.359,48
G_{best}^4	4-22B-2A-27-29A-1C-30C-32B-32A-24-1E-22C-31-11-33A-19B-34B-22A-9A-16C-21B-15-2B-1D-17-14A-26-29B-33B-37-12B-19-16B-10-16A-7A-35A-28-6A-35B-9B-12A-36-5-23A-13-6B-21A-25-30A-16D-7B-20-18-14B-30B-3-34-8-23B-4	11.043.359,48
G_{best}^5	4-27-1C-22A-22B-37-34B-16A-32B-9B-10-23A-12B-32A-15-21B-30A-2A-1D-11-29A-7B-16C-30C-14A-19B-22C-1E-31-33B-17-14B-24-18-33A-29B-25-12A-8-3-30B-9A-26-16B-23B-2B-19-7A-36-5-35A-28-16D-35B-6A-6B-21A-13-20-34-4	10.800.734,48

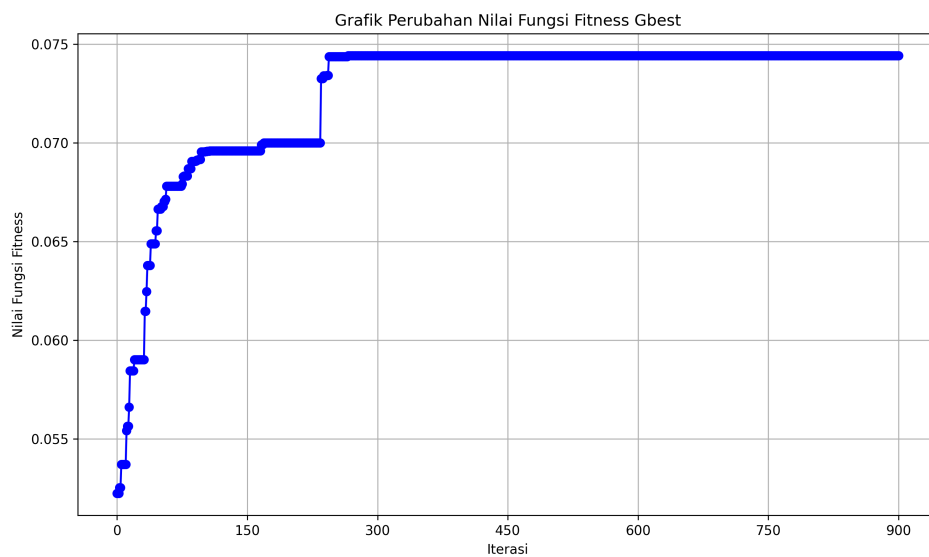
Tabel 4.24 Urutan *Scene* dan Total Biaya G_{best} pada Setiap Iterasi (Lanjutan)

G_{best}^i	Rute	Total Biaya Syuting (Rp)
G_{best}^6	4-27-1C-22A-22B-37-34B-16A-32B-9B-10-23A-12B-32A-15-21B-30A-2A-1D-11-29A-7B-16C-30C-14A-19B-22C-1E-31-33B-17-14B-24-18-33A-29B-25-12A-8-3-30B-9A-26-16B-23B-2B-19-7A-36-5-35A-28-16D-35B-6A-6B-21A-13-20-34-4	10.800.734,48
G_{best}^7	4-27-1C-22A-22B-37-34B-16A-32B-9B-10-23A-12B-32A-15-21B-30A-2A-1D-11-29A-7B-16C-30C-14A-19B-22C-1E-31-33B-17-14B-24-18-33A-29B-25-12A-8-3-30B-9A-26-16B-23B-2B-19-7A-36-5-35A-28-16D-35B-6A-6B-21A-13-20-34-4	10.800.734,48
G_{best}^8	4-27-1C-22A-22B-37-34B-16A-32B-9B-10-23A-12B-32A-15-21B-30A-2A-1D-11-29A-7B-16C-30C-14A-19B-22C-1E-31-33B-17-14B-24-18-33A-29B-25-12A-8-3-30B-9A-26-16B-23B-2B-19-7A-36-5-35A-28-16D-35B-6A-6B-21A-13-20-34-4	10.800.734,48
G_{best}^9	4-27-1C-22A-22B-37-34B-16A-32B-9B-10-23A-12B-32A-15-21B-30A-2A-1D-11-29A-7B-16C-30C-14A-19B-22C-1E-31-33B-17-14B-24-18-33A-29B-25-12A-8-3-30B-9A-26-16B-23B-2B-19-7A-36-5-35A-28-16D-35B-6A-6B-21A-13-20-34-4	10.800.734,48
G_{best}^{10}	4-27-1C-22A-22B-37-34B-16A-32B-9B-10-23A-12B-32A-15-21B-30A-2A-1D-11-29A-7B-16C-30C-14A-19B-22C-1E-31-33B-17-14B-24-18-33A-29B-25-12A-8-3-30B-9A-26-16B-23B-2B-19-7A-36-5-35A-28-16D-35B-6A-6B-21A-13-20-34-4	10.800.734,48
...	...	...
G_{best}^{994}	4-1C-22B-27-10-12B-32A-23B-22C-2A-34B-23A-33B-33A-19-31-1E-7A-32B-24-16B-22A-3-35B-14B-9A-28-12A-29A-21B-35A-37-6A-25-8-7B-30A-26-16A-11-2B-19B-1D-13-5-34-6B-20-9B-15-17-16D-18-29B-30B-16C-30C-14A-21A-36-4	7.794.359,48
G_{best}^{995}	4-1C-22B-27-10-12B-32A-23B-22C-2A-34B-23A-33B-33A-19-31-1E-7A-32B-24-16B-22A-3-35B-14B-9A-28-12A-29A-21B-35A-37-6A-25-8-7B-30A-26-16A-11-2B-19B-1D-13-5-34-6B-20-9B-15-17-16D-18-29B-30B-16C-30C-14A-21A-36-4	7.794.359,48
G_{best}^{996}	4-1C-22B-27-10-12B-32A-23B-22C-2A-34B-23A-33B-33A-19-31-1E-7A-32B-24-16B-22A-3-35B-14B-9A-28-12A-29A-21B-35A-37-6A-25-8-7B-30A-26-16A-11-2B-19B-1D-13-5-34-6B-20-9B-15-17-16D-18-29B-30B-16C-30C-14A-21A-36-4	7.794.359,48
G_{best}^{997}	4-1C-22B-27-10-12B-32A-23B-22C-2A-34B-23A-33B-33A-19-31-1E-7A-32B-24-16B-22A-3-35B-14B-9A-28-12A-29A-21B-35A-37-6A-25-8-7B-30A-26-16A-11-2B-19B-1D-13-5-34-6B-20-9B-15-17-16D-18-29B-30B-16C-30C-14A-21A-36-4	7.794.359,48
G_{best}^{998}	4-1C-22B-27-10-12B-32A-23B-22C-2A-34B-23A-33B-33A-19-31-1E-7A-32B-24-16B-22A-3-35B-14B-9A-28-12A-29A-21B-35A-37-6A-25-8-7B-30A-26-16A-11-2B-19B-1D-13-5-34-6B-20-9B-15-17-16D-18-29B-30B-16C-30C-14A-21A-36-4	7.794.359,48

Tabel 4.24 Urutan *Scene* dan Total Biaya G_{best} pada Setiap Iterasi (Lanjutan)

G_{best}^i	Rute	Total Biaya Syuting (Rp)
G_{best}^{999}	4-1C-22B-27-10-12B-32A-23B-22C-2A-34B-23A-33B-33A-19-31-1E-7A-32B-24-16B-22A-3-35B-14B-9A-28-12A-29A-21B-35A-37-6A-25-8-7B-30A-26-16A-11-2B-19B-1D-13-5-34-6B-20-9B-15-17-16D-18-29B-30B-16C-30C-14A-21A-36-4	7.794.359,48
G_{best}^{1000}	4-1C-22B-27-10-12B-32A-23B-22C-2A-34B-23A-33B-33A-19-31-1E-7A-32B-24-16B-22A-3-35B-14B-9A-28-12A-29A-21B-35A-37-6A-25-8-7B-30A-26-16A-11-2B-19B-1D-13-5-34-6B-20-9B-15-17-16D-18-29B-30B-16C-30C-14A-21A-36-4	7.794.359,48

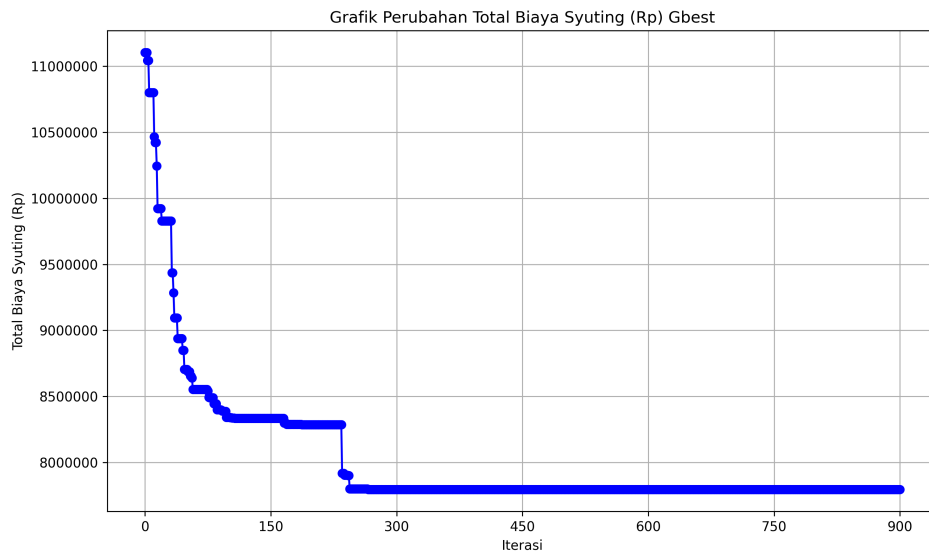
Proses perhitungan algoritma PSO untuk optimalisasi pemilihan urutan *scene* syuting film ”Ketika Adzan Sudah Tidak Lagi Berkumandang” menggunakan bantuan bahasa pemrograman Python diperoleh hasil nilai fungsi *fitness* yang dapat ditampilkan dalam grafik pada Gambar 4.6.

**Gambar 4.6** Grafik Perubahan Nilai Fungsi *Fitness* G_{best} pada Setiap Iterasi

Berdasarkan grafik pada Gambar 4.6 menunjukkan bahwa proses perhitungan algoritma PSO untuk optimalisasi pemilihan urutan *scene* syuting film ”Ketika Adzan Sudah Tidak Lagi Berkumandang” menghasilkan nilai fungsi *fitness* G_{best} yang semakin meningkat dan mencapai konvergen. G_{best} mencapai nilai fungsi *fitness* konvergen pada iterasi ke-266 sampai dengan iterasi ke-900. Nilai fungsi *fitness* konvergen yang dicapai sebesar 0,07442.

Hasil total biaya syuting dari G_{best} pada setiap iterasi dalam proses perhitungan algoritma PSO menggunakan bantuan bahasa pemrograman Python dapat dita-

mpilkan dalam grafik pada Gambar 4.7.



Gambar 4.7 Grafik Perubahan Total Biaya Syuting (Rp) G_{best} pada Setiap Iterasi

Berdasarkan grafik pada Gambar 4.7 menunjukkan bahwa proses perhitungan algoritma PSO untuk optimalisasi pemilihan urutan *scene* syuting film ”Ketika Adzan Sudah Tidak Lagi Berkumandang” menghasilkan total biaya syuting G_{best} yang semakin menurun dan mencapai konvergen. G_{best} mencapai total biaya syuting konvergen pada iterasi ke-266 sampai dengan iterasi ke-900. Total biaya syuting konvergen yang dicapai sebesar Rp7.794.359, 48.

4.3 Hasil Optimalisasi Pemilihan Urutan *Scene* Syuting Film ”Ketika Adzan Sudah Tidak Lagi Berkumandang” menggunakan Algoritma *Particle Swarm Optimization* (PSO)

Hasil optimalisasi pemilihan urutan *scene* syuting film ”Ketika Adzan Sudah Tidak Lagi Berkumandang” diperoleh melalui proses perhitungan algoritma *Particle Swarm Optimization* (PSO) yang telah dilakukan sebelumnya. Grafik perhitungan algoritma PSO pada Gambar 4.7 menunjukkan bahwa total biaya syuting konvergen diperoleh pada iterasi ke-266 sebesar Rp7.794.359, 48. Total biaya syuting tersebut diperoleh berdasarkan urutan indeks *scene* dari G_{best}^{266} pada Tabel 4.22 yaitu 0-1-35-42-15-18-50-38-36-4-55-37-53-52-29-49-3-10-51-39-24-34-6-57-21-13-43-17-44-33-56-59-8-40-12-11-46-41-23-16-5-30-2-19-7-54-9-31-14-22-27-26-28-45-47-25-48-20-32-58-0. Urutan indeks *scene* dari G_{best}^{266} kemudian diubah men-

jadi urutan *scene* dari G_{best}^{266} seperti pada Tabel 4.24 yaitu 4-1C-22B-27-10-12B-32A-23B-22C-2A-34B-23A-33B-33A-19-31-1E-7A-32B-24-16B-22A-3-35B-14B-9A-28-12A-29A-21B-35A-37-6A-25-8-7B-30A-26-16A-11-2B-19B-1D-13-5-34-6B-20-9B-15-17-16D-18-29B-30B-16C-30C-14A-21A-36-4.

Berdasarkan hasil yang diperoleh melalui proses perhitungan algoritma PSO, bentuk solusi optimal dalam model matematika konsep TSP untuk pemilihan urutan *scene* syuting film ”Ketika Adzan Sudah Tidak Lagi Berkumandang” sebagai berikut.

$$\begin{aligned}
 \min f(a_{j,k}) &= \sum_{i=0}^{59} \sum_{j=0}^{59} a_{j,k}(v_j v_k) \\
 &= (a_{0,1})(v_0 v_1) + (a_{1,35})(v_1 v_{35}) + (a_{35,42})(v_{35} v_{42}) + (a_{42,15})(v_{42} v_{15}) \\
 &\quad + (a_{15,18})(v_{15} v_{18}) + (a_{18,50})(v_{18} v_{50}) + (a_{50,38})(v_{50} v_{38}) + (a_{38,36})(v_{38} v_{36}) \\
 &\quad + (a_{36,4})(v_{36} v_4) + (a_{4,55})(v_4 v_{55}) + (a_{55,37})(v_{55} v_{37}) + (a_{37,53})(v_{37} v_{53}) + \dots \\
 &\quad + (a_{22,27})(v_{22} v_{27}) + (a_{27,26})(v_{27} v_{26}) + (a_{26,28})(v_{26} v_{28}) + (a_{28,45})(v_{28} v_{45}) \\
 &\quad + (a_{45,47})(v_{45} v_{47}) + (a_{47,25})(v_{47} v_{25}) + (a_{25,48})(v_{25} v_{48}) + (a_{48,20})(v_{48} v_{20}) \\
 &\quad + (a_{20,32})(v_{20} v_{32}) + (a_{32,58})(v_{32} v_{58}) + (a_{58,0})(v_{58} v_0) \\
 &= (1)(99817, 81) + (1)(0) + (1)(114027, 78) + (1)(154248, 37) + (1)(306359, 48) \\
 &\quad + (1)(79248, 37) + (1)(503183, 08) + (1)(183474, 75) + (1)(99817, 81) \\
 &\quad + (1)(64902, 78) + (1)(183474, 75) + (1)(63777, 78) + \dots + (1)(158581, 7) \\
 &\quad + (1)(90248, 37) + (1)(63777, 78) + (1)(158581, 7) + (1)(0) + (1)(79248, 37) \\
 &\quad + (1)(71111, 11) + (1)(0) + (1)(136331, 7) + (1)(252919, 19) + (1)(157004, 16) \\
 &\quad + (1)(99817, 81) \\
 &= \text{Rp } 7.794.359, 48
 \end{aligned}$$

BAB V

PENUTUP

5.1 Kesimpulan

Kesimpulan yang dapat diambil dari hasil penelitian Optimalisasi Pemilihan Urutan Syuting *Scene* Film Menggunakan Algoritma *Particle Swarm Optimization* (PSO) (Studi Kasus: Film "Ketika Adzan Sudah Tidak Lagi Berkumandang") adalah sebagai berikut:

1. Model graf dari pemilihan urutan syuting *scene* film "Ketika Adzan Sudah Tidak Lagi Berkumandang" merupakan graf lengkap yang dapat dilihat pada Gambar 4.2. Tiap *scene* dalam data *scene* dianalogikan sebagai *vertex* dan bobot biaya syuting antar *scene* sebagai *edge*. Model matematika pada pemilihan urutan syuting *scene* film "Ketika Adzan Sudah Tidak Lagi Berkumandang" dibentuk menggunakan konsep *traveling salesman problem* (TSP) adalah sebagai berikut.

Fungsi tujuan

$$\min f(a_{j,k}) = \sum_{i=0}^{59} \sum_{j=0}^{59} a_{j,k} (v_j v_k) \quad (5.1)$$

Fungsi kendala

$$\begin{aligned} \sum_{j=0}^{59} a_{j,k} &= 1, \quad k = 0, 1, \dots, 59 \\ \sum_{k=0}^{59} a_{j,k} &= 1, \quad j = 0, 1, \dots, 59 \\ a_{j,k} &\in \{0, 1\}, \quad j \neq k \end{aligned}$$

Dimana $v_j v_k$ merupakan elemen matriks biaya antar *scene* dari *scene* ke- j menuju *scene* ke- k sesuai Tabel 4.6 dan $a_{j,k}$ merupakan elemen matriks biner solusi dari TSP seperti pada Tabel 4.7.

2. Algoritma *particle swarm optimization* (PSO) untuk optimalisasi pemilihan urutan syuting *scene* film "Ketika Adzan Sudah Tidak Lagi Berkumandang" yang diterapkan meliputi proses normalisasi data biaya antar *scene*, proses algoritma PSO, pengujian modifikasi parameter PSO, dan analisis hasil perhitungan algoritma PSO. Proses perhitungan algoritma PSO menggunakan nilai parameter

terbaik yang diperoleh dari hasil pengujian modifikasi nilai parameter PSO. Nilai parameter terbaik yang dipakai yaitu iterasi maksimum (n) sebesar 900, ukuran *swarm* (N) sebesar 250, dan faktor *clamping* (ε) pada *velocity* maksimum sebesar 0.3. Berdasarkan nilai parameter terbaik diperoleh hasil G_{best} konvergen pada iterasi ke-226 sampai dengan iterasi ke-900 dengan nilai fungsi *fitness* konvergen yang dicapai sebesar 0,07442.

3. Hasil optimalisasi pemilihan urutan syuting *scene* film "Ketika Adzan Sudah Tidak Lagi Berkumandang" menggunakan algoritma (PSO) memperoleh urutan syuting *scene* yang optimal. Urutan syuting *scene* yang diperoleh dimulai dari *scene* 4-1C-22B-27-10-12B-32A-23B-22C-2A-34B-23A-33B-33A-19-31-1E-7A-32B-24-16B-22A-3-35B-14B-9A-28-12A-29A-21B-35A-37-6A-25-8-7B-30A-26-16A-11-2B-19B-1D-13-5-34-6B-20-9B-15-17-16D-18-29B-30B-16C-30C-14A-21A-36-4. Total biaya syuting yang diperoleh dengan urutan syuting *scene* tersebut sebesar Rp7.794.359,48.

5.2 Saran

Adapun saran pada penelitian ini sebagai berikut.

1. Penggunaan konsep *traveling salesman problem* (TSP) dapat memberikan hasil lebih akurat jika variabel yang digunakan lebih kompleks atau lebih spesifik, misalnya mempertimbangkan variabel biaya properti tiap *scene*, waktu perjalanan antar lokasi *scene*, durasi syuting, atau batasan lainnya.
2. Penggunaan algoritma *particle swarm optimization* (PSO) yang telah dimodifikasi dan terbaru seperti algoritma *Multi-Objective* PSO dapat memberikan solusi optimal dengan model yang lebih kompleks.

DAFTAR PUSTAKA

- Akhirina, T. Y., & Afrizal, T. (2020). Pendekatan Matriks Ketetangaan Berbobot Untuk Solusi *Minimum Spanning Tree* (MST). *Satuan Tulisan Riset dan Inovasi Teknologi*, 4(3) (2020), 280–287.
- Akpudo, U. E., & Jang-Wook, H. (2020). A Multi-Domain Diagnostics Approach for Solenoid Pumps Based on Discriminative Features. *IEEE Access*, 8 (2020), 175020–175034.
- Ambarwari, A., Adrian, Q. J., & Herdiyani, Y. (2019). Analisis pengaruh data *Scaling* terhadap performa algoritme *Machine Learning* untuk identifikasi tanaman. *JURNAL RESTI: Jurnal Rekayasa Sistem dan Teknologi Informasi*, 4(1) (2019), 117–122.
- Amin, N. F., Garancang., S., & Abunawas, K. (2023). Konsep umum populasi dan sampel dalam penelitian. *Jurnal Kajian Islam Kontemporer (Jurnal PILAR)*, 14(1) (2023), 15–31.
- Anwar, M., Pratamaningtyas., S., Rosita., Deni., A., Lusono., A., Hermayanti., Ulfa., A. K., Wibowo., M. E. S., Fatmawati., E. R., & Chasanah, A. N. (2024). *Pengambilan keputusan*. Yayasan Cendikia Mulia Mandiri.
- Atiqoh, A. N. (2020). *Analisis inertia weight pada algoritma particle swarm optimization (pso) untuk optimalisasi dan pemodelan sistem terhadap persoalan vehicle routing problem with time window (vrptw) (skripsi)*. UIN Sunan Ampel Surabaya.
- Azhari, M. K., Cholissodin, I., & Bachtiar., F. A. (2018). Optimasi *Travel Salesman Problem* pada angkutan sekolah dengan algoritma *Particle Swarm Optimization*. *Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer*, 2(11) (2018), 5691–5699.
- Bisma, M. A., & Sanggala, E. (2023). *Genetic Algorithm* untuk memperbaiki rute *Travelling Salesman Problem* yang dihasilkan dari *Saving Algorithm*. *SAIN-TEKS: Jurnal Sain dan Teknik*, 5(1) (2023), 102–111.
- Chafi, Z. S., & Afrakhte., H. (2021). *Short-Term Load Forecasting Using Neural Network and Particle Swarm Optimization (pso) Algorithm*. *Mathematical Problems in Engineering*, 2021(1) (2021), 5598267.

- Chen, F., Wang., Y.-C., Wang., B., & C.-C. Jay Kuo. (2020). *Graph Representation Learning: a Survey*. *APSIPA Transactions on Signal and Information Processing*, 9(e15) (2020), 1–21.
- Dalyono, Y. P., Herdiani, A., & Rohmawati., A. A. (2017). Penentuan rute pariwisata kota bandung menggunakan algoritma *Particle Swarm Optimization*. *eProcidings of Engineering*, 4(3) (2017), 4811–4817.
- Darmawan, R., Indra., & Surahmat, A. (2022). Optimalisasi *Support Vector Machine* (svm) berbasis *Particle Swarm Optimization* (pso) pada analisis sentimen terhadap *Official Account* ruang guru di twitter. *Jurnal Kajian Ilmiah*, 22(2) (2022), 143–152s.
- Devita, R. N., & Wibawa, A. P. (2020). Teknik-teknik optimasi *Knapsack Problem*. *Sains, Aplikasi, Komputasi dan Teknologi Informasi (SAKTI)*, 2(1) (2020), 35–40.
- Eltamaly, A. M. (2021). *A Novel Strategy for Optimal PSO Control Parameters Determination for PV Energy Systems*. *Sustainability*, 13(2) (2021), 1008.
- Gad, A. (2022). *Particle Swarm Optimization Algorithm and Its Applications: A Systematic Review*. *Achives of Computational Methods in Engineering*, 29 (2022), 2531–2561.
- Haren, S. M. (2020). Model manajemen produksi film pendek cerita masa tua. *Jurnal Audiens*, 1(1) (2020), 107–112.
- Jain, M., Vibha, S., Narinder, S., & Satya, B. S. (2022). *An Overview of Variants and Advancements of PSO Algorithm*. *Applied Science*, 12(17) (2022), 8392.
- Jek, J. (2016). *Matematika diskrit dan aplikasinya pada ilmu komputer*.
- Kusrahman, N. Y., Purnamasari., I., & Amijaya, F. D. T. (2020). Optimasi *Self-Organizing Map* menggunakan *Particle Swarm Optimization* untuk mengelompokkan desa/kelurahan tertinggal di kabupaten kutai kertanegara provinsi kalimantan timur (studi kasus: Data potensi desa tahun 2018). *Jurnal EKSPONENSIAL*, 11(2) (2020), 139–144.
- Levin, O. (2021). *Discrete mathematics: An open introduction*.
- Muhammad., Febrianty., & Sentanu, I. G. E. P. S. (2023). *Manajemen pengambilan keputusan*. Perkumpulan Rumah Cemerlang Indonesia.

- Muhardeny, M., Irfani., M. H., & Alie, J. (2023). Penjadwalan mata pelajaran menggunakan algoritma *Particle Swarm Optimization* (psa) pada smpit mufidatul ilmi. *Journal Software Engineering and Computational Intelligence (JSECI)*, 1(1) (2023), 51–63.
- Nasir, A. M., Faisal, & Setiawan, D. (2022). Optimalisasi penjadwalan mata kuliah menggunakan teori pewarnaan graf. *Jurnal Penelitian Matematika dan Pendidikan Matematika (PROXIMAL)*, 5(1) (2022), 57–69.
- Natalia, D., Yundari, & Yudhi. (2019). Optimasi jarak penjemputan penumpang cv. eira saudara menggunakan metode *Particle Swarm Optimization*. *Buletin Ilmiah Math, Stat, dan Terapannya (BIMASTER)*, 8(3) (2019), 531–538.
- Nurdiyanto, T., & Susanti, E. (2019). Efisiensi penggunaan matriks *In-Degree* untuk mengkontruksi *Spanning-Tree* pada graf berarah. *Jurnal Edukasi dan Sains Matematika (JES-MAT)*, 5(1) (2019), 1–15.
- Piotrowski, A. P., Napiorkowski., J. J., & Piotrowska, A. E. (2020). *Population Size in Particle Swarm Optimization*. *Swarm and Evolutionary Computation*, 58 (2020), 100718.
- Pratama, V. L., & Darmawan, D. A. (2022). Sistem informasi geografis pencarian rute terdekat bengkel motor di kota surabaya menggunakan algoritma *Bellman-Ford*. *Journal of Informatics and Computer Science (JINACS)*, 3(4) (2022), 580–599.
- Qin, H., Zhang, Z., Lim, A., & Liang., X. (2016). *An Enhanced Branch-and-Bound Algorithm for The Talent Schedulling Problem*. *European Journal of Operational Research*, 250(2) (2016), 412–426.
- Rahayuningsih, S. (2018). *Teori graph dan penerapannya*. Universitas Wisnuwardhana Press Malang (Unidha Press).
- Rahimi, A., Arthur, M., Nurfadillah, Amijaya, F. D. T., & Putri., D. F. (2023). Implementasi algoritma genetika dalam penentuan rute terbaik pendistribusian bbm pada spbu yang ada di samarinda. *Prosiding Seminar Nasional Matematika, Statistika, dan Aplikasinya (SNMSA)*, 3(1) (2023), 196–207.
- Ramadhan, M. R., Setianingsih., C., & Dirgantoro., B. (2023). Sistem penjadwalan perangkat listrik dengan metode algoritma *Particle Swarm Optimization* berbasis Website. *eProceedings of Engineering*, 10(1) (2023), 865–872.

- Sari, F. F., Pujiarti., T., Hidayat., & Anjosa. (2024). Pembelajaran matematika diskrit mengacu pada teori beban kognitif (*Cognitive Load Theory*). *Jurnal Kajian Pendidikan dan Sosial (DIKSI)*, 5(1) (2024), 10–17.
- Saud, S., Kodaz., H., & Babaoğlu, I. (2018). *Solving Traveling Salesman Problem Using Optimization Algorithms*. *KnE Social Sciences*, 3(1) (2018), 17–32.
- Setiawati, J., Febrilian, M., & Ekawati, D. (2023). Pelabelan total tak reguler sisi pada graf *Direction*, graf *Direction Right*, dan graf *Heart*. *Jurnal Penelitian Matematika dan Pendidikan Matematika (PROXIMAL)*, 6(2) (2023), 64–72.
- Sinaga, R. P., & Marpaung, F. (2023). Perbandingan algoritma *Cheapest Insertion Heuristic* dan *Nearest Neighbor* dalam menyelesaikan *Traveling Salesman Problem*. *Jurnal Riset Rumpun Matematika dan Ilmu Pengetahuan Alam (JURRIMIPA)*, 2(2) (2023), 238–247.
- Wang, Z., Ding., H., Wang., J., Hou., P., Li., A., Yang., Z., & Hu., X. (2022). *Adaptive Guided Salp Swarm Algorithm with Velocity Clamping Mechanism for Solving Optimization Problems*. *Journal of Computational Design and Engineering (CDE)*, Oxford, 9(6) (2022), 2196–2234.
- Yurinanda, S., & Rozi, S. (2023). Penerapan pembelajaran berbasis proyek pada mata kuliah matematika diskrit untuk meningkatkan keterampilan mahasiswa dalam memanfaatkan struktur diskrit dalam menyelesaikan masalah. *Jurnal Pendidikan Matematika dan Matematika (ABSIS)*, 5(2) (2023), 666–679.
- Zahro, H. Z., & Wahyuni, F. S. (2020). Optimasi *Particle Swarm Optimization* (pso) untuk penentuan *Base Transceiver System* (bts). *Jurnal MNEMONIC*, 3(1) (2020), 7–10.
- Zdiri, S., Chroua., J., & Zaafouri., A. (2021). *An Expanded Heterogeneous Particle Swarm Optimization Based on Adaptive Inertia Weight*. *Mathematical Problems in Engineering, Hindawi*, 2021(1) (2021), 4194263.

LAMPIRAN

Lampiran 1 Scenes_Data df

Lampiran 2 Daftar Total Biaya *Talent*

Peran	<i>Talent</i>	Total Biaya (Rp)	Total Scene
Wawan	Daniel Imran	1250000	45
Manto	A. Muslih Navis	900000	34
Usman	KimKim	900000	25
Rita	Emilda Sohay	400000	16
Nina	Aqila Yumi	650000	15
Ustadz Udas	Dedi Nala Arung	400000	17
Irah	Ika Puspita	400000	9
Mali	Nanda Fajar Amrullah	400000	12
Ismail	Muhtadin	400000	11
Kakek Tua	Muhammad Syabir	300000	3

Lampiran 3 Contoh

Lampiran 4 Contoh

--

--

Lampiran 5 Contoh

--

--

Lampiran 6 Kecepatan Awal

Lampiran 7 Posisi Awal

Lampiran 8 Rute Awal

Lampiran 9 Total Biaya Syuting dan Nilai *Fitness* tiap Partikel

Lampiran 10 *Velocity* Partikel Iterasi $i = 1$

Lampiran 11 Posisi Partikel Iterasi $i = 1$

Lampiran 12 Rute pada Partikel Iterasi $i = 1$

Lampiran 13 Total Biaya Syuting dan Nilai *Fitness* Iterasi $i = 1$

Lampiran 14 P_{best} Iterasi $i = 1$

Lampiran 15 G_{best} Seluruh Iterasi

Lampiran 16 Rute G_{best} Pada Setiap Iterasi

Lampiran 17 Total Biaya Syuting G_{best} pada Setiap Iterasi